

# Defensa completa SOS2526-29

Documento canonico del proyecto para la defensa. Es la fuente principal para estudiar, modificar y explicar el proyecto; el `README.md` queda como entrada breve al repositorio.

Sustituye toda la documentacion de defensa que antes estaba repartida en varias guias. Esas guias antiguas ya no se conservan como fuentes independientes dentro de `docs/`.

Objetivo: que una persona que no conoce el repositorio pueda entender que hace el proyecto, como se instala, como se ejecuta, como esta organizado, que rutas existen, que funciones importan, como fluyen los datos y como defenderlo ante un profesor o tribunal.

Nota de nombre: el archivo real del repositorio es `docs/DEFENSA_COMPLETA.md`. Cuando en conversaciones o planes aparezca `DEFENSA_COMPLETA.md`, se refiere a este documento.

Copia para leer o imprimir en PDF: [DEFENSA\\_COMPLETA.pdf](#).

---

## Indice

- 1. Resumen ejecutivo
- 2. Problema, objetivos y publico
- 3. Equipo y recursos
- 4. Stack tecnologico
- 5. Arquitectura general
- 6. Estructura real de carpetas
- 7. Instalacion, configuracion y ejecucion
- 8. Scripts, tests y validaciones
- 9. Rutas utiles para abrir
- 10. Modelo de datos
- 11. Backend: arranque y responsabilidades
- 12. APIs REST del proyecto
- 13. Frontend: rutas, servicios y pantallas
- 14. Parte LCC: `citys-stats`
- 15. Integraciones LCC
- 16. Funciones importantes explicadas
- 17. Flujos de ejecucion
- 18. Codigos HTTP y contrato REST
- 19. Cambios que pueden pedir en directo
- 20. Errores comunes y soluciones
- 21. Decisiones tecnicas
- 22. Limitaciones y mejoras futuras
- 23. Guion de defensa oral

- 24. Preguntas dificiles y respuestas
- 25. Glosario para personas no tecnicas
- 26. Checklists finales
- 27. Anexos tecnicos

---

## 1. Resumen ejecutivo

**SOS2526-29** es una aplicacion web de la asignatura SOS2526. Tiene backend y frontend y permite gestionar, consultar, visualizar e integrar tres recursos de datos:

- **natural-disasters** : desastres naturales por pais y anio.
- **citys-stats** : poblacion estimada de ciudades para 2025.
- **wine-stats** : informacion de vinos, precios, tipos y caracteristicas.

La aplicacion permite:

- Listar datos.
- Buscar y filtrar datos.
- Crear registros.
- Editar registros en pantallas separadas.
- Borrar registros concretos.
- Borrar colecciones completas.
- Cargar datos iniciales.
- Mostrar graficas con Highcharts.
- Mostrar mapas.
- Consumir integraciones externas mediante proxy propio.

Frase corta de defensa:

Nuestro proyecto es una aplicacion web con Node.js, Express, NeDB y Svelte. El backend ofrece una API REST para tres recursos: **natural-disasters**, **citys-stats** y **wine-stats**. El frontend consume esas APIs con **fetch**, muestra CRUDs, graficas, mapas e integraciones. Los datos se guardan en archivos NeDB y las APIs externas se consumen desde el backend para controlar errores y normalizar las respuestas.

---

## 2. Problema, objetivos y publico

### Problema que resuelve

El proyecto organiza varias fuentes de datos heterogeneas en una aplicacion unica. En vez de consultar JSON crudo o APIs separadas, el usuario puede navegar por pantallas, ver tablas, cargar datos, hacer operaciones CRUD y entender los datos mediante visualizaciones.

## Objetivos funcionales

- Exponer una API REST para cada recurso.
- Mantener versiones de API cuando un recurso evoluciona.
- Persistir datos localmente con NeDB.
- Ofrecer una interfaz web usable con Svelte.
- Validar entradas para evitar datos incompletos o inconsistentes.
- Probar el contrato de API con Newman.
- Probar flujos de usuario con Playwright.
- Mostrar visualizaciones individuales y grupales.
- Integrar APIs externas sin que el navegador dependa directamente de ellas.

## Objetivos de defensa

- Saber explicar el arranque completo desde `npm start`.
- Saber localizar cada capa: servidor, API, servicio frontend y pantalla.
- Saber justificar REST, versionado, codigos HTTP, validaciones y proxy.
- Saber hacer cambios pequenos en directo sin perderse.
- Saber responder por que `citys-stats` usa v2 para CRUD y v1 para integraciones.

## Publico objetivo

- Profesor o evaluador de SOS2526.
- Companeros del grupo.
- Cualquier persona tecnica que quiera ejecutar o revisar el proyecto.
- Cualquier stakeholder no tecnico que quiera entender el resultado final.

## 3. Equipo y recursos

| Miembro               | Recurso           | API principal             | Frontend           |
|-----------------------|-------------------|---------------------------|--------------------|
| Rufino Moreno Pacheco | wine-stats        | /api/v1/wine-stats        | /wine-stats        |
| Luis Cortes Cobos     | citys-stats       | /api/v2/citys-stats       | /citys-stats       |
| Alberto Lirola Gomez  | natural-disasters | /api/v2/natural-disasters | /natural-disasters |

Repositorio:

<https://github.com/gti-sos/SOS2526-29>

Aplicacion desplegada:

<https://sos2526-29.onrender.com/>

## 4. Stack tecnologico

| Tecnologia                              | Donde aparece                                      | Para que se usa                            |
|-----------------------------------------|----------------------------------------------------|--------------------------------------------|
| Node.js                                 | <code>index.js</code> , scripts npm                | Ejecutar el backend                        |
| Express                                 | <code>index.js</code> , <code>src/back/**</code>   | Definir servidor y rutas REST              |
| <code>@seald-io/nedb</code>             | <code>index.js</code> , <code>src/back/*.db</code> | Persistencia local en archivos             |
| CORS                                    | <code>index.js</code>                              | Permitir llamadas desde Vite en desarrollo |
| Svelte                                  | <code>frontend-group/src</code>                    | Construir la interfaz                      |
| Vite                                    | <code>frontend-group/vite.config.js</code>         | Desarrollo y build del frontend            |
| Highcharts                              | analytics e integraciones                          | Graficas, mapas y widgets                  |
| <code>@highcharts/map-collection</code> | mapa de ciudades                                   | TopoJSON del mapa mundial                  |
| Newman                                  | <code>tests/**/*.json</code>                       | Ejecutar colecciones Postman               |
| Playwright                              | <code>tests/*/e2e</code>                           | Pruebas end-to-end                         |
| <code>start-server-and-test</code>      | <code>package.json</code>                          | Arrancar servidor y lanzar tests           |

Dependencias principales en `package.json` :

```
express
@seald-io/nedb
cors
svelte
highcharts
@highcharts/map-collection
leaflet
cool-ascii-faces
```

Dependencias del frontend en `frontend-group/package.json` :

```
@sveltejs/vite-plugin-svelte
vite
svelte
svelte-spa-router
highcharts
@highcharts/map-collection
```

Nota: aunque aparece `svelte-spa-router` , el enrutado actual no usa rutas `#/` : `App.svelte` descubre pantallas con `import.meta.glob` y resuelve `window.location.pathname` ; `navigation.js` usa `history.pushState` para navegar dentro de la SPA.

## 5. Arquitectura general

El proyecto sigue una arquitectura por capas:

```
Navegador
|
| HTML, CSS, JS compilado
v
Frontend Svelte en public/
|
| fetch HTTP JSON
v
Backend Express en index.js
|
| registra modulos REST
v
src/back/v1 y src/back/v2
|
| consultas y modificaciones
v
Bases NeDB en src/back/*.db

Integraciones externas:

Frontend Svelte -> API propia /api/v1/citys-stats/integrations/... -> fetch desde Express -> APIs externas JSON
```

Separacion principal:

- `index.js`: arranque, middleware, bases de datos, registro de APIs, frontend estatico y fallback SPA.
- `src/back/v1` y `src/back/v2`: contratos REST por recurso y version.
- `frontend-group/src/services`: funciones `fetch` que encapsulan URLs y errores.
- `frontend-group/src/routes`: pantallas visibles, organizadas por carpetas con `+page.svelte`.
- `public`: build final del frontend que sirve Express en produccion.
- `tests`: validaciones API y navegador.

Frase de defensa:

La aplicacion separa responsabilidades: Svelte solo pinta y llama a servicios, Express valida y coordina, NeDB persiste, y las integraciones externas pasan por nuestro backend para evitar problemas de origen y controlar errores.

## 6. Estructura real de carpetas

Estructura resumida:

```
SOS2526-29/
|-- index.js
|-- package.json
```

```
|-- package-lock.json
|-- README.md
|-- docs/
|   |-- DEFENSA_COMPLETA.md
|   |-- DEFENSA_COMPLETA.pdf
|-- src/
|   |-- back/
|       |-- v1/
|           |-- citys-stats.js
|           |-- natural-disasters.js
|           |-- wine-stats.js
|       |-- v2/
|           |-- citys-stats.js
|           |-- natural-disasters.js
|           |-- citys-stats.db
|           |-- natural-disasters.db
|           |-- wine-stats.db
|-- frontend-group/
|   |-- package.json
|   |-- vite.config.js
|   |-- src/
|       |-- App.svelte
|       |-- main.js
|       |-- app.css
|       |-- components/
|           |-- Navbar.svelte
|       |-- lib/
|           |-- navigation.js
|       |-- services/
|           |-- apiBase.js
|           |-- citysStatsApi.js
|           |-- citysStatsIntegrations.js
|           |-- natural-disasters.js
|           |-- wine-stats.js
|       |-- routes/
|           |-- +page.svelte
|           |-- citys-stats/
|           |-- natural-disasters/
|           |-- wine-stats/
|           |-- analytics/
|           |-- integrations/
|-- public/
|-- tests/
|   |-- ALG/
|   |-- LCC/
|   |-- RMP/
|-- playwright.config.js
```

Archivos clave:

| Archivo                                                                                 | Importancia                                    |
|-----------------------------------------------------------------------------------------|------------------------------------------------|
| <code>index.js</code>                                                                   | Punto de entrada del backend y servidor unico  |
| <code>src/back/v2/citys-stats.js</code>                                                 | API principal del CRUD LCC                     |
| <code>src/back/v1/citys-stats.js</code>                                                 | API v1 LCC, agregados por pais e integraciones |
| <code>src/back/v2/natural-disasters.js</code>                                           | API principal de desastres naturales           |
| <code>src/back/v1/wine-stats.js</code>                                                  | API principal de vinos                         |
| <code>frontend-group/src/App.svelte</code>                                              | Router real de la SPA                          |
| <code>frontend-group/src/components/Navbar.svelte</code>                                | Menu superior                                  |
| <code>frontend-group/src/routes/+page.svelte</code>                                     | Portada                                        |
| <code>frontend-group/src/routes/citys-stats/+page.svelte</code>                         | CRUD LCC                                       |
| <code>frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte</code> | Edicion LCC                                    |
| <code>frontend-group/src/routes/analytics/+page.svelte</code>                           | Widget grupal                                  |
| <code>frontend-group/src/routes/analytics/citys-stats/+page.svelte</code>               | Grafico individual LCC                         |
| <code>frontend-group/src/routes/analytics/citys-stats/map/+page.svelte</code>           | Mapa LCC                                       |
| <code>frontend-group/src/routes/integrations/citys-stats/+page.svelte</code>            | Integraciones LCC                              |
| <code>frontend-group/src/services/citysStatsApi.js</code>                               | Fetch CRUD LCC v2                              |
| <code>frontend-group/src/services/citysStatsIntegrations.js</code>                      | Fetch integraciones LCC v1                     |
| <code>frontend-group/vite.config.js</code>                                              | Build hacia <code>../public</code>             |

Rutas alias:

- Existen carpetas `analytics/city-stats`, `analytics/city-stats/map` e `integrations/city-stats`.
- Esas rutas importan y reutilizan las pantallas reales de `citys-stats`.
- La ruta canonica para defensa es `citys-stats`, porque ese es el nombre del recurso publicado.

## 7. Instalacion, configuracion y ejecucion

### Requisitos

- Node.js instalado.
- npm instalado.
- Acceso a internet si se van a probar integraciones externas.

## Instalacion desde cero

En la raiz del repositorio:

```
npm.cmd install  
npm.cmd --prefix frontend-group install
```

Tambien funciona:

```
npm install  
npm --prefix frontend-group install
```

En PowerShell se recomienda `npm.cmd` si aparece el error de `npm.ps1`.

## Build del frontend

```
npm.cmd run build
```

Que hace:

1. Entra en `frontend-group`.
2. Ejecuta `vite build`.
3. Genera el frontend compilado en `public`.
4. Borra previamente `public` por `emptyOutDir: true`.

## Arranque local

```
npm.cmd start
```

Equivale a:

```
node index.js
```

URL local:

```
http://localhost:10000
```

El puerto por defecto esta en `index.js`:

```
const port = process.env.PORT || 10000;
```

En Render manda `process.env.PORT`; en local, si no se define, usa `10000`.

## Desarrollo frontend con Vite

```
npm.cmd run dev-front
```



Ese script ejecuta:

```
cd frontend-group && npm run dev -- --open
```

Durante desarrollo, `frontend-group/vite.config.js` tiene proxy:

```
/api -> http://localhost:10000
```

Ademas, `apiBase.js` fuerza las llamadas API a `http://localhost:10000` cuando `import.meta.env.DEV` es verdadero.

Variables de entorno reconocidas

| Variable        | Uso                                                         |
|-----------------|-------------------------------------------------------------|
| PORT            | Puerto del servidor Express                                 |
| LCC_DOCS_URL    | URL de documentacion Postman v1 de <code>citys-stats</code> |
| LCC_DOCS_V2_URL | URL de documentacion Postman v2 de <code>citys-stats</code> |
| RMP_DOCS_URL    | URL de documentacion Postman de <code>wine-stats</code>     |

8. Scripts, tests y validaciones

Scripts principales de `package.json`:

| Script                                      | Comando real                                                    | Uso                                        |
|---------------------------------------------|-----------------------------------------------------------------|--------------------------------------------|
| <code>npm start</code>                      | <code>node index.js</code>                                      | Arrancar servidor                          |
| <code>npm run build</code>                  | <code>cd frontend-group &amp;&amp; npm run build</code>         | Compilar Svelte a <code>public</code>      |
| <code>npm run dev-front</code>              | <code>cd frontend-group &amp;&amp; npm run dev -- --open</code> | Desarrollo frontend                        |
| <code>npm run test-natural-disasters</code> | Newman ALG local                                                | API desastres                              |
| <code>npm run test-citys-stats</code>       | Newman LCC v1 local                                             | API ciudades v1                            |
| <code>npm run test-citys-stats-v2</code>    | Newman LCC v2 local                                             | API ciudades v2                            |
| <code>npm run test-wine-stats</code>        | Newman RMP local                                                | API vinos                                  |
| <code>npm run test-ALG</code>               | Arranca servidor y ejecuta Newman ALG                           | Validacion completa ALG                    |
| <code>npm run test-LCC</code>               | Arranca servidor y ejecuta Newman LCC v1                        | Validacion LCC v1                          |
| <code>npm run test-LCC-v2</code>            | Arranca servidor y ejecuta Newman LCC v2                        | Validacion LCC v2                          |
| <code>npm run test-LCC-e2e</code>           | Playwright con<br><code>tests/LCC/playwright.config.js</code>   | Flujo navegador LCC                        |
| <code>npm run test-RMP</code>               | Arranca servidor y ejecuta Newman RMP                           | Validacion RMP                             |
| <code>npm test</code>                       | <code>test-ALG &amp;&amp; test-RMP &amp;&amp; test-LCC</code>   | Suite por defecto: ALG v2, RMP v1 y LCC v1 |

Comandos recomendados antes de defender:

```
npm.cmd run build
npm.cmd start
```

En otra terminal, si se quiere validar:

```
npm.cmd run test-LCC-v2
npm.cmd run test-LCC-e2e
```

Tests LCC e2e cubren:

- Portada con tarjeta de Luis y enlaces LCC.
- Listado de `citys-stats`.
- Creacion de ciudad.
- Borrado individual.
- Borrado total.
- Edicion en `/citys-stats/editar/:city/:country`.
- Búsqueda con filtros, orden y limite.

## 9. Rutas utiles para abrir

### Local

```
http://localhost:10000/
http://localhost:10000/citys-stats
http://localhost:10000/natural-disasters
http://localhost:10000/wine-stats
http://localhost:10000/analytics
http://localhost:10000/analytics/citys-stats
http://localhost:10000/analytics/citys-stats/map
http://localhost:10000/integrations
http://localhost:10000/integrations/citys-stats
http://localhost:10000/api/v2/citys-stats
http://localhost:10000/api/v1/citys-stats
http://localhost:10000/api/v2/natural-disasters
http://localhost:10000/api/v1/natural-disasters
http://localhost:10000/api/v1/wine-stats
```

### Desplegado

```
https://sos2526-29.onrender.com/
https://sos2526-29.onrender.com/citys-stats
https://sos2526-29.onrender.com/analytics
https://sos2526-29.onrender.com/analytics/citys-stats
https://sos2526-29.onrender.com/analytics/citys-stats/map
https://sos2526-29.onrender.com/integrations
https://sos2526-29.onrender.com/integrations/citys-stats
https://sos2526-29.onrender.com/api/v2/citys-stats/docs
```

Orden recomendado para la defensa LCC:

| Orden | Ruta                       | Que demostrar                                      |
|-------|----------------------------|----------------------------------------------------|
| 1     | /                          | Portada, miembros, enlaces de APIs y documentacion |
| 2     | /api/v2/citys-stats/docs   | Documentacion Postman de API v2                    |
| 3     | /citys-stats               | CRUD completo, filtros, ordenacion y paginacion    |
| 4     | /analytics/citys-stats     | Highcharts individual no lineal                    |
| 5     | /analytics/citys-stats/map | Mapa geoespacial                                   |
| 6     | /integrations              | Entrada comun de integraciones                     |
| 7     | /integrations/citys-stats  | 7 integraciones por pais con proxy                 |
| 8     | /analytics                 | Widget grupal unico                                |

Comprobaciones rapidas con PowerShell:

```
Invoke-WebRequest http://localhost:10000/ -UseBasicParsing
Invoke-WebRequest http://localhost:10000/citys-stats -UseBasicParsing
Invoke-WebRequest http://localhost:10000/analytics -UseBasicParsing
Invoke-WebRequest http://localhost:10000/integrations/citys-stats -UseBasicParsing
Invoke-WebRequest "http://localhost:10000/api/v1/citys-stats/integrations/summary?limit=8" -UseBasicParsing
```

## 10. Modelo de datos

### natural-disasters

Ejemplo:

```
{
  "country": "spain",
  "year": 2024,
  "death_count": 220,
  "injured_count": 500,
  "economic_damage_usd": 30000
}
```

Clave del recurso:

```
country + year
```

Campos:

| Campo               | Tipo esperado | Significado            |
|---------------------|---------------|------------------------|
| country             | texto         | Pais                   |
| year                | numero        | Anio                   |
| death_count         | numero        | Muertes registradas    |
| injured_count       | numero        | Heridos registrados    |
| economic_damage_usd | numero        | Danio economico en USD |

### citys-stats

Ejemplo:

```
{
  "city": "tokyo",
  "country": "japan",
  "un_2025_population": 33412512
}
```

Clave del recurso:

city + country

Campos:

| Campo              | Tipo esperado   | Significado                                      |
|--------------------|-----------------|--------------------------------------------------|
| city               | texto           | Ciudad                                           |
| country            | texto           | Pais                                             |
| un_2025_population | entero positivo | Poblacion estimada por Naciones Unidas para 2025 |

Datos iniciales principales:

jakarta, dhaka, tokyo, delhi, shanghai, guangzhou, cairo, manila, kolkata, seoul, karachi, mumbai

wine-stats

Ejemplo para crear o actualizar:

```
{
  "title": "The Guv'nor, Spain",
  "country": "spain",
  "region": "",
  "year": 2026,
  "price": 9.99,
  "abv": 14,
  "unit": 105,
  "grape": "Tempranillo",
  "type": "Red",
  "capacity": 75
}
```

Campo generado por backend:

```
{
  "id": 1
}
```

Clave del recurso:

id

Duplicado:

title + year

# 11. Backend: arranque y responsabilidades

Archivo principal:

```
index.js
```

Responsabilidades:

1. Importa Express, path, NeDB y CORS.
2. Crea `app`.
3. Define `port`.
4. Activa `cors()`.
5. Activa `express.json()` para leer cuerpos JSON.
6. Crea tres bases NeDB:

- `naturalDisastersDb`
- `citysStatsDb`
- `wineStatsDb`

1. Carga y registra modulos de API:

- `src/back/v1/natural-disasters.js`
- `src/back/v2/natural-disasters.js`
- `src/back/v1/citys-stats.js`
- `src/back/v2/citys-stats.js`
- `src/back/v1/wine-stats.js`

1. Sirve el frontend compilado desde `public`.
2. Define `/` y `/about` para devolver el frontend. En la SPA actual, `/about` cae en la portada porque no hay una pantalla `routes/about`.
3. Define proxy `/api/proxy/exportations-stats`.
4. Define fallback para toda ruta que no empiece por `/api`.
5. Lanza `app.listen`.

Frase de defensa:

`index.js` es el punto donde se juntan las capas. Crea el servidor, abre las bases, registra las APIs REST y sirve la SPA de Svelte desde `public`.

## Bases de datos

| Variable                        | Archivo                                    |
|---------------------------------|--------------------------------------------|
| <code>naturalDisastersDb</code> | <code>src/back/natural-disasters.db</code> |
| <code>citysStatsDb</code>       | <code>src/back/citys-stats.db</code>       |
| <code>wineStatsDb</code>        | <code>src/back/wine-stats.db</code>        |

Todas usan:

```
autoload: true
```

Eso abre el archivo al arrancar.

## Fallback de la SPA

Esta ruta es clave:

```
app.get(/^\/(?!api\/).*/, (request, response) => {  
  response.sendFile(frontendIndexPath);  
});
```

Significa:

- Si la ruta no empieza por `/api`, se devuelve `public/index.html`.
- Svelte decide despues que pantalla mostrar.
- Por eso funcionan rutas directas como `/analytics` o `/citys-stats/editar/tokyo/japan`.

## 12. APIs REST del proyecto

### 12.1 `citys-stats` v2

Archivo:

```
src/back/v2/citys-stats.js
```

Ruta base:

```
/api/v2/citys-stats
```

Uso principal:

- CRUD de LCC desde el frontend.
- Búsqueda libre.
- Filtros exactos.

- Ordenacion.
- Paginacion.
- Validacion estricta.

Endpoints:

| Metodo | Ruta                                | Resultado                                                                                        |
|--------|-------------------------------------|--------------------------------------------------------------------------------------------------|
| GET    | /api/v2/citys-stats/docs            | Redirige a Postman                                                                               |
| GET    | /api/v2/citys-stats/loadInitialData | Inserta datos si la base esta vacia o devuelve existentes                                        |
| GET    | /api/v2/citys-stats                 | Lista con filtros, <code>q</code> , <code>sort</code> , <code>offset</code> , <code>limit</code> |
| GET    | /api/v2/citys-stats/:city/:country  | Obtiene un registro                                                                              |
| POST   | /api/v2/citys-stats                 | Crea un registro                                                                                 |
| POST   | /api/v2/citys-stats/:city/:country  | 405                                                                                              |
| PUT    | /api/v2/citys-stats                 | 405                                                                                              |
| PUT    | /api/v2/citys-stats/:city/:country  | Actualiza registro                                                                               |
| DELETE | /api/v2/citys-stats                 | Borra todos y devuelve 204                                                                       |
| DELETE | /api/v2/citys-stats/:city/:country  | Borra uno y devuelve 204                                                                         |

Queries utiles:

```
GET /api/v2/citys-stats?q=india
GET /api/v2/citys-stats?city=tokyo
GET /api/v2/citys-stats?country=china
GET /api/v2/citys-stats?un_2025_population=33412512
GET /api/v2/citys-stats?sort=-un_2025_population
GET /api/v2/citys-stats?sort=city&offset=0&limit=5
```

## 12.2 citys-stats v1

Archivo:

```
src/back/v1/citys-stats.js
```

Ruta base:

```
/api/v1/citys-stats
```

Uso principal:

- Compatibilidad v1.
- CRUD basico con filtros exactos y paginacion.
- Endpoints de agregacion.
- Proxies e integraciones externas.



Endpoints especiales:

| Metodo | Ruta                                                     | Uso                     |
|--------|----------------------------------------------------------|-------------------------|
| GET    | /api/v1/citys-stats/top-cities                           | Top por poblacion       |
| GET    | /api/v1/citys-stats/country-summaries                    | Agregado local por pais |
| GET    | /api/v1/citys-stats/integrations/geocoding/:city         | Proxy Open-Meteo        |
| GET    | /api/v1/citys-stats/integrations/country/:country        | Proxy REST Countries    |
| GET    | /api/v1/citys-stats/integrations/world-bank/:countryCode | Proxy World Bank        |
| GET    | /api/v1/citys-stats/integrations/sos-tourist-arrivals    | Proxy SOS2526-25        |
| GET    | /api/v1/citys-stats/integrations/sos-earthquakes         | Proxy SOS2526-19        |
| GET    | /api/v1/citys-stats/integrations/sos-fifa-squad-values   | Proxy SOS2526-26        |
| GET    | /api/v1/citys-stats/integrations/sos-esports-earnings    | Proxy SOS2526-30        |
| GET    | /api/v1/citys-stats/integrations/summary                 | Resumen integrado       |

12.3 natural-disasters v2

Archivo:

```
src/back/v2/natural-disasters.js
```

Ruta base:

```
/api/v2/natural-disasters
```

Endpoints:

| Metodo | Ruta                                      | Resultado                           |
|--------|-------------------------------------------|-------------------------------------|
| GET    | /api/v2/natural-disasters/docs            | Redirige a Postman                  |
| GET    | /api/v2/natural-disasters/loadInitialData | Inserta datos si la base esta vacia |
| GET    | /api/v2/natural-disasters                 | Lista con filtros y paginacion      |
| GET    | /api/v2/natural-disasters/:country/:year  | Obtiene un registro                 |
| POST   | /api/v2/natural-disasters                 | Crea un registro                    |
| PUT    | /api/v2/natural-disasters/:country/:year  | Actualiza un registro               |
| DELETE | /api/v2/natural-disasters/:country/:year  | Borra uno                           |
| DELETE | /api/v2/natural-disasters                 | Borra todos                         |
| POST   | /api/v2/natural-disasters/:country/:year  | 405                                 |
| PUT    | /api/v2/natural-disasters                 | 405                                 |

Queries v2:

```
GET /api/v2/natural-disasters?country=spa
GET /api/v2/natural-disasters?year=2024
GET /api/v2/natural-disasters?from=1990&to=2010
GET /api/v2/natural-disasters?offset=0&limit=10
```

Nota: `loadInitialData` de desastres v2 responde `400` si la base ya tiene datos.

## 12.4 `natural-disasters` v1

Archivo:

```
src/back/v1/natural-disasters.js
```

Ruta base:

```
/api/v1/natural-disasters
```

Es la version inicial. Mantiene CRUD y busquedas mas basicas.

## 12.5 `wine-stats` v1

Archivo:

```
src/back/v1/wine-stats.js
```

Ruta base:

```
/api/v1/wine-stats
```

Endpoints:

| Metodo | Ruta                               | Resultado                       |
|--------|------------------------------------|---------------------------------|
| GET    | /api/v1/wine-stats/docs            | Redirige a Postman              |
| GET    | /api/v1/wine-stats/loadInitialData | Inserta datos iniciales con ids |
| GET    | /api/v1/wine-stats                 | Lista con filtros y paginacion  |
| GET    | /api/v1/wine-stats/:id             | Obtiene un vino                 |
| POST   | /api/v1/wine-stats                 | Crea un vino con id generado    |
| POST   | /api/v1/wine-stats/:id             | 405                             |
| PUT    | /api/v1/wine-stats                 | 405                             |
| PUT    | /api/v1/wine-stats/:id             | Actualiza un vino               |
| DELETE | /api/v1/wine-stats                 | Borra todos y devuelve 204      |
| DELETE | /api/v1/wine-stats/:id             | Borra uno y devuelve 204        |

Filtros:

```
title, country, region, grape, type
id, year, price, abv, unit, capacity
offset, limit
```

## 13. Frontend: rutas, servicios y pantallas

### Punto de entrada

Archivo:

```
frontend-group/src/main.js
```

Hace:

1. Importa `mount` de Svelte.
2. Importa `app.css`.
3. Importa `App.svelte`.
4. Monta la app en `document.getElementById("app")`.

### Router real

Archivo:

```
frontend-group/src/App.svelte
```

Funcionamiento:

1. Usa `import.meta.glob("./routes/**/*.svelte", { eager: true })`.
2. Convierte cada ruta de archivo en ruta web.
3. Compila segmentos dinamicos como `[city]` o `[country]` a expresiones regulares.
4. Lee `window.location.pathname`.
5. Elige el componente correspondiente.
6. Escucha `popstate` para actualizar la pantalla.

Ejemplos:

| Archivo                                                              | Ruta                                            |
|----------------------------------------------------------------------|-------------------------------------------------|
| <code>routes/+page.svelte</code>                                     | <code>/</code>                                  |
| <code>routes/citys-stats/+page.svelte</code>                         | <code>/citys-stats</code>                       |
| <code>routes/citys-stats/editar/[city]/[country]/+page.svelte</code> | <code>/citys-stats/editar/:city/:country</code> |
| <code>routes/analytics/+page.svelte</code>                           | <code>/analytics</code>                         |
| <code>routes/integrations/citys-stats/+page.svelte</code>            | <code>/integrations/citys-stats</code>          |

## Navegacion interna

Archivo:

```
frontend-group/src/lib/navigation.js
```

Funciones:

- `navigate(path)` : usa `history.pushState` y dispara `PopStateEvent`.
- `replace(path)` : reemplaza la URL actual y dispara `PopStateEvent`.
- `back()` : llama a `window.history.back()`.

## Services

Los services evitan duplicar URLs y parsing de errores en cada pantalla.

| Service                                | API que llama                          | Uso                         |
|----------------------------------------|----------------------------------------|-----------------------------|
| <code>apiBase.js</code>                | Calcula origen                         | Local con Vite o despliegue |
| <code>citysStatsApi.js</code>          | <code>/api/v2/citys-stats</code>       | CRUD LCC                    |
| <code>citysStatsIntegrations.js</code> | <code>/api/v1/citys-stats</code>       | Integraciones LCC           |
| <code>natural-disasters.js</code>      | <code>/api/v2/natural-disasters</code> | CRUD desastres              |
| <code>wine-stats.js</code>             | <code>/api/v1/wine-stats</code>        | CRUD vinos                  |

## Pantallas principales

| Ruta                                     | Archivo                                                                          |
|------------------------------------------|----------------------------------------------------------------------------------|
| /                                        | frontend-group/src/routes/+page.svelte                                           |
| /citys-stats                             | frontend-group/src/routes/citys-stats/+page.svelte                               |
| /citys-stats/editar/:city/:country       | frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte       |
| /natural-disasters                       | frontend-group/src/routes/natural-disasters/+page.svelte                         |
| /natural-disasters/editar/:country/:year | frontend-group/src/routes/natural-disasters/editar/[country]/[year]/+page.svelte |
| /wine-stats                              | frontend-group/src/routes/wine-stats/+page.svelte                                |
| /wine-stats/editar/:id                   | frontend-group/src/routes/wine-stats/editar/[id]/+page.svelte                    |
| /analytics                               | frontend-group/src/routes/analytics/+page.svelte                                 |
| /analytics/citys-stats                   | frontend-group/src/routes/analytics/citys-stats/+page.svelte                     |
| /analytics/citys-stats/map               | frontend-group/src/routes/analytics/citys-stats/map/+page.svelte                 |
| /integrations                            | frontend-group/src/routes/integrations/+page.svelte                              |
| /integrations/citys-stats                | frontend-group/src/routes/integrations/citys-stats/+page.svelte                  |

## 14. Parte LCC: citys-stats

### Que es

**citys-stats** gestiona estadísticas de población estimada para ciudades en 2025.

Campos:

```
city
country
un_2025_population
```

Identificador:

```
city + country
```

### Por que se llama citys-stats

Aunque en inglés correcto sería **cities-stats**, el recurso publicado en la práctica es **citys-stats**. Cambiarlo rompería:

- URLs.

- Tests.
- Documentacion Postman.
- Enlaces del frontend.
- Integraciones ya entregadas.

Frase de defensa:

Mantengo `citys-stats` porque es el contrato publico del proyecto. En APIs, estabilidad del contrato pesa mas que corregir un nombre historico.

## API LCC principal

Para CRUD se usa:

```
/api/v2/citys-stats
```

Motivo:

- Tiene busqueda libre `q`.
- Tiene ordenacion `sort`.
- Tiene paginacion `limit` y `offset`.
- Tiene validacion estricta.
- Devuelve JSON limpio sin `_id`.

## API LCC de integraciones

Para integraciones se usa:

```
/api/v1/citys-stats/integrations/...
```

Motivo:

- El proxy externo esta implementado en v1.
- Mantiene compatibilidad con entregas previas.
- Centraliza llamadas a Open-Meteo, REST Countries, World Bank y APIs SOS externas.

## CRUD visible

Pantalla:

```
frontend-group/src/routes/citys-stats/+page.svelte
```

Funciones principales de la pantalla:

- `emptycreateForm`
- `emptySearchForm`
- `clearFeedback`

- `hasQueryValues`
- `parsePositiveInteger`
- `parseOptionalNonNegativeInteger`
- `parseOptionalPositiveInteger`
- `validateCityStatForm`
- `buildSearchQuery`
- `refreshList`
- `handleSearch`
- `handleResetSearch`
- `handleCreate`
- `handleLoadInitialData`
- `handleDeleteAll`
- `handleDeleteOne`
- `openEdit`

## Edicion separada

Pantalla:

```
frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte
```

Comportamiento importante:

- Carga el registro con `getOneCityStat(params.city, params.country)`.
- Si el usuario cambia solo poblacion, hace `PUT`.
- Si el usuario cambia `city` o `country`, crea el registro nuevo y borra el antiguo.
- Usa `replace(...)` para actualizar la URL cuando cambia la clave.

Esto evita una limitacion natural de `PUT`: la URL identifica el recurso original, pero si cambia la clave compuesta, en realidad nace otra URL.

## Visualizacion individual

Pantalla:

```
frontend-group/src/routes/analytics/citys-stats/+page.svelte
```

Usa:

- Highcharts.
- Grafico `pie`.
- Datos de `getAllCityStats({ sort: "-un_2025_population" })`.

Frase:

La visualizacion individual no es `line`; es un `pie` que representa la proporcion de poblacion estimada por

ciudad.

## Mapa

Pantalla:

```
frontend-group/src/routes/analytics/citys-stats/map/+page.svelte
```

Usa:

- Highcharts Maps.
- `@highcharts/map-collection/custom/world.topo.json`.
- Coordenadas locales en el objeto `coordinates`.
- Marcadores `mappoint`.
- Color y radio segun poblacion.

Claves de coordenadas:

```
city|country
```

Ejemplo:

```
"tokyo|japan": { lat: 35.6762, lon: 139.6503 }
```

---

## 15. Integraciones LCC

Pantalla:

```
frontend-group/src/routes/integrations/citys-stats/+page.svelte
```

Backend:

```
src/back/v1/citys-stats.js
```

Service:

```
frontend-group/src/services/citysStatsIntegrations.js
```

### Decision clave: integrar por pais

La integracion se hace por `country`, no por ciudad.

Motivos:

- REST Countries trabaja por pais.



- World Bank trabaja por pais o codigo ISO.
- Muchas APIs SOS externas publican pais o ISO3.
- Los nombres de ciudad no coinciden bien entre fuentes.
- El cruce por pais produce menos huecos y mas informacion real.

Frase de defensa:

Integro por pais porque es el campo comun mas estable entre mi recurso y las APIs externas. Asi evito una integracion fragil por nombres de ciudades y consigo widgets con datos reales comparables.

APIs integradas

| API                                       | Tipo       | Fuente                                                                | Endpoint proxy local                                     | Widget   |
|-------------------------------------------|------------|-----------------------------------------------------------------------|----------------------------------------------------------|----------|
| Open-Meteo Geocoding                      | No SOS     | https://geocoding-api.open-meteo.com/v1/search                        | /api/v1/citys-stats/integrations/geocoding/:city         | treemap  |
| REST Countries                            | No SOS     | https://restcountries.com/v3.1/name/:country                          | /api/v1/citys-stats/integrations/country/:country        | sankey   |
| World Bank Indicators                     | No SOS     | https://api.worldbank.org/v2/country/:code/indicator/SP.POP.TOTL      | /api/v1/citys-stats/integrations/world-bank/:countryCode | lollipop |
| SOS2526-25 international-tourist-arrivals | Alumno SOS | https://sos2526-25.onrender.com/api/v2/international-tourist-arrivals | /api/v1/citys-stats/integrations/sos-tourist-arrivals    | variwide |
| SOS2526-19 earthquakes                    | Alumno SOS | https://sos2526-19.onrender.com/api/v1/earthquakes                    | /api/v1/citys-stats/integrations/sos-earthquakes         | bullet   |
| SOS2526-26 fifa-squad-value-per-years     | Alumno SOS | https://sos2526-26.onrender.com/api/v2/fifa-squad-value-per-years     | /api/v1/citys-stats/integrations/sos-fifa-squad-values   | dumbbell |
| SOS2526-30 esports-earnings-stats         | Alumno SOS | https://sos2526-30.onrender.com/api/v1/esportsearnings-stats          | /api/v1/citys-stats/integrations/sos-esports-earnings    | sunburst |

Cumplimiento D03

- Usa 7 APIs externas.
- Usa 3 APIs no SOS: Open-Meteo, REST Countries y World Bank.
- Usa 4 APIs SOS de otros grupos: 25, 19, 26 y 30.
- Todas devuelven JSON.
- Todas pasan por proxy propio en Express.
- No se muestra JSON crudo: se transforma en graficas, tarjetas, tablas y metricas.

- Los widgets no son `line`.
- La vista esta enlazada desde `/integrations`.
- El widget grupal esta en `/analytics`.

## Flujo de integraciones

1. El usuario abre `/integrations/citys-stats`.
2. La pantalla llama a `getCountrySummaries(selectedLimit)`.
3. El service hace `GET /api/v1/citys-stats/country-summaries?limit=N`.
4. El backend lee NeDB.
5. `buildCityCountrySummaries` agrega ciudades por pais.
6. La pantalla usa esos paises como base.
7. Para cada pais o ciudad principal, llama a endpoints proxy.
8. Express llama a APIs externas con `fetchJson`.
9. El backend normaliza datos y controla errores.
10. El frontend construye widgets Highcharts distintos.

## Endpoint resumen

Ruta:

```
GET /api/v1/citys-stats/integrations/summary?limit=8
```

Respuesta real resumida:

```
{
  localResource: "/api/v1/citys-stats/country-summaries",
  externalApis: string[],
  studentApis: object[],
  count: number,
  items: object[]
}
```

Campos importantes de cada item:

| Campo               | Significado                              |
|---------------------|------------------------------------------|
| city                | Ciudad principal del pais                |
| country             | Pais usado como clave de cruce           |
| cityCount           | Numero de ciudades locales de ese pais   |
| topCity             | Ciudad local mas poblada                 |
| topCityPopulation   | Poblacion de la ciudad local mas poblada |
| cities              | Lista de ciudades locales del pais       |
| un_2025_population  | Suma local de poblacion 2025 del pais    |
| geocoding           | Datos de Open-Meteo                      |
| countryInfo         | Datos de REST Countries                  |
| worldBankPopulation | Dato de poblacion World Bank             |
| touristArrivals     | Datos agregados de turismo               |
| earthquakeStats     | Datos agregados de terremotos            |
| fifaSquadValue      | Datos agregados FIFA                     |
| esportsEarnings     | Datos agregados eSports                  |
| integrationErrors   | Errores parciales                        |

## 16. Funciones importantes explicadas

Esta seccion no intenta repetir todo el codigo. Recoge las funciones y bloques que conviene saber explicar durante la defensa.

### registerNaturalDisastersV1

Ubicacion:

```
src/back/v1/natural-disasters.js
```

Proposito:

Registra todas las rutas v1 de `natural-disasters`.

Cuando se ejecuta:

Al arrancar `index.js`, cuando se hace:

```
const naturalDisastersApiV1 = require("../src/back/v1/natural-disasters");
naturalDisastersApiV1(app, naturalDisastersDb);
```

Explicacion sencilla:

Es una funcion que recibe el servidor Express y la base de datos. Dentro va anadiendo rutas con `app.get`, `app.post`, `app.put` y `app.delete`.

Importancia:

Permite separar el recurso en su propio archivo y no llenar `index.js` de rutas.

## registerNaturalDisastersV2

Ubicacion:

```
src/back/v2/natural-disasters.js
```

Proposito:

Registra la API v2 de desastres, con busqueda parcial por pais y rango de anios.

Funciones internas importantes:

- `removeDatabaseId`
- `hasValidDisasterBody`
- `buildSearchQuery`
- `getPagination`

Explicacion sencilla:

La v2 usa la misma base que v1, pero ofrece una forma mejor de consultar datos. Por ejemplo, `country=spa` puede encontrar `spain` y `from=1990&to=2010` permite rangos.

## removeDatabaseId

Ubicacion:

```
src/back/v1/natural-disasters.js
src/back/v2/natural-disasters.js
src/back/v1/citys-stats.js
src/back/v2/citys-stats.js
src/back/v1/wine-stats.js
```

Proposito:

Eliminar `_id`, el identificador interno de NeDB, antes de responder al cliente.

Codigo conceptual:

```
function removeDatabaseId(doc) {
  if (!doc) return doc;
  const { _id, ...rest } = doc;
  return rest;
}
```

Explicacion tecnica:

Usa destructuring para separar `_id` del resto de propiedades. Devuelve solo el objeto publico.

Explicacion para principiantes:

NeDB guarda un campo interno para organizarse. Ese campo no pertenece al contrato de nuestra API, asi que lo quitamos antes de enviar JSON.

Respuesta de defensa:

Ocultamos `_id` porque la API publica no debe depender de detalles de persistencia.

## hasExactCityFields

Ubicacion:

```
src/back/v1/citys-stats.js  
src/back/v2/citys-stats.js
```

Proposito:

Comprobar que un body de `citys-stats` tiene exactamente:

```
city  
country  
un_2025_population
```

Parametros:

- `body`: objeto recibido por `POST` o `PUT`.

Devuelve:

- `true` si tiene exactamente esos campos.
- `false` si falta alguno, sobra alguno, es array, `null` o no es objeto.

Explicacion sencilla:

Es el guardia de entrada. No deja pasar objetos incompletos ni objetos con campos que la API no conoce.

Importancia:

Evita guardar datos con forma inesperada y permite responder `400` cuando el cliente manda mal el JSON.

## normalizeCityStat

Ubicacion:

```
src/back/v1/citys-stats.js  
src/back/v2/citys-stats.js
```

Proposito:

Limpiar y validar una ciudad antes de guardarla.

Parametros:

- `body`: objeto con `city`, `country` y `un_2025_population`.

Devuelve:

- `{ city, country, un_2025_population }` si es valido.
- `null` si no es valido.

Que hace:

1. Comprueba estructura exacta con `hasExactCityFields`.
2. Convierte `city` a texto, quita espacios y lo pasa a minusculas.
3. Convierte `country` a texto, quita espacios y lo pasa a minusculas.
4. Convierte `un_2025_population` a numero.
5. Exige poblacion entera y mayor que cero.

Explicacion sencilla:

Si el usuario escribe `Tokyo` y `Japan`, el backend guarda `tokyo` y `japan`. Asi las busquedas y claves son consistentes.

## GET /api/v2/citys-stats

Ubicacion:

```
src/back/v2/citys-stats.js
```

Proposito:

Devolver la coleccion de ciudades con filtros, busqueda libre, orden y paginacion.

Orden interno:

1. Lee todos los documentos con `db.find({})`.
2. Quita `_id`.
3. Aplica filtros exactos: `city`, `country`, `un_2025_population`.
4. Aplica busqueda libre `q` en ciudad o pais.
5. Aplica `sort`.
6. Valida y aplica `offset`.
7. Valida y aplica `limit`.
8. Devuelve `200` con JSON.

Explicacion sencilla:

Primero trae los datos, luego va recortando la lista segun lo que el usuario haya pedido en la URL.

Caso especial:

Si `sort` no esta en `["city", "country", "un_2025_population"]`, devuelve `400`.

## POST /api/v2/citys-stats

Ubicacion:

```
src/back/v2/citys-stats.js
```

Proposito:

Crear una ciudad nueva.

Orden interno:

1. Normaliza el body con `normalizeCityStat`.
2. Si falla, responde `400`.
3. Busca duplicado por `city + country`.
4. Si existe, responde `409`.
5. Inserta en NeDB.
6. Devuelve `201` con el documento sin `_id`.

Explicacion sencilla:

Antes de guardar, comprueba que el formulario esta bien y que no existe ya una ciudad con el mismo pais.

## PUT /api/v2/citys-stats/:city/:country

Ubicacion:

```
src/back/v2/citys-stats.js
```

Proposito:

Actualizar una ciudad existente.

Regla importante:

El `city` y `country` del body deben coincidir con los de la URL.

Motivo:

La URL identifica el recurso que se esta modificando. Si la URL dice `tokyo/japan` pero el body dice `madrid/spain`, la peticion es ambigua.

Respuesta:

- `400`: body invalido o URL/body no coinciden.
- `404`: no existe el recurso.
- `200`: actualizado correctamente.

## buildCityCountrySummaries

Ubicacion:

```
src/back/v1/citys-stats.js
```

Proposito:

Agrupar registros de `citys-stats` por pais para integraciones.

Parametros:

- `items`: lista de ciudades locales.

Devuelve:

Una lista de paises con:

- `country`
- `countryKey`
- `cityCount`
- `un_2025_population`
- `topCity`
- `topCityPopulation`
- `cities`

Explicacion sencilla:

Si hay varias ciudades de India, las junta en un bloque India, suma sus poblaciones y guarda cual es la ciudad mas poblada.

Importancia:

Es la base de la integracion por pais.

## `fetchJson`

Ubicacion:

```
src/back/v1/citys-stats.js
```

Proposito:

Hacer llamadas HTTP a APIs externas con control de JSON, errores y timeout.

Parametros:

- `url`: URL externa.
- `sourceName`: nombre de la API para mensajes.
- `timeoutMs`: tiempo maximo antes de abortar.

Devuelve:

- Datos JSON si todo va bien.
- Lanza error si la API no responde, no devuelve JSON, devuelve estado HTTP de error o supera timeout.

Explicacion sencilla:



Es un `fetch` mas protegido. No se fia de que la API externa funcione siempre.

Importancia:

Permite que las integraciones fallen de forma controlada.

## `safeExternal`

Ubicacion:

```
src/back/v1/citys-stats.js
```

Proposito:

Ejecutar una llamada externa y convertir cualquier fallo en un objeto de error controlado.

Devuelve:

```
{ source, data, error }
```

Explicacion sencilla:

En vez de dejar que un fallo rompa toda la pagina, guarda el error y permite seguir con las demas fuentes.

Frase de defensa:

`safeExternal` hace que una API externa caida no tumbe toda la integracion.

## `buildIntegratedCityBase`

Ubicacion:

```
src/back/v1/citys-stats.js
```

Proposito:

Pedir en paralelo Open-Meteo y REST Countries para una ciudad/pais local.

Usa:

```
Promise.all([
  safeExternal("Open-Meteo Geocoding API", ...),
  safeExternal("REST Countries API", ...)
])
```

Explicacion sencilla:

Para cada pais base, prepara las dos primeras fuentes externas necesarias.

## `buildIntegratedCity`

Ubicacion:

```
src/back/v1/citys-stats.js
```

Proposito:

Unir datos locales, Open-Meteo, REST Countries, World Bank y APIs SOS externas en un unico objeto.

Devuelve campos como:

- `city`
- `country`
- `cityCount`
- `un_2025_population`
- `geocoding`
- `countryInfo`
- `worldBankPopulation`
- `touristArrivals`
- `earthquakeStats`
- `fifaSquadValue`
- `esportsEarnings`
- `integrationErrors`

Explicacion sencilla:

Es el punto donde se monta la ficha final de integracion por pais.

## Normalizadores de APIs SOS externas

Ubicacion:

```
src/back/v1/citys-stats.js
```

Funciones:

- `normalizeTouristArrival`
- `normalizeEarthquake`
- `normalizeFifaSquadValue`
- `normalizeEsportsEarning`

Proposito:

Convertir cada API externa a un formato interno comparable.

Explicacion sencilla:

Cada API externa llama a sus campos de una forma distinta. Estas funciones traducen esos campos a nombres y numeros que el proyecto entiende.

## buildUrl

Ubicacion:

```
frontend-group/src/services/citysStatsApi.js
```

Proposito:

Construir URLs completas para `fetch`, incluyendo query params.

Explicacion sencilla:

En vez de concatenar texto a mano, usa `URL` y `searchParams`. Asi evita errores con `?`, `&` o espacios.

## handleResponse

Ubicacion:

```
frontend-group/src/services/citysStatsApi.js  
frontend-group/src/services/citysStatsIntegrations.js  
frontend-group/src/services/wine-stats.js
```

Proposito:

Procesar respuestas HTTP.

Que hace:

1. Si el estado es `204`, devuelve `null`.
2. Lee el cuerpo como texto.
3. Intenta parsear JSON.
4. Si `response.ok` es falso, lanza `Error`.
5. Si todo va bien, devuelve datos.

Explicacion sencilla:

Transforma respuestas HTTP en datos normales o errores que la pantalla puede mostrar.

## getAllCitysStats

Ubicacion:

```
frontend-group/src/services/citysStatsApi.js
```

Proposito:

Pedir registros de `citys-stats` al backend.

Parametros:

- `query`: filtros opcionales.

Ejemplo:

```
getAllCityStats({ country: "china", sort: "-un_2025_population", limit: 1 })
```

Resultado:

Hace `GET /api/v2/citys-stats?country=china&sort=-un_2025_population&limit=1`.

### **validateCityStatForm**

Ubicacion:

```
frontend-group/src/routes/citys-stats/+page.svelte
```

Proposito:

Validar el formulario de creacion antes de llamar al backend.

Devuelve:

Un payload limpio:

```
{ city, country, un_2025_population }
```

Explicacion sencilla:

Evita mandar una peticion si faltan ciudad, pais o poblacion valida.

### **buildSearchQuery**

Ubicacion:

```
frontend-group/src/routes/citys-stats/+page.svelte
```

Proposito:

Convertir los inputs de busqueda en un objeto de query para el service.

Campos:

- `q`
- `city`
- `country`
- `un_2025_population`
- `sort`
- `limit`
- `offset`

Explicacion sencilla:

Traduce el formulario de busqueda a parametros que entiende la API.

## refreshList

Ubicacion:

```
frontend-group/src/routes/citys-stats/+page.svelte
```

Proposito:

Recargar la tabla segun la busqueda activa.

Que actualiza:

- `loading`
- `error`
- `citysStats`
- `activeQuery`
- `message`

Explicacion sencilla:

Es la funcion que mantiene sincronizada la tabla con el backend.

## handleCreate

Ubicacion:

```
frontend-group/src/routes/citys-stats/+page.svelte
```

Proposito:

Crear un registro desde la pantalla.

Orden:

1. Limpia mensajes.
2. Valida formulario.
3. Llama a `createCityStat`.
4. Limpia formulario.
5. Recarga la lista.
6. Muestra mensaje de exito.

## loadHighcharts y renderChart

Ubicacion:

```
frontend-group/src/routes/analytics/+page.svelte  
frontend-group/src/routes/analytics/citys-stats/+page.svelte  
frontend-group/src/routes/integrations/citys-stats/+page.svelte
```

Proposito:

- `loadHighcharts` : cargar Highcharts y modulos cuando hacen falta.
- `renderChart` : crear o recrear la grafica.

Explicacion sencilla:

Primero se cargan datos, luego se espera a que exista el contenedor HTML y despues Highcharts pinta.

## `renderMap`

Ubicacion:

```
frontend-group/src/routes/analytics/citys-stats/map/+page.svelte
```

Proposito:

Crear el mapa mundial de ciudades con Highcharts Maps.

Usa:

- `worldMap`
- `coordinates`
- `keyFor`
- `colorFor`
- `radiusFor`
- `points`

Explicacion sencilla:

Convierte cada ciudad con coordenadas en un marcador del mapa. El marcador cambia de color y tamano segun la poblacion.

## `loadIntegrations`

Ubicacion:

```
frontend-group/src/routes/integrations/citys-stats/+page.svelte
```

Proposito:

Cargar todas las integraciones LCC.

Orden:

1. Pide resumen local por pais.
2. Si no hay datos, intenta cargar datos iniciales.
3. Pide Open-Meteo para ciudades principales.
4. Pide REST Countries.
5. Pide World Bank usando ISO3.
6. Pide cuatro APIs SOS externas.
7. Recoge errores parciales.

8. Carga Highcharts.
9. Renderiza siete widgets.

### normalizeWineStat

Ubicacion:

```
src/back/v1/wine-stats.js
```

Proposito:

Validar y limpiar los datos de un vino.

Importancia:

Convierte campos numericos, normaliza `country` y evita registros sin `title`, `country`, `year` o `price`.

### getNextWineId

Ubicacion:

```
src/back/v1/wine-stats.js
```

Proposito:

Calcular el siguiente `id` numerico para un vino nuevo.

Explicacion sencilla:

Lee todos los vinos, busca el `id` mas alto y suma 1.

## Como estudiar una funcion en defensa

El plan de accion pide que las funciones no se documenten como una lista de nombres. Para cada funcion importante hay que saber responder estas preguntas:

- Que problema resuelve.
- En que archivo esta.
- Cuando se ejecuta.
- Que parametros recibe.
- Que devuelve.
- Que funciones llama.
- Que funciones la llaman.
- Que errores o casos especiales controla.
- Como explicarla a una persona principiante.
- Como encaja en el flujo completo.

Plantilla corta para defender cualquier funcion:

Esta funcion esta en <archivo>. Se ejecuta cuando <momento>.  
Recibe <parametros>, valida o transforma <datos>, llama a <otras funciones>  
y devuelve <resultado>. Es importante porque <impacto en el sistema>.  
Si algo va mal, controla <errores> y evita <problema>.

Mapa de funciones LCC por capa

| Capa                           | Funciones clave                                                                                                        | Archivo                                                                    |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Backend<br>CRUD v2             | hasExactCityFields, normalizeCityStat, handler GET, handler POST, handler PUT, handler DELETE                          | src/back/v2/citys-stats.js                                                 |
| Backend<br>integraciones<br>v1 | buildCityCountrySummaries, fetchJson, safeExternal, buildIntegratedCityBase, buildIntegratedCity                       | src/back/v1/citys-stats.js                                                 |
| Normalizacio<br>n externa      | normalizeTouristArrival, normalizeEarthquake, normalizeFifaSquadValue, normalizeEsportsEarning                         | src/back/v1/citys-stats.js                                                 |
| Agregacion<br>externa          | buildTouristArrivalsByCountry, buildEarthquakesByCountry, buildFifaSquadValuesByCountry, buildEsportsEarningsByCountry | src/back/v1/citys-stats.js                                                 |
| Service<br>CRUD                | buildUrl, handleResponse, getAllCitysStats, createCityStat, updateCityStat                                             | frontend-group/src/services/citysStatsApi.js                               |
| Service<br>integraciones       | getCountrySummaries, getGeocoding, getCountryInfo, getWorldBankPopulation, getSos...                                   | frontend-group/src/services/citysStatsIntegrations.js                      |
| Pantalla<br>CRUD               | validateCityStatForm, buildSearchQuery, refreshList, handleCreate, handleSearch, openEdit                              | frontend-group/src/routes/citys-stats/+page.svelte                         |
| Pantalla<br>editar             | loadResource, isSameResource, handleUpdate, updateRoute                                                                | frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte |
| Analytics                      | loadHighcharts, buildMetrics, renderChart, loadAnalytics                                                               | frontend-group/src/routes/analytics/+page.svelte                           |
| Mapa                           | keyFor, colorFor, radiusFor, renderMap, loadMapData                                                                    | frontend-group/src/routes/analytics/citys-stats/map/+page.svelte           |
| Integraciones<br>UI            | safeLoad, loadHighcharts, createChart, renderIntegrationCharts, loadIntegrations                                       | frontend-group/src/routes/integrations/citys-stats/+page.svelte            |

Funcion ampliada: registro del modulo LCC v2

Ubicacion:

src/back/v2/citys-stats.js



**Funcion real:**

```
module.exports = (app, db) => { ... }
```

**Proposito general:**

Registrar todas las rutas de la API v2 de `citys-stats`.

**Cuando se ejecuta:**

Al arrancar `index.js`, despues de crear `app` y `citysStatsDb`.

**Parametros:**

- `app`: instancia de Express. Sirve para registrar rutas con `app.get`, `app.post`, `app.put` y `app.delete`.
- `db`: base NeDB de ciudades. Sirve para leer, insertar, actualizar y borrar documentos.

**Devuelve:**

No devuelve un valor util. Su efecto importante es dejar registradas las rutas de Express.

**Llama a:**

- `app.get`
- `app.post`
- `app.put`
- `app.delete`
- Funciones internas como `normalizeCityStat` y `removeDatabaseId`
- Metodos de NeDB como `db.find`, `db.findOne`, `db.insert`, `db.update`, `db.remove`

**La llama:**

`index.js`, con:

```
const citysStatsApiV2 = require("../src/back/v2/citys-stats");
citysStatsApiV2(app, citysStatsDb);
```

**Explicacion tecnica:**

Es un modulo CommonJS que exporta una funcion. Al recibir `app` y `db`, no crea un servidor nuevo: usa el servidor ya existente y le enchufa rutas. Esto permite que `index.js` sea el coordinador y que cada recurso tenga su archivo.

**Explicacion para principiantes:**

Imagina que Express es una centralita. Esta funcion registra que debe pasar cuando alguien llama a URLs como `/api/v2/citys-stats` o `/api/v2/citys-stats/tokyo/japan`.

**Bloques importantes:**

1. Define `BASE_API_URL`.
2. Define `DOCS_URL`.
3. Define `initialData`.

4. Define helpers de validacion y limpieza.
5. Registra rutas de documentacion, carga inicial, CRUD y metodos no permitidos.

**Errores o casos especiales:**

- Si NeDB falla, devuelve `500`.
- Si el body no cumple estructura, devuelve `400`.
- Si el recurso ya existe, devuelve `409`.
- Si no existe, devuelve `404`.
- Si se usa un metodo incorrecto, devuelve `405`.

**Importancia:**

Es el contrato principal de LCC para el CRUD.

**Funcion ampliada: `hasExactCityFields`****Ubicacion:**

```
src/back/v1/citys-stats.js  
src/back/v2/citys-stats.js
```

**Proposito general:**

Comprobar que el JSON recibido tiene exactamente los campos del recurso.

**Cuando se ejecuta:**

Cada vez que se crea o actualiza una ciudad, porque `normalizeCityStat` la llama.

**Parametros:**

- `body`: contenido JSON recibido en `req.body`.

**Devuelve:**

- `true` si `body` es un objeto con exactamente `city`, `country` y `un_2025_population`.
- `false` si falta un campo, sobra un campo, es `null`, no es objeto o es un array.

**Llama a:**

- `Object.keys`
- `Array.isArray`
- `sort`
- `every`

**La llama:**

- `normalizeCityStat`

**Codigo analizado:**

```
function hasExactCityFields(body) {  
  if (!body || typeof body !== "object" || Array.isArray(body)) return false;
```

```
const expected = ["city", "country", "un_2025_population"].sort();
const keys = Object.keys(body).sort();

return keys.length === expected.length &&
  keys.every((k, i) => k === expected[i]);
}
```

### Explicacion bloque por bloque:

1. `if (!body...)`: rechaza valores vacios, arrays o tipos que no sean objeto.
2. `expected`: lista oficial de campos aceptados.
3. `keys`: campos reales enviados por el cliente.
4. Compara longitud y nombre campo a campo.

### Explicacion para principiantes:

Es como revisar una ficha antes de archivarla. Si la ficha de ciudad debe tener tres casillas, no se acepta si trae dos, cuatro o una casilla desconocida.

### Errores o casos especiales:

- Un body con `{ city, country }` falla.
- Un body con `{ city, country, un_2025_population, continent }` falla.
- Un array `[]` falla aunque sea tecnicamente un objeto en JavaScript.

### Importancia:

Hace que el contrato de la API sea estricto y predecible.

## Funcion ampliada: `normalizeCityStat`

### Ubicacion:

```
src/back/v1/citys-stats.js
src/back/v2/citys-stats.js
```

### Proposito general:

Convertir el body recibido en un objeto limpio y valido para guardar.

### Cuando se ejecuta:

En `POST /api/v2/citys-stats` y en `PUT /api/v2/citys-stats/:city/:country`.

### Parametros:

- `body`: JSON enviado por el cliente.

### Devuelve:

- Objeto normalizado: `{ city, country, un_2025_population }`.
- `null` si el dato no es valido.

### Llama a:

- `hasExactCityFields`
- `String`
- `trim`
- `toLowerCase`
- `Number`
- `Number.isInteger`

### La llaman:

- Handler `POST` de ciudades.
- Handler `PUT` de ciudades.

### Codigo analizado:

```
function normalizeCityStat(body) {
  if (!hasExactCityFields(body)) return null;

  const city = String(body.city).trim().toLowerCase();
  const country = String(body.country).trim().toLowerCase();
  const un_2025_population = Number(body.un_2025_population);

  if (
    !city ||
    !country ||
    !Number.isInteger(un_2025_population) ||
    un_2025_population <= 0
  ) {
    return null;
  }

  return { city, country, un_2025_population };
}
```

### Explicacion bloque por bloque:

1. Primero valida la estructura.
2. Limpia `city`: fuerza texto, quita espacios y pasa a minusculas.
3. Limpia `country` igual.
4. Convierte poblacion a numero.
5. Rechaza ciudad vacia, pais vacio, poblacion no entera o menor/igual que cero.
6. Devuelve el objeto ya preparado.

### Explicacion para principiantes:

Esta funcion convierte datos "sucios" de formulario en datos consistentes. Por ejemplo, `Tokyo` se guarda como `tokyo`.

### Ejemplo de entrada:

```
{
  "city": " Tokyo ",
  "country": " Japan ",
```

```
"un_2025_population": "33412512"
}
```

### Ejemplo de salida:

```
{
  "city": "tokyo",
  "country": "japan",
  "un_2025_population": 33412512
}
```

### Errores o casos especiales:

- Poblacion `"abc"` devuelve `null`.
- Poblacion `0` devuelve `null`.
- Pais vacio devuelve `null`.

### Relacion con otras partes:

El frontend tambien valida, pero esta funcion es la barrera real. Aunque alguien llame a la API sin usar la interfaz, el backend sigue protegido.

## Funcion ampliada: handler `GET /api/v2/citys-stats`

### Ubicacion:

```
src/back/v2/citys-stats.js
```

### Proposito general:

Leer la coleccion de ciudades y aplicar filtros, busqueda, ordenacion y paginacion.

### Cuando se ejecuta:

Cuando el frontend llama a `getAllCitysStats`, al abrir `/citys-stats`, buscar, abrir analytics o abrir mapa.

### Parametros:

- `req`: peticion Express. Contiene `req.query`.
- `res`: respuesta Express.

### Devuelve:

Respuesta HTTP `200` con array JSON de ciudades.

### Llama a:

- `db.find`
- `removeDatabaseId`
- Filtros JavaScript con `filter`
- Ordenacion con `sort`
- Paginacion con `slice`

**La llaman:**

- `getAllCityStats` desde `cityStatsApi.js`.
- Cualquier cliente HTTP que pida `/api/v2/citys-stats`.

**Orden bloque por bloque:**

1. `db.find({})` : obtiene todos los documentos.
2. `docs.map(removeDatabaseId)` : quita `_id`.
3. Filtra por `city` si existe.
4. Filtra por `country` si existe.
5. Filtra por `un_2025_population` si existe.
6. Aplica `q`, buscando coincidencia parcial en ciudad o pais.
7. Aplica `sort`, validando lista blanca.
8. Lee `offset` y comprueba entero no negativo.
9. Lee `limit` y comprueba entero no negativo.
10. Devuelve `result.slice(offset, offset + limit)`.

**Explicacion para principiantes:**

La API trae toda la lista y despues la va pasando por coladores: primero ciudad, luego pais, luego texto libre, luego orden y por ultimo pagina.

**Errores o casos especiales:**

- `un_2025_population=abc` devuelve `400`.
- `sort=campo_inventado` devuelve `400`.
- `offset=-1` devuelve `400`.
- `limit=abc` devuelve `400`.

**Importancia:**

Es el endpoint que alimenta la tabla, el grafico individual, el mapa y parte del widget grupal.

**Funcion ampliada: handler `POST /api/v2/citys-stats`****Ubicacion:**

```
src/back/v2/citys-stats.js
```

**Proposito general:**

Crear un registro nuevo de ciudad.

**Cuando se ejecuta:**

Cuando el usuario pulsa "Guardar registro" en `/citys-stats`.

**Parametros:**

- `req.body` : ciudad enviada por el frontend.
- `res` : respuesta HTTP.

**Devuelve:**

- `201` con la ciudad creada si todo va bien.
- `400`, `409` o `500` si hay problema.

**Llama a:**

- `normalizeCityStat`
- `db.findOne`
- `db.insert`
- `removeDatabaseId`

**La llama:**

- `createCityStat` en `frontend-group/src/services/citysStatsApi.js`.

**Explicacion bloque por bloque:**

1. Normaliza el body.
2. Si no hay item valido, devuelve `400`.
3. Busca si ya existe una ciudad con mismo `city` y `country`.
4. Si existe, devuelve `409`.
5. Inserta el nuevo documento.
6. Devuelve el documento sin `_id`.

**Explicacion para principiantes:**

Antes de guardar, comprueba dos cosas: que los datos esten bien y que no exista ya la misma ciudad en el mismo pais.

**Errores o casos especiales:**

- Body incompleto: `400`.
- Registro duplicado: `409`.
- Fallo de base de datos: `500`.

**Relacion con frontend:**

`handleCreate` prepara el payload y `createCityStat` lo envia. Si el backend responde error, `handleResponse` lo convierte en un mensaje visible.

**Funcion ampliada: handler `PUT /api/v2/citys-stats/:city/:country`****Ubicacion:**

```
src/back/v2/citys-stats.js
```

**Proposito general:**

Actualizar un registro existente identificado por ciudad y pais.

**Cuando se ejecuta:**

Cuando el usuario guarda cambios en la pantalla de edicion y no ha cambiado la clave del recurso.

**Parametros:**

- `req.params.city` : ciudad de la URL.
- `req.params.country` : pais de la URL.
- `req.body` : datos nuevos.
- `res` : respuesta HTTP.

**Devuelve:**

- `200` con documento actualizado.
- `400` , `404` o `500` si hay problema.

**Llama a:**

- `normalizeCityStat`
- `db.findOne`
- `db.update`
- `removeDatabaseId`

**La llama:**

- `updateCityStat` en `cityStatsApi.js`.
- `handleUpdate` en la pantalla de edicion.

**Regla critica:**

```
item.city === city && item.country === country
```

**Explicacion para principiantes:**

Si la URL dice "voy a editar Tokyo/Japan", el body tambien debe hablar de Tokyo/Japan. Asi se evita editar una cosa y guardar otra distinta por accidente.

**Errores o casos especiales:**

- Si body no tiene estructura exacta: `400` .
- Si URL y body no coinciden: `400` .
- Si el recurso no existe: `404` .
- Si NeDB falla: `500` .

**Relacion con la pantalla de edicion:**

La pantalla de edicion tiene una estrategia especial: si el usuario cambia `city` o `country` , no usa este `PUT` ; crea un recurso nuevo y borra el anterior.

**Funcion ampliada: `parseLimit`****Ubicacion:**

```
src/back/v1/citys-stats.js
```



**Proposito general:**

Validar limites de integraciones para evitar pedir demasiados datos.

**Cuando se ejecuta:**

En endpoints como:

- `/top-cities`
- `/country-summaries`
- `/integrations/summary`

**Parametros:**

- `value`: valor recibido por query param.
- `fallback`: valor por defecto.
- `max`: maximo permitido.

**Devuelve:**

- Numero valido si todo va bien.
- `null` si el limite no es entero, menor que 1 o mayor que `max`.

**Explicacion bloque por bloque:**

1. Si `value` es `undefined`, usa `fallback`.
2. Convierte a numero.
3. Comprueba entero y rango.
4. Devuelve numero o `null`.

**Explicacion para principiantes:**

Es un control de seguridad y rendimiento. No permite que alguien pida mil paises si la pantalla solo esta pensada para pocos.

**Funcion ampliada:** `normalizeCountryKey`**Ubicacion:**

```
src/back/v1/citys-stats.js  
frontend-group/src/routes/integrations/citys-stats/+page.svelte
```

**Proposito general:**

Crear una clave comparable para nombres de pais.

**Cuando se ejecuta:**

Al agrupar paises y al cruzar datos locales con APIs externas.

**Parametros:**

- `value`: nombre de pais recibido desde datos locales o externos.

**Devuelve:**

Texto normalizado:

- sin espacios externos,
- sin diacriticos,
- con guiones convertidos a espacios,
- en minusculas.

### Explicacion para principiantes:

Convierte formas distintas de escribir un pais a una version comun. Asi `south-korea`, `South Korea` o textos con espacios se pueden comparar mejor.

### Importancia:

Sin esta funcion, las integraciones fallarian mas porque cada API puede escribir paises de forma distinta.

### Funcion ampliada: `buildCityCountrySummaries`

#### Ubicacion:

```
src/back/v1/citys-stats.js
```

#### Proposito general:

Convertir una lista de ciudades en una lista agregada por pais.

#### Cuando se ejecuta:

En:

- `GET /api/v1/citys-stats/country-summaries`
- `GET /api/v1/citys-stats/integrations/summary`

#### Parametros:

- `items`: array de registros `citys-stats`.

#### Devuelve:

Array ordenado por poblacion agregada descendente.

#### Llama a:

- `normalizeCountryKey`
- `readFiniteNumber`
- `Map`
- `sort`

#### La llaman:

- Handler `/country-summaries`
- Handler `/integrations/summary`

#### Explicacion bloque por bloque:

1. Crea `byCountry`, un `Map` donde la clave es el pais normalizado.
2. Recorre cada ciudad.
3. Calcula la poblacion como numero seguro.
4. Si el pais no existe en el mapa, crea un resumen inicial.
5. Incrementa `cityCount`.
6. Suma `un_2025_population`.
7. Guarda la ciudad en `cities`.
8. Si esa ciudad es la mas poblada, actualiza `topCity`.
9. Convierte el `Map` a array.
10. Ordena ciudades internas por poblacion.
11. Ordena paises por poblacion total.

### Explicacion para principiantes:

Si la tabla tiene varias ciudades de un mismo pais, esta funcion las junta en una sola ficha de pais.

### Ejemplo:

Entrada:

```
delhi, india, 30222405
kolkata, india, 22549738
mumbai, india, 20203056
```

Salida conceptual:

```
india:
  cityCount = 3
  un_2025_population = suma de las tres
  topCity = delhi
```

### Importancia:

Es la decision tecnica central de D03 LCC: integrar por pais.

### Funcion ampliada: `fetchJson`

#### Ubicacion:

```
src/back/v1/citys-stats.js
```

#### Proposito general:

Consultar APIs externas de forma robusta.

#### Cuando se ejecuta:

Cada vez que el backend llama a Open-Meteo, REST Countries, World Bank o APIs SOS externas.

#### Parametros:

- `url`: URL externa.
- `sourceName`: nombre legible de la fuente.
- `timeoutMs`: milisegundos antes de abortar. Por defecto, `20000`.

**Devuelve:**

JSON parseado.

**Lanza error si:**

- La respuesta no es JSON.
- La respuesta HTTP no es correcta.
- Se supera el timeout.
- `fetch` falla.

**Llama a:**

- `AbortController`
- `setTimeout`
- `fetch`
- `response.text`
- `JSON.parse`
- `clearTimeout`

**La llaman:**

- `getGeocoding`
- `getCountryInfo`
- `getWorldBankPopulation`
- `getWorldBankPopulations`
- `getTouristArrivals`
- `getEarthquakes`
- `getFifaSquadValues`
- `getEsportsEarnings`

**Explicacion bloque por bloque:**

1. Crea un controlador para poder abortar.
2. Programa un timeout.
3. Hace `fetch` con headers JSON.
4. Lee texto.
5. Intenta convertirlo a JSON.
6. Si `response.ok` es falso, crea error con mensaje de la API.
7. Si todo va bien, devuelve datos.
8. En `finally`, limpia el timeout.

**Explicacion para principiantes:**

Es una llamada HTTP con casco. No solo pide datos: tambien comprueba que sean JSON, que la API no devuelva error y que no tarde infinito.

**Importancia:**

Hace defendible el proxy propio: no es un simple `fetch`, es una capa de control.

**Funcion ampliada:** `safeExternal`**Ubicacion:**

```
src/back/v1/citys-stats.js
```

**Proposito general:**

Convertir errores de APIs externas en datos controlados.

**Cuando se ejecuta:**

En integraciones, especialmente cuando se usa `Promise.all`.

**Parametros:**

- `source`: nombre de la fuente.
- `task`: funcion asincrona que hace la llamada externa.

**Devuelve:**

```
{ source, data, error }
```

**Llama a:**

- La funcion `task`.

**La llaman:**

- `buildIntegratedCityBase`
- Handler `/integrations/summary`

**Explicacion bloque por bloque:**

1. Intenta ejecutar `task`.
2. Si funciona, devuelve `{ data, error: null }`.
3. Si falla, captura el error y devuelve `{ data: null, error: err.message }`.

**Explicacion para principiantes:**

Permite decir: "esta API fallo, pero las demas siguen funcionando".

**Importancia:**

Evita que una API externa caida rompa toda la vista de integraciones.

## Funcion ampliada: `getGeocoding`

### Ubicacion:

```
src/back/v1/citys-stats.js
```

### Proposito general:

Pedir a Open-Meteo coordenadas y datos basicos de una ciudad.

### Cuando se ejecuta:

Cuando se llama a:

```
GET /api/v1/citys-stats/integrations/geocoding/:city
```

o al construir resumen integrado.

### Parametros:

- `city`: ciudad a buscar.
- `country`: pais opcional para elegir mejor coincidencia.

### Devuelve:

Objeto con:

- `source`
- `matchedName`
- `country`
- `countryCode`
- `latitude`
- `longitude`
- `elevation`
- `timezone`
- `population`

### Llama a:

- `cleanSearchTerm`
- `fetchJson`
- `URLSearchParams`

### Explicacion para principiantes:

Busca una ciudad fuera de nuestra base para poder obtener coordenadas y datos extra.

### Errores o casos especiales:

- Si Open-Meteo no devuelve resultados, devuelve `null`.
- Si la API externa falla, `fetchJson` lanza error y el endpoint puede responder `502`.

## Funcion ampliada: `getCountryInfo`

### Ubicacion:

```
src/back/v1/citys-stats.js
```

### Proposito general:

Pedir datos nacionales a REST Countries.

### Parametros:

- `country` : pais a buscar.

### Devuelve:

Objeto con nombre, capital, region, poblacion, area, codigos `cca2` / `cca3` , bandera y mapa.

### Llama a:

- `cleanSearchTerm`
- `fetchJson`

### La llaman:

- Endpoint `/integrations/country/:country`
- `buildIntegratedCityBase`

### Importancia:

REST Countries aporta el codigo ISO3 necesario para consultar World Bank.

### Explicacion para principiantes:

Primero averiguamos bien el pais y su codigo internacional. Despues usamos ese codigo para otras fuentes.

## Funcion ampliada: `getWorldBankPopulations`

### Ubicacion:

```
src/back/v1/citys-stats.js
```

### Proposito general:

Consultar poblacion de varios paises en World Bank de una sola vez.

### Parametros:

- `countryCodes` : array de codigos ISO3.

### Devuelve:

`Map` con codigo ISO3 como clave y dato World Bank como valor.

### Llama a:

- `worldBankPopulationCache`

- `fetchJson`
- `normalizeWorldBankRow`

**La llama:**

- Handler `/integrations/summary`

**Explicacion bloque por bloque:**

1. Limpia codigos y elimina duplicados.
2. Revisa cuales ya estan en cache.
3. Pide solo los que faltan.
4. Normaliza filas World Bank.
5. Guarda resultados en cache.
6. Devuelve un `Map` con los datos disponibles.

**Explicacion para principiantes:**

En vez de pedir pais por pais, agrupa la consulta. Eso reduce tiempo y llamadas externas.

**Importancia:**

Mejora rendimiento de integraciones.

**Funcion ampliada:** `buildIntegratedCityBase`**Ubicacion:**

```
src/back/v1/citys-stats.js
```

**Proposito general:**

Preparar datos externos base para un pais/ciudad local.

**Parametros:**

- `item`: resumen local por pais generado por `buildCityCountrySummaries`.

**Devuelve:**

Objeto con:

- `item`
- `geocodingResult`
- `countryResult`

**Llama a:**

- `safeExternal`
- `getGeocoding`
- `getCountryInfo`
- `Promise.all`



**La llama:**

- Handler `/integrations/summary`.

**Explicacion para principiantes:**

Para cada pais de nuestra tabla, pide en paralelo la ubicacion de su ciudad principal y la ficha del pais.

**Funcion ampliada:** `buildIntegratedCity`**Ubicacion:**

```
src/back/v1/citys-stats.js
```

**Proposito general:**

Construir el objeto final que mezcla datos locales y externos.

**Parametros:**

- `base`: resultado de `buildIntegratedCityBase`.
- `worldBankByCode`: mapa con datos World Bank.
- `worldBankBatchError`: error si World Bank fallo por lotes.
- `studentApis`: mapas agregados de APIs SOS externas.

**Devuelve:**

Objeto integrado con datos de:

- `citys-stats`
- Open-Meteo
- REST Countries
- World Bank
- SOS2526-25
- SOS2526-19
- SOS2526-26
- SOS2526-30

**Llama a:**

- `normalizeCountryKey`
- Mapas `touristByCountry`, `earthquakesByCountry`, `fifaByCountry`, `esportsByCountry`

**La llama:**

- Handler `/integrations/summary`.

**Explicacion bloque por bloque:**

1. Obtiene codigo ISO3 desde REST Countries.
2. Decide si hay dato World Bank, error o codigo no disponible.
3. Normaliza pais local a `countryKey`.

4. Busca coincidencias en APIs SOS externas.
5. Junta resultados.
6. Construye `integrationErrors` con fallos parciales.
7. Devuelve el objeto final.

**Explicacion para principiantes:**

Es como montar una ficha final de pais: una parte viene de nuestra base y otras partes vienen de distintas APIs externas.

**Importancia:**

Es el nucleo conceptual de las integraciones LCC.

**Funciones ampliadas: normalizadores de APIs SOS****Ubicacion:**

```
src/back/v1/citys-stats.js
```

**Funciones:**

- `normalizeTouristArrival`
- `normalizeEarthquake`
- `normalizeFifaSquadValue`
- `normalizeEsportsEarning`

**Proposito general:**

Traducir filas de APIs externas a objetos internos consistentes.

**Cuando se ejecutan:**

Despues de descargar cada API SOS externa.

**Parametros:**

- `row`: una fila de una API externa.

**Devuelve:**

- Objeto normalizado si la fila tiene datos suficientes.
- `null` si faltan campos clave.

**Llaman a:**

- `readFiniteNumber`
- `countryFromIso3` en el caso de terremotos.

**La llaman:**

- `getTouristArrivals`
- `getEarthquakes`
- `getFifaSquadValues`

- `getEsportsEarnings`

### Explicacion para principiantes:

Cada API externa habla su propio idioma. Los normalizadores traducen esos datos al idioma del proyecto.

### Ejemplos de traduccion:

- Turismo: suma aire, agua y tierra en `totalArrivals`.
- Terremotos: convierte ISO3 a pais y extrae severidad.
- FIFA: extrae valor de mercado, plantilla y anio.
- eSports: extrae premios, jugadores, torneos, juego y genero.

## Funcion ampliada: `buildUrl`

### Ubicacion:

```
frontend-group/src/services/citysStatsApi.js
```

### Proposito general:

Construir URLs seguras para la API LCC v2.

### Parametros:

- `path`: parte opcional despues de `/api/v2/citys-stats`.
- `query`: objeto con filtros opcionales.

### Devuelve:

String con URL completa.

### Llama a:

- `new URL`
- `Object.entries`
- `url.searchParams.set`

### La llaman:

- `getAllCitysStats`
- `createCityStat`
- `deleteAllCitysStats`
- `loadInitialCitysStats`
- `deleteCityStat`
- `getOneCityStat`
- `updateCityStat`

### Explicacion para principiantes:

Sirve para no montar URLs a mano. Si hay filtros, los anade correctamente con `?` y `&`.

### Ejemplo:

```
buildUrl("", { country: "china", limit: 1 })
```

Resultado conceptual:

```
http://localhost:10000/api/v2/citys-stats?country=china&limit=1
```

## Funcion ampliada: `friendlyApiMessage`

**Ubicacion:**

```
frontend-group/src/services/citysStatsApi.js
```

**Proposito general:**

Traducir errores tecnicos del backend a mensajes entendibles para el usuario.

**Parametros:**

- `status`: codigo HTTP.
- `rawMessage`: mensaje tecnico recibido del backend.

**Devuelve:**

Texto legible en castellano.

**La llama:**

- `handleResponse`.

**Explicacion para principiantes:**

Si el backend dice `Invalid sort field`, esta funcion lo convierte en "La opcion elegida para ordenar no es valida".

**Importancia:**

Separa contrato tecnico de experiencia de usuario.

## Funcion ampliada: `handleResponse` del service CRUD

**Ubicacion:**

```
frontend-group/src/services/citysStatsApi.js
```

**Proposito general:**

Convertir una respuesta HTTP en datos o en un error controlado.

**Parametros:**

- `response`: objeto `Response` devuelto por `fetch`.

**Devuelve:**

- Datos JSON.
- `null` si la respuesta es `204`.
- Lanza `Error` si HTTP no es correcto.

**Llama a:**

- `response.text`
- `JSON.parse`
- `friendlyApiMessage`

**La llaman:**

- Todos los metodos exportados del service CRUD.

**Explicacion bloque por bloque:**

1. Si es `204`, no intenta leer JSON.
2. Lee el cuerpo como texto.
3. Intenta parsear JSON.
4. Si la respuesta es error, lanza `Error`.
5. Si todo va bien, devuelve datos.

**Explicacion para principiantes:**

Hace de traductor entre HTTP y el componente Svelte.

**Funcion ampliada:** `getAllCityStats`**Ubicacion:**

```
frontend-group/src/services/citysStatsApi.js
```

**Proposito general:**

Obtener ciudades desde la API v2.

**Parametros:**

- `query`: objeto opcional con filtros.

**Devuelve:**

Array de ciudades.

**Llama a:**

- `buildUrl`
- `fetch`
- `handleResponse`

**La llaman:**

- `refreshList`

- `loadAnalytics` de grafico individual.
- `loadMapData` .
- `loadAnalytics` del widget grupal.

**Explicacion para principiantes:**

Es la funcion que trae ciudades al frontend.

**Funcion ampliada:** `validateCityStatForm`**Ubicacion:**

```
frontend-group/src/routes/citys-stats/+page.svelte
```

**Proposito general:**

Validar los datos del formulario de creacion antes de enviarlos.

**Parametros:**

- `form`: objeto con campos enlazados a inputs.

**Devuelve:**

Payload limpio para `createCityStat` .

**Llama a:**

- `parsePositiveInteger`
- `String`
- `trim`

**La llama:**

- `handleCreate` .

**Errores:**

Lanza `Error` si:

- Falta ciudad.
- Falta pais.
- La poblacion no es entero positivo.

**Explicacion para principiantes:**

Evita hacer una llamada al servidor si el formulario ya esta mal en el navegador.

**Funcion ampliada:** `buildSearchQuery`**Ubicacion:**

```
frontend-group/src/routes/citys-stats/+page.svelte
```

**Proposito general:**

Convertir el formulario de busqueda en query params para la API.

**Parametros:**

No recibe parametros directos. Usa el estado reactivo `searchForm`.

**Devuelve:**

Objeto con los filtros activos.

**Llama a:**

- `parseOptionalPositiveInteger`
- `parseOptionalNonNegativeInteger`
- `Object.keys`

**La llama:**

- `handleSearch`.

**Explicacion bloque por bloque:**

1. Lee `q`, `city`, `country`, poblacion, `sort`, `limit` y `offset`.
2. Convierte numeros opcionales.
3. Elimina claves vacias.
4. Devuelve solo parametros reales.

**Explicacion para principiantes:**

Si el usuario solo rellena pais, la API solo recibe `country`. No manda campos vacios.

**Funcion ampliada: `refreshList`****Ubicacion:**

```
frontend-group/src/routes/citys-stats/+page.svelte
```

**Proposito general:**

Recargar la tabla de ciudades.

**Parametros:**

- `query`: filtros que se aplican. Por defecto usa `activeQuery`.
- `successMessage`: mensaje opcional para mostrar.

**Devuelve:**

Array de ciudades cargadas o array vacio si falla.

**Llama a:**

- `getAllCitysStats`

**La llaman:**

- `onMount`
- `handleSearch`
- `handleResetSearch`
- `handleCreate`
- `handleLoadInitialData`
- `handleDeleteAll`
- `handleDeleteOne`

**Explicacion bloque por bloque:**

1. Activa `loading`.
2. Limpia error.
3. Pide datos al service.
4. Actualiza `citysStats`.
5. Guarda `activeQuery`.
6. Muestra mensaje si procede.
7. Si falla, vacia tabla y muestra error.
8. Desactiva `loading`.

**Importancia:**

Es el centro de sincronizacion entre pantalla y backend.

**Funcion ampliada:** `handleCreate`**Ubicacion:**

```
frontend-group/src/routes/citys-stats/+page.svelte
```

**Proposito general:**

Gestionar el evento de crear ciudad desde la interfaz.

**Cuando se ejecuta:**

Al enviar el formulario "Crear nuevo registro".

**Parametros:**

No recibe parametros. Usa `createForm`.

**Devuelve:**

No devuelve un valor relevante. Actualiza estado visual.

**Llama a:**

- `clearFeedback`
- `validateCityStatForm`
- `createCityStat`



- `refreshList`

**Explicacion para principiantes:**

Es la funcion que conecta el boton "Guardar registro" con la API.

**Errores:**

Si falla validacion o backend, guarda mensaje en `error`.

**Funcion ampliada: `openEdit`****Ubicacion:**

```
frontend-group/src/routes/citys-stats/+page.svelte
```

**Proposito general:**

Navegar a la pantalla de edicion de una ciudad.

**Parametros:**

- `city`: ciudad.
- `country`: pais.

**Devuelve:**

No devuelve nada. Cambia la URL.

**Llama a:**

- `navigate`
- `encodeURIComponent`

**La llaman:**

- Boton `Editar` de cada fila.

**Explicacion para principiantes:**

Construye una URL con la ciudad y el pais seleccionados y cambia de pantalla sin recargar toda la aplicacion.

**Funcion ampliada: `loadResource`****Ubicacion:**

```
frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte
```

**Proposito general:**

Cargar el registro que se va a editar.

**Cuando se ejecuta:**

En `onMount` de la pantalla de edicion.

**Parametros:**

No recibe parametros directos. Usa `params.city` y `params.country`.

**Devuelve:**

No devuelve valor relevante. Rellena `form` y `originalKey`.

**Llama a:**

- `getOneCityStat`

**Errores:**

Si no encuentra el registro, muestra error.

**Explicacion para principiantes:**

Cuando abres la pantalla de editar, esta funcion pide al backend los datos actuales para rellenar el formulario.

**Funcion ampliada: `isSameResource`****Ubicacion:**

```
frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte
```

**Proposito general:**

Saber si el usuario ha cambiado la clave compuesta del recurso.

**Parametros:**

- `payload`: datos validados del formulario.

**Devuelve:**

- `true` si `city` y `country` siguen siendo los originales.
- `false` si alguno cambio.

**La llama:**

- `handleUpdate`.

**Explicacion sencilla:**

Si solo cambia poblacion, se hace `PUT`. Si cambia ciudad o pais, el recurso ya tiene otra URL y se hace crear + borrar.

**Funcion ampliada: `handleUpdate`****Ubicacion:**

```
frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte
```

**Proposito general:**

Guardar cambios de una ciudad.

**Cuando se ejecuta:**

Al enviar el formulario de edicion.

**Llama a:**

- `clearFeedback`
- `validateForm`
- `isSameResource`
- `updateCityStat`
- `createCityStat`
- `deleteCityStat`
- `updateRoute`

**Dos caminos:**

1. Misma clave: `PUT`.
2. Nueva clave: `POST` del nuevo recurso y `DELETE` del antiguo.

**Explicacion para principiantes:**

Si editas solo la poblacion, actualiza el registro. Si cambias ciudad o pais, crea una ficha nueva y elimina la anterior porque la identidad del recurso ha cambiado.

**Importancia:**

Resuelve de forma practica el problema de editar claves compuestas.

**Funcion ampliada: `buildMetrics`****Ubicacion:**

```
frontend-group/src/routes/analytics/+page.svelte
```

**Proposito general:**

Construir las metricas del widget grupal.

**Parametros:**

- `citysStats`: array de ciudades.
- `disasters`: array de desastres.
- `wines`: array de vinos.

**Devuelve:**

Array con:

- nombre del recurso,
- numero de registros,
- nombre del indicador,

- indicador bruto,
- indice normalizado de 0 a 100.

**Llama a:**

- `sum`
- `Math.max`
- `map`

**La llama:**

- `loadAnalytics` de `/analytics`.

**Explicacion para principiantes:**

Como cada recurso mide cosas distintas, esta funcion calcula un indice comun para poder ponerlos juntos en una grafica sin confundir unidades.

**Funcion ampliada: `renderChart` del analytics grupal****Ubicacion:**

```
frontend-group/src/routes/analytics/+page.svelte
```

**Proposito general:**

Pintar el widget grupal con Highcharts.

**Cuando se ejecuta:**

Despues de cargar datos y esperar a que exista el contenedor.

**Parametros:**

No recibe parametros directos. Usa `metrics`, `chartContainer` y `Highcharts`.

**Devuelve:**

No devuelve valor. Guarda el grafico en `chart`.

**Llama a:**

- `chart?.destroy`
- `Highcharts.chart`

**Explicacion para principiantes:**

Convierte el array de metricas en columnas visibles.

**Caso especial:**

Antes de crear una grafica nueva destruye la anterior para no duplicar instancias.

**Funcion ampliada: `keyFor`****Ubicacion:**

```
frontend-group/src/routes/analytics/citys-stats/map/+page.svelte
```

**Proposito general:**

Crear la clave que relaciona una ciudad con sus coordenadas.

**Parametros:**

- `item`: registro de `citys-stats`.

**Devuelve:**

Texto con formato:

```
city|country
```

**La usan:**

- Calculo reactivo `geolocated`.
- Calculo reactivo `missing`.

**Explicacion sencilla:**

Si la API devuelve `tokyo` y `japan`, esta funcion genera `tokyo|japan`, que se busca en el objeto `coordinates`.

**Funcion ampliada: `colorFor`****Ubicacion:**

```
frontend-group/src/routes/analytics/citys-stats/map/+page.svelte
```

**Proposito general:**

Asignar color al marcador segun poblacion.

**Parametros:**

- `population`: poblacion numerica.

**Devuelve:**

Color hexadecimal.

**Explicacion para principiantes:**

Cuanto mayor es la ciudad, mas intenso cambia el color. Ayuda a leer el mapa rapidamente.

**Funcion ampliada: `radiusFor`****Ubicacion:**

```
frontend-group/src/routes/analytics/citys-stats/map/+page.svelte
```

**Proposito general:**

Calcular el tamaño del marcador.

**Parametros:**

- `population`: población de la ciudad.

**Devuelve:**

Número usado como radio del marcador.

**Llama a:**

- `Math.sqrt`

**Usa estado reactivo:**

- `maxPopulation`

**Explicación sencilla:**

Usa raíz cuadrada para que las ciudades grandes se vean mayores, pero sin que ocupen todo el mapa.

**Función ampliada:** `renderMap`**Ubicación:**

```
frontend-group/src/routes/analytics/citys-stats/map/+page.svelte
```

**Propósito general:**

Crear el mapa Highcharts con países y marcadores de ciudades.

**Cuando se ejecuta:**

Después de cargar `citys-stats`, cargar Highcharts Maps y esperar a que exista el contenedor.

**Parametros:**

No recibe parámetros directos. Usa:

- `mapContainer`
- `Highcharts`
- `points`
- `worldMap`
- `minPopulation`
- `maxPopulation`

**Devuelve:**

No devuelve valor. Guarda instancia en `mapChart`.

**Llama a:**

- `mapChart?.destroy`
- `Highcharts.mapChart`
- `removeCityMarkerClip`

- `selectPoint`

**Explicacion bloque por bloque:**

1. Si no hay contenedor, Highcharts o puntos, no hace nada.
2. Destruye mapa previo.
3. Crea mapa mundial.
4. Configura navegacion, tooltip y accesibilidad.
5. Dibuja serie de paises.
6. Dibuja serie de puntos `mappoint`.
7. Cada punto incluye latitud, longitud, color, radio y datos custom.

**Explicacion para principiantes:**

Transforma cada ciudad con coordenadas en un punto sobre el mapa.

**Funcion ampliada: `safeLoad`****Ubicacion:**

```
frontend-group/src/routes/integrations/citys-stats/+page.svelte
```

**Proposito general:**

Capturar errores de llamadas frontend a integraciones.

**Parametros:**

- `task`: funcion asincrona que carga una fuente.

**Devuelve:**

```
{ data, error }
```

**La llama:**

- `loadIntegrations`.

**Explicacion para principiantes:**

Hace en el frontend lo mismo que `safeExternal` en el backend: si una carga falla, se guarda el error y la pantalla puede seguir.

**Funcion ampliada: `createChart`****Ubicacion:**

```
frontend-group/src/routes/integrations/citys-stats/+page.svelte
```

**Proposito general:**

Crear graficas de integraciones con configuracion comun.

**Parametros:**

- `container`: elemento HTML donde pintar.
- `config`: configuracion Highcharts.

**Devuelve:**

No devuelve valor. Inserta la instancia en `integrationCharts`.

**Llama a:**

- `Highcharts.chart`

**La llaman:**

- `renderGeocodingChart`
- `renderCountryChart`
- `renderWorldBankChart`
- `renderTourismChart`
- `renderEarthquakeChart`
- `renderFifaChart`
- `renderEsportsChart`

**Explicacion sencilla:**

Evita repetir la misma configuracion base en los siete widgets.

**Funcion ampliada:** `destroyIntegrationCharts`**Ubicacion:**

```
frontend-group/src/routes/integrations/citys-stats/+page.svelte
```

**Proposito general:**

Destruir graficas antiguas antes de recargar o salir.

**Cuando se ejecuta:**

- Antes de renderizar integraciones de nuevo.
- En `onDestroy`.

**Explicacion para principiantes:**

Limpia graficas anteriores para no duplicarlas ni dejar memoria ocupada.

**Funcion ampliada:** `loadIntegrations`**Ubicacion:**

```
frontend-group/src/routes/integrations/citys-stats/+page.svelte
```

**Proposito general:**



Cargar todos los datos necesarios para la vista de integraciones LCC.

**Cuando se ejecuta:**

- Al abrir `/integrations/citys-stats`.
- Al cambiar el selector de paises.
- Al pulsar actualizar.

**Parametros:**

No recibe parametros directos. Usa `selectedLimit`.

**Devuelve:**

No devuelve valor relevante. Actualiza estado y renderiza graficas.

**Llama a:**

- `destroyIntegrationCharts`
- `getCountrySummaries`
- `loadInitialCitysStats`
- `getGeocoding`
- `getCountryInfo`
- `getWorldBankPopulation`
- `getSosTouristArrivals`
- `getSosEarthquakes`
- `getSosFifaSquadValues`
- `getSosEsportsEarnings`
- `safeLoad`
- `collectError`
- `topCountries`
- `loadHighcharts`
- `renderIntegrationCharts`

**Explicacion bloque por bloque:**

1. Activa carga y limpia errores.
2. Destruye graficas anteriores.
3. Pide resumen local por pais.
4. Si no hay datos, intenta cargar datos iniciales.
5. Pide Open-Meteo por ciudad principal.
6. Pide REST Countries por pais.
7. Pide World Bank con ISO3 obtenido.
8. Pide APIs SOS externas.
9. Construye listas top para widgets.
10. Guarda errores parciales.
11. Carga Highcharts y modulos.
12. Renderiza todos los widgets.

**Explicacion para principiantes:**

Es el director de orquesta de la pantalla de integraciones. Coordina todas las fuentes y, cuando estan listas, manda pintar las graficas.

**Errores o casos especiales:**

- Si no hay datos locales, carga iniciales.
- Si una API falla, se guarda aviso.
- Si falla la carga general, muestra error principal.

**Importancia:**

Es la funcion mas importante del frontend de integraciones LCC.

**Funcion ampliada:** `renderIntegrationCharts`**Ubicacion:**

```
frontend-group/src/routes/integrations/citys-stats/+page.svelte
```

**Proposito general:**

Lanzar el pintado de los siete widgets de integracion.

**Llama a:**

- `destroyIntegrationCharts`
- `renderGeocodingChart`
- `renderCountryChart`
- `renderWorldBankChart`
- `renderTourismChart`
- `renderEarthquakeChart`
- `renderFifaChart`
- `renderEsportsChart`

**Explicacion sencilla:**

Agrupa en un solo sitio el renderizado de todos los graficos para que `loadIntegrations` no tenga siete llamadas sueltas mezcladas con carga de datos.

**Mini respuestas para funciones en defensa**

Si preguntan por validacion:

`hasExactCityFields` comprueba estructura y `normalizeCityStat` limpia tipos y valores. Una valida la forma; la otra valida el contenido.

Si preguntan por filtros:

El handler `GET /api/v2/citys-stats` lee todos los datos, aplica filtros exactos, busqueda libre, ordenacion y paginacion en ese orden.

Si preguntan por integraciones:

`buildCityCountrySummaries` convierte ciudades en paises, `fetchJson` consulta APIs externas, `safeExternal` evita caidas totales y `buildIntegratedCity` monta el objeto final.

Si preguntan por frontend:

Los services encapsulan `fetch`; las pantallas no conocen todos los detalles HTTP. Por ejemplo, `refreshList` solo llama a `getAllCitysStats` y actualiza estado visual.

Si preguntan por mapa:

`keyFor` relaciona API y coordenadas, `colorFor` y `radiusFor` convierten poblacion en estilo visual, y `renderMap` crea el mapa Highcharts.

## Antipilladas sobre funciones

Esta es la seccion para no quedarse bloqueado si el profesor senala una funcion concreta del codigo y pregunta "explicame esta".

Regla para cualquier funcion:

1. Que capa es? backend, service, pantalla, grafica o integracion.
2. Quien la llama?
3. Que recibe?
4. Que devuelve?
5. Cambia estado o solo calcula?
6. Es sincronica, async/await o callback?
7. Que errores evita?
8. Que pasaria si la quito o la cambio mal?

Frase comodin:

Esta funcion tiene una responsabilidad concreta dentro de su capa. No intenta hacerlo todo: recibe datos, los valida/transforma o llama a otra capa, y devuelve un resultado o actualiza el estado de la pantalla.

## Preguntas trampa generales

| Pregunta                               | Respuesta segura                                                                                                                                                      |
|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| "Esta funcion devuelve algo?"          | Si es de calculo, suele devolver un valor. Si es de UI, muchas veces actualiza estado. Si es handler Express, responde con <code>res.status(...).json(...)</code> .   |
| "Por que es <code>async</code> ?"      | Porque dentro espera una peticion HTTP, una carga dinamica de modulo o una operacion que tarda.                                                                       |
| "Que pasa si falla?"                   | Se captura con <code>try/catch</code> , <code>handleResponse</code> , <code>safeLoad</code> , <code>safeExternal</code> o callback <code>err</code> , segun la capa.  |
| "Donde se llama?"                      | Buscar con <code>rg "nombreFuncion"</code> . En Svelte muchas se llaman desde <code>onMount</code> , eventos de botones o formularios.                                |
| "Cambia la base de datos?"             | Solo handlers backend con <code>db.insert</code> , <code>db.update</code> o <code>db.remove</code> . Las funciones frontend no tocan NeDB directamente.               |
| "Cambia la pantalla?"                  | Las funciones Svelte cambian variables reactivas como <code>cityStats</code> , <code>loading</code> , <code>error</code> , <code>message</code> o arrays de graficas. |
| "Por que no lo hace todo una funcion?" | Porque separar responsabilidades hace que sea mas facil probar, modificar y defender.                                                                                 |

## Funciones backend CRUD v2 que pueden senalar

| Funcion/bloque                                       | Como defenderla en una frase                                                                                         | Riesgo si se cambia mal                             |
|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| <code>removeDatabaseId</code>                        | Quita <code>_id</code> interno de NeDB antes de responder al cliente.                                                | Exponer detalles internos de la base.               |
| <code>hasExactCityFields</code>                      | Comprueba que el body tenga exactamente <code>city</code> , <code>country</code> y <code>un_2025_population</code> . | Aceptar datos incompletos o campos basura.          |
| <code>normalizeCityStat</code>                       | Limpia strings y convierte poblacion a numero antes de guardar.                                                      | Guardar formatos inconsistentes.                    |
| Handler <code>GET</code> coleccion                   | Lee datos, filtra, busca, ordena y pagina.                                                                           | Devolver resultados incorrectos o romper queries.   |
| Handler <code>GET</code> <code>:city/:country</code> | Busca un recurso concreto por clave compuesta.                                                                       | No distinguir entre coleccion y recurso individual. |
| Handler <code>POST</code>                            | Valida, comprueba duplicado e inserta.                                                                               | Crear duplicados o aceptar body invalido.           |
| Handler <code>PUT</code>                             | Valida URL/body, comprueba existencia y actualiza.                                                                   | Modificar un recurso distinto al de la URL.         |
| Handler <code>DELETE</code>                          | Borra coleccion o recurso concreto y responde <code>204</code> .                                                     | Borrar lo que no toca o devolver codigo incorrecto. |

Frase para handlers Express:

Un handler Express no "devuelve" como una funcion normal; termina enviando una respuesta HTTP con `res.status`, `res.json`, `res.sendStatus` o `res.redirect`.

## Funciones service CRUD que pueden senalar

| Funcion                          | Quien la llama                                                                          | Que hace realmente                                                                                       |
|----------------------------------|-----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>buildUrl</code>            | Todas las funciones del service                                                         | Une base API, path y query params sin construir URLs a mano en pantallas.                                |
| <code>encodePathValue</code>     | <code>getOneCityStat</code> , <code>updateCityStat</code> , <code>deleteCityStat</code> | Codifica <code>city</code> y <code>country</code> para que espacios o caracteres raros no rompan la URL. |
| <code>friendlyApiResponse</code> | <code>handleResponse</code>                                                             | Traduce codigos HTTP a mensajes entendibles para usuario.                                                |
| <code>handleResponse</code>      | Todas las llamadas <code>fetch</code>                                                   | Lee la respuesta, detecta errores HTTP y devuelve JSON si todo va bien.                                  |
| <code>getAllCityStats</code>     | CRUD, analytics, mapa                                                                   | Hace <code>GET</code> con filtros opcionales.                                                            |
| <code>createCityStat</code>      | <code>handleCreate</code> , edicion con nueva clave                                     | Hace <code>POST</code> con JSON.                                                                         |
| <code>updateCityStat</code>      | <code>handleUpdate</code> sin cambiar clave                                             | Hace <code>PUT</code> con JSON.                                                                          |
| <code>deleteCityStat</code>      | Borrar uno y edicion con nueva clave                                                    | Hace <code>DELETE</code> sobre recurso concreto.                                                         |

Frase para services:

El service es el traductor entre Svelte y HTTP. La pantalla pide "crea ciudad" y el service sabe que eso significa `POST /api/v2/citys-stats` con JSON.

## Funciones de pantalla CRUD que pueden senalar

| Funcion                                      | Tipo           | Defensa rapida                                                         |
|----------------------------------------------|----------------|------------------------------------------------------------------------|
| <code>emptycreateForm</code>                 | Estado inicial | Devuelve un formulario limpio para crear.                              |
| <code>emptySearchForm</code>                 | Estado inicial | Devuelve filtros vacios.                                               |
| <code>clearFeedback</code>                   | UI             | Limpia errores y mensajes antes de una accion nueva.                   |
| <code>parsePositiveInteger</code>            | Validacion     | Convierte poblacion a entero positivo.                                 |
| <code>parseOptionalNonNegativeInteger</code> | Validacion     | Valida <code>offset</code> opcional.                                   |
| <code>parseOptionalPositiveInteger</code>    | Validacion     | Valida <code>limit</code> opcional.                                    |
| <code>validateCityStatForm</code>            | Validacion     | Evita enviar formularios claramente invalidos.                         |
| <code>buildSearchQuery</code>                | Transformacion | Convierte formulario de busqueda en query params limpios.              |
| <code>refreshList</code>                     | Async + estado | Pide datos, actualiza tabla y controla <code>loading/error</code> .    |
| <code>handleSearch</code>                    | Evento usuario | Construye query, espera lista y muestra mensaje segun resultados.      |
| <code>handleCreate</code>                    | Evento usuario | Valida, hace <code>POST</code> , limpia formulario y recarga lista.    |
| <code>handleLoadInitialData</code>           | Evento usuario | Pide datos iniciales y recarga tabla.                                  |
| <code>handleDeleteAll</code>                 | Evento usuario | Borra todo y recarga.                                                  |
| <code>handleDeleteOne</code>                 | Evento usuario | Borra un registro concreto y recarga.                                  |
| <code>openEdit</code>                        | Navegacion     | Cambia a la ruta de edicion con <code>city/country</code> codificados. |

Pillada probable:

"Por que `handleCreate` recarga con `refreshList` si ya tiene `created`?"

Respuesta:

Porque la fuente de verdad es el backend. Recargar evita que la tabla quede desincronizada con filtros, ordenacion o cambios reales de la base.

## Funciones de edicion que pueden senalar

| Funcion                     | Defensa rapida                                                                          |
|-----------------------------|-----------------------------------------------------------------------------------------|
| <code>loadResource</code>   | Al montar la pantalla, pide el registro original y rellena el formulario.               |
| <code>isSameResource</code> | Comprueba si la clave compuesta <code>city + country</code> sigue igual.                |
| <code>updateRoute</code>    | Si cambia la clave, actualiza la URL para que apunte al nuevo recurso.                  |
| <code>handleUpdate</code>   | Decide entre <code>PUT</code> normal o crear nuevo + borrar antiguo si cambia la clave. |

Pillada probable:

"Por que si cambia la ciudad no haces simplemente PUT ?"

Respuesta:

Porque `city + country` forma parte de la identidad del recurso. El backend exige que URL y body coincidan. Si cambia la clave, creo el nuevo recurso y luego borro el anterior para no violar el contrato REST.

Funciones de analytics y mapa que pueden senalar

| Funcion                           | Defensa rapida                                                           |                                 |
|-----------------------------------|--------------------------------------------------------------------------|---------------------------------|
| <code>labelFor</code>             | Crea etiquetas legibles para la grafica individual.                      |                                 |
| <code>loadHighcharts</code>       | Carga Highcharts solo cuando la pantalla lo necesita.                    |                                 |
| <code>renderChart</code>          | Destruye grafica anterior y pinta la nueva.                              |                                 |
| <code>loadAnalytics</code>        | Pide datos, espera DOM con <code>tick</code> , carga Highcharts y pinta. |                                 |
| <code>keyFor</code>               | Construye clave estable `city`                                           | country` para relacionar datos. |
| <code>titleCase</code>            | Mejora formato visual de nombres.                                        |                                 |
| <code>colorFor</code>             | Convierte poblacion en color.                                            |                                 |
| <code>radiusFor</code>            | Convierte poblacion en tamano de burbuja.                                |                                 |
| <code>selectPoint</code>          | Guarda el punto seleccionado por el usuario.                             |                                 |
| <code>removeCityMarkerClip</code> | Ajuste visual para que los marcadores no queden recortados.              |                                 |
| <code>loadHighchartsMap</code>    | Carga Highcharts Maps y modulos necesarios.                              |                                 |
| <code>renderMap</code>            | Crea el mapa con <code>mapChart</code> y serie <code>mappoint</code> .   |                                 |
| <code>loadMapData</code>          | Carga ciudades, prepara puntos, espera DOM y pinta mapa.                 |                                 |

Pillada probable:

"Por que usas `tick` antes de pintar?"

Respuesta:

Porque Highcharts necesita que el contenedor exista en el DOM. `tick` espera a que Svelte haya actualizado la pantalla antes de llamar a `Highcharts.chart` o `Highcharts.mapChart` .

## Funciones de integraciones UI que pueden senalar

| Funcion                               | Defensa rapida                                                           |
|---------------------------------------|--------------------------------------------------------------------------|
| <code>numberOrNull</code>             | Convierte valores externos a numero o <code>null</code> .                |
| <code>topCountries</code>             | Ordena y limita resultados externos por metrica.                         |
| <code>normalizeCountryKey</code>      | Normaliza paises para cruzar datos aunque vengan con formatos distintos. |
| <code>localCountryIndex</code>        | Crea un indice rapido de paises locales.                                 |
| <code>findLocalCountry</code>         | Busca si un pais externo coincide con uno local.                         |
| <code>localAveragePopulation</code>   | Calcula media local para comparaciones.                                  |
| <code>localPopulationFor</code>       | Obtiene poblacion local de un pais cruzado.                              |
| <code>combinedCountryRows</code>      | Junta datos externos con datos locales.                                  |
| <code>normalizedIndex</code>          | Convierte magnitudes distintas a escala comparable.                      |
| <code>collectError</code>             | Guarda errores parciales de integracion con contexto.                    |
| <code>safeLoad</code>                 | Evita que una peticion fallida rompa toda la pantalla.                   |
| <code>loadHighcharts</code>           | Carga Highcharts y modulos avanzados.                                    |
| <code>destroyIntegrationCharts</code> | Limpia graficas anteriores antes de repintar.                            |
| <code>createChart</code>              | Crea una grafica y guarda su instancia para poder destruirla.            |
| <code>render...Chart</code>           | Cada funcion pinta un widget concreto.                                   |
| <code>renderIntegrationCharts</code>  | Llama ordenadamente a los siete renderizados.                            |
| <code>loadIntegrations</code>         | Coordina toda la carga asincrona de datos y graficas.                    |

Pillada probable:

"Por que `safeLoad` devuelve `{ data, error }` en vez de lanzar el error?"

Respuesta:

Porque en integraciones interesa mostrar datos parciales. Si Open-Meteo falla, no quiero perder REST Countries, World Bank o las APIs SOS. Guardo el error y sigo.



## Funciones backend de integraciones que pueden senalar

| Funcion                                | Defensa rapida                                             |
|----------------------------------------|------------------------------------------------------------|
| <code>cleanSearchTerm</code>           | Limpia busquedas antes de comparar.                        |
| <code>normalizeCountryKey</code>       | Hace comparables nombres de pais.                          |
| <code>countryFromIso3</code>           | Convierte ISO3 a pais legible cuando se puede.             |
| <code>readFiniteNumber</code>          | Evita usar numeros invalidos de APIs externas.             |
| <code>asArray</code>                   | Convierte respuestas externas variables en arrays seguros. |
| <code>parseLimit</code>                | Limita cuantos resultados se piden o muestran.             |
| <code>findAllCityStats</code>          | Lee todos los registros locales como Promise.              |
| <code>buildCityCountrySummaries</code> | Agrupar ciudades por pais para integrar mejor.             |
| <code>fetchJson</code>                 | Hace fetch externo con timeout y validacion HTTP/JSON.     |
| <code>getGeocoding</code>              | Pide coordenadas de ciudad a Open-Meteo.                   |
| <code>getCountryInfo</code>            | Pide ficha de pais a REST Countries.                       |
| <code>getWorldBankPopulation(s)</code> | Pide poblacion World Bank, con cache/lote.                 |
| <code>safeExternal</code>              | Convierte fallo externo en error parcial controlado.       |
| <code>buildIntegratedCityBase</code>   | Junta ciudad local con geocoding y pais.                   |
| <code>buildIntegratedCity</code>       | Monta el objeto final enriquecido.                         |
| <code>normalize... externo</code>      | Limpia filas de APIs SOS externas.                         |
| <code>build...ByCountry</code>         | Agrega datos externos por pais.                            |

Pillada probable:

"Por que hay tanta normalizacion en integraciones?"

Respuesta:

Porque cada API externa usa nombres, campos y formatos distintos. Normalizar evita que el frontend tenga que conocer todos esos detalles.

## Si no sabes que contestar ante una funcion

Usa este guion de emergencia:

Esta funcion pertenece a la capa de `[backend/service/pantalla/grafica/integracion]`. Se llama desde `[evento, handler o funcion superior]`. Recibe `[parametros]`, prepara o valida `[datos]`, y despues `[devuelve resultado / actualiza estado / responde HTTP]`. Es importante porque evita `[error concreto]` y mantiene separada la responsabilidad de esta capa.

Ejemplo rapido:

`collectError` pertenece a la pantalla de integraciones. Se llama despues de cada carga externa. Recibe una lista, una etiqueta y el resultado de la peticion. Si hay error, lo anade con contexto. Es importante porque permite mostrar fallos parciales sin romper toda la vista.

## Codigo real de funciones clave

Esta seccion existe para estudiar con el codigo delante. No sustituye a los archivos reales, pero evita tener que saltar constantemente entre explicacion y proyecto.

Regla:

```
Funciones pequenas -> codigo completo.  
Funciones largas -> fragmento clave que se defiende en clase.
```

### Backend v2: `removeDatabaseId`

Archivo:

```
src/back/v2/citys-stats.js
```

Codigo real:

```
function removeDatabaseId(doc) {  
  if (!doc) return doc;  
  const { _id, ...rest } = doc;  
  return rest;  
}
```

Como leerlo:

1. Si no hay documento, devuelve lo mismo.
2. Extrae `_id` y deja el resto en `rest`.
3. Devuelve el objeto sin `_id`.

Frase:

`_id` pertenece a NeDB, no al contrato publico de la API.

### Backend v2: `hasExactCityFields`

Codigo real:

```
function hasExactCityFields(body) {  
  if (!body || typeof body !== "object" || Array.isArray(body)) return false;  
  
  const expected = ["city", "country", "un_2025_population"].sort();  
  const keys = Object.keys(body).sort();  
  
  return keys.length === expected.length &&
```

```
keys.every((k, i) => k === expected[i]);
}
```

Como leerlo:

1. Rechaza `null`, valores que no son objeto y arrays.
2. Define los campos exactos esperados.
3. Obtiene las claves reales del body.
4. Compara longitud y nombre de cada clave.

Si te preguntan por que se ordena:

Porque el JSON podria llegar con los campos en distinto orden. Ordenar permite comparar conjuntos de claves, no posicion original.

## Backend v2: `normalizeCityStat`

Codigo real:

```
function normalizeCityStat(body) {
  if (!hasExactCityFields(body)) return null;

  const city = String(body.city).trim().toLowerCase();
  const country = String(body.country).trim().toLowerCase();
  const un_2025_population = Number(body.un_2025_population);

  if (
    !city ||
    !country ||
    !Number.isInteger(un_2025_population) ||
    un_2025_population <= 0
  ) {
    return null;
  }

  return { city, country, un_2025_population };
}
```

Como leerlo:

1. Primero comprueba la estructura exacta.
2. Convierte `city` y `country` a texto limpio y minusculas.
3. Convierte poblacion a numero.
4. Rechaza ciudad vacia, pais vacio, poblacion no entera o menor/igual a cero.
5. Devuelve el objeto limpio.

Frase:

Esta funcion es la puerta de entrada del dato. Si devuelve `null`, el endpoint responde `400`.

## Backend v2: fragmento clave del GET /api/v2/citys-stats

Codigo real resumido:

```
app.get(BASE_API_URL, (req, res) => {
  db.find({}, (err, docs) => {
    if (err) return res.sendStatus(500);

    let result = docs.map(removeDatabaseId);

    if (req.query.city !== undefined) {
      result = result.filter(
        d => d.city === String(req.query.city).trim().toLowerCase()
      );
    }

    if (req.query.country !== undefined) {
      result = result.filter(
        d => d.country === String(req.query.country).trim().toLowerCase()
      );
    }

    if (req.query.q !== undefined) {
      const q = String(req.query.q).trim().toLowerCase();
      result = result.filter(d =>
        d.city.includes(q) || d.country.includes(q)
      );
    }

    if (req.query.sort !== undefined) {
      const sort = String(req.query.sort).trim();
      let field = sort;
      let direction = 1;

      if (sort.startsWith("-")) {
        field = sort.slice(1);
        direction = -1;
      }

      const allowedFields = ["city", "country", "un_2025_population"];

      if (!allowedFields.includes(field)) {
        return res.status(400).json({ error: "Invalid sort field" });
      }

      result.sort((a, b) => {
        if (a[field] < b[field]) return -1 * direction;
        if (a[field] > b[field]) return 1 * direction;
        return 0;
      });
    }

    return res.status(200).json(result.slice(offset, offset + limit));
  });
});
```

Como defenderlo:

El **GET** no solo lista. Lee todo, elimina **\_id**, aplica filtros, busqueda libre, ordenacion permitida y paginacion. Si un parametro no es valido, responde **400**.

Pillada:

En el codigo real tambien se validan **un\_2025\_population**, **offset** y **limit**. Si me senalan esa parte, explico que evita queries numericas invalidas.

## Backend v2: fragmento clave del **POST**

Codigo real:

```
app.post(BASE_API_URL, (req, res) => {
  const item = normalizeCityStat(req.body);

  if (!item) {
    return res.status(400).json({
      error: "JSON body does not match expected structure"
    });
  }

  db.findOne({ city: item.city, country: item.country }, (err, doc) => {
    if (err) return res.sendStatus(500);
    if (doc) return res.status(409).json({ error: "Resource already exists" });

    db.insert(item, (err2, newDoc) => {
      if (err2) return res.sendStatus(500);
      return res.status(201).json(removeDatabaseId(newDoc));
    });
  });
});
```

Orden:

```
normalize -> 400 si mal -> findOne duplicado -> 409 si existe -> insert -> 201
```

Frase:

**POST** crea en la coleccion, por eso la ruta no lleva **city/country**.

## Backend v2: fragmento clave del **PUT**

Codigo real:

```
app.put(`${BASE_API_URL}/${city}/${country}`, (req, res) => {
  const city = req.params.city.trim().toLowerCase();
  const country = req.params.country.trim().toLowerCase();

  const item = normalizeCityStat(req.body);

  if (!item) {
    return res.status(400).json({
```

```

        error: "JSON body does not match expected structure"
    });
}

if (item.city !== city || item.country !== country) {
    return res.status(400).json({ error: "URL and body do not match" });
}

db.findOne({ city, country }, (err, doc) => {
    if (err) return res.sendStatus(500);
    if (!doc) return res.status(404).json({ error: "Resource not found" });

    db.update({ city, country }, item, {}, (err2) => {
        if (err2) return res.sendStatus(500);

        db.findOne({ city, country }, (err3, updated) => {
            if (err3) return res.sendStatus(500);
            return res.status(200).json(removeDatabaseId(updated));
        });
    });
});
});
});
});

```

Pillada:

"Por que compruebas URL y body?"

Respuesta:

Porque `PUT /tokyo/japan` no deberia poder modificar `madrid/spain`. La URL identifica el recurso.

## Service CRUD: `buildUrl`

Archivo:

```
frontend-group/src/services/citysStatsApi.js
```

Codigo real:

```

function buildUrl(path = "", query = {}) {
    const url = new URL(`${CITYS_STATS_API_BASE}${path}`, window.location.origin);

    Object.entries(query).forEach(([key, value]) => {
        if (value === undefined || value === null || value === "") {
            return;
        }

        url.searchParams.set(key, value);
    });

    return url.toString();
}

```

Como leerlo:

1. Crea una URL segura.
2. Recorre filtros.
3. Ignora valores vacios.
4. Anade query params.
5. Devuelve texto para `fetch`.

Frase:

Evita concatenar strings a mano y reduce errores en URLs.

### Service CRUD: `handleResponse`

Codigo real:

```
async function handleResponse(response) {
  if (response.status === 204) {
    return null;
  }

  const text = await response.text();
  let data = null;

  try {
    data = text ? JSON.parse(text) : null;
  } catch {
    data = text;
  }

  if (!response.ok) {
    throw new Error(friendlyApiMessage(response.status, data?.error));
  }

  return data;
}
```

Como leerlo:

1. Si es `204`, no hay cuerpo y devuelve `null`.
2. Lee texto de la respuesta.
3. Intenta convertir a JSON.
4. Si HTTP no es correcto, lanza un error con mensaje amigable.
5. Si todo va bien, devuelve datos.

Frase:

Centraliza el tratamiento de errores HTTP para que cada pantalla no repita la misma logica.

### Service CRUD: `createCityStat` y `updateCityStat`

Codigo real:

```

export async function createCityStat(cityStat) {
  const response = await fetch(buildUrl(), {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(cityStat)
  });

  return handleResponse(response);
}

export async function updateCityStat(city, country, cityStat) {
  const response = await fetch(
    buildUrl(`/${encodeURIComponent(city)}/${encodeURIComponent(country)}`),
    {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(cityStat)
    }
  );

  return handleResponse(response);
}

```

Frase:

La diferencia es que **POST** crea en la coleccion y **PUT** actualiza una URL concreta con **city/country**.

### Pantalla CRUD: **validateCityStatForm**

Archivo:

```
frontend-group/src/routes/citys-stats/+page.svelte
```

Codigo real:

```

function validateCityStatForm(form) {
  const city = String(form.city ?? "").trim();
  const country = String(form.country ?? "").trim();
  const un_2025_population = parsePositiveInteger(
    form.un_2025_population,
    "Poblacion estimada en 2025"
  );

  if (!city) {
    throw new Error("Indique una ciudad.");
  }

  if (!country) {
    throw new Error("Indique un pais.");
  }

  return {
    city,
    country,

```



```

    un_2025_population
  };
}

```

Frase:

Valida en frontend para mejorar la experiencia, pero la validacion importante sigue estando tambien en backend.

## Pantalla CRUD: **buildSearchQuery**

Codigo real:

```

function buildSearchQuery() {
  const query = {
    q: String(searchForm.q ?? "").trim(),
    city: String(searchForm.city ?? "").trim(),
    country: String(searchForm.country ?? "").trim(),
    un_2025_population: parseOptionalPositiveInteger(
      searchForm.un_2025_population,
      "Poblacion exacta"
    ),
    sort: String(searchForm.sort ?? "").trim(),
    limit: parseOptionalNonNegativeInteger(
      searchForm.limit,
      "Numero maximo de resultados"
    ),
    offset: parseOptionalNonNegativeInteger(
      searchForm.offset,
      "Posicion inicial"
    )
  };

  Object.keys(query).forEach((key) => {
    if (query[key] === "" || query[key] === undefined) {
      delete query[key];
    }
  });

  return query;
}

```

Como defenderlo:

Convierte el formulario visual en query params limpios. Si un campo esta vacio, no se envia.

## Pantalla CRUD: **refreshList**

Codigo real:

```

async function refreshList(query = activeQuery, successMessage = "") {
  loading = true;
  error = "";

```

```

try {
  citysStats = await getAllCitysStats(query);
  activeQuery = { ...query };

  if (successMessage) {
    message = successMessage;
  }

  return citysStats;
} catch (e) {
  citysStats = [];
  error = e.message || "No se pudieron cargar los datos de ciudades.";
  return [];
} finally {
  loading = false;
}
}

```

Como leerlo:

1. Activa carga.
2. Espera datos del service.
3. Guarda datos y query activa.
4. Si falla, limpia tabla y muestra error.
5. Siempre desactiva carga en **finally**.

Frase:

**finally** garantiza que el spinner se apague tanto si hay éxito como si hay error.

## Pantalla CRUD: **handleCreate**

Código real:

```

async function handleCreate() {
  clearFeedback();

  try {
    const payload = validateCityStatForm(createForm);
    const created = await createCityStat(payload);

    createForm = emptyCreateForm();
    await refreshList(
      activeQuery,
      `Se ha creado el registro de ${created.city} (${created.country}).`
    );
  } catch (e) {
    error = e.message || "No se pudo crear el registro.";
  }
}

```

Frase:

Valida, hace **POST**, limpia el formulario y recarga desde backend.

### Pantalla editar: **isSameResource** y **handleUpdate**

Archivo:

```
frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte
```

Codigo real:

```
function isSameResource(payload) {
  return (
    payload.city.trim().toLowerCase() === originalKey.city &&
    payload.country.trim().toLowerCase() === originalKey.country
  );
}
```

Fragmento clave de **handleUpdate**:

```
if (isSameResource(payload)) {
  const updated = await updateCityStat(
    originalKey.city,
    originalKey.country,
    payload
  );

  form = {
    city: updated.city,
    country: updated.country,
    un_2025_population: updated.un_2025_population
  };

  message = "Los cambios se han guardado correctamente.";
  return;
}

const created = await createCityStat(payload);
await deleteCityStat(originalKey.city, originalKey.country);
```

Frase:

Si no cambia la clave, hago **PUT**. Si cambia **city/country**, creo el nuevo recurso y borro el anterior.

### Analytics LCC: **loadHighcharts**, **renderChart**, **loadAnalytics**

Archivo:

```
frontend-group/src/routes/analytics/citys-stats/+page.svelte
```

Codigo real de carga:

```

async function loadHighcharts() {
  if (Highcharts) return Highcharts;

  const module = await import("highcharts");
  Highcharts = module.default;
  window._Highcharts = Highcharts;
  await import("highcharts/modules/accessibility.js");

  return Highcharts;
}

```

Fragmento clave de `renderChart`:

```

const data = citysStats
  .slice()
  .sort((a, b) => Number(b.un_2025_population) - Number(a.un_2025_population))
  .map((item) => ({
    name: labelFor(item),
    y: Number(item.un_2025_population)
  }));

chart?.destroy();

chart = Highcharts.chart(chartContainer, {
  chart: {
    type: "pie",
    backgroundColor: "transparent"
  },
  series: [
    {
      name: "Poblacion 2025",
      colorByPoint: true,
      data
    }
  ]
});

```

Codigo real de orden asincrono:

```

async function loadAnalytics() {
  loading = true;
  error = "";

  try {
    citysStats = await getAllCitysStats({ sort: "-un_2025_population" });
    loading = false;
    await tick();
    await loadHighcharts();
    renderChart();
  } catch (e) {
    error = e.message || "No se pudieron cargar los datos de citys-stats.";
    loading = false;
  }
}

```

Frase:

Primero carga datos, luego espera DOM con `tick`, despues carga Highcharts y finalmente pinta.

**Mapa LCC:** `keyFor`, `colorFor`, `radiusFor`

Archivo:

```
frontend-group/src/routes/analytics/citys-stats/map/+page.svelte
```

Codigo real:

```
function keyFor(item) {
  return `${String(item.city).toLowerCase()}|${String(item.country).toLowerCase()}`;
}

function colorFor(population) {
  if (population >= 35000000) return "#b91c1c";
  if (population >= 30000000) return "#ea580c";
  if (population >= 25000000) return "#f766e";
  return "#2563eb";
}

function radiusFor(population) {
  return 8 + Math.sqrt(population / maxPopulation) * 15;
}
```

Frase:

Estas funciones convierten datos de dominio en propiedades visuales del mapa.

**Mapa LCC: fragmento clave de** `renderMap`

Codigo real resumido:

```
function renderMap() {
  if (!mapContainer || !Highcharts || points.length === 0) return;

  mapChart?.destroy();

  mapChart = Highcharts.mapChart(mapContainer, {
    chart: {
      map: worldMap,
      backgroundColor: "transparent"
    },
    series: [
      {
        name: "Países",
        nullColor: "#dbe6dd"
      },
      {
        type: "mappoint",
        name: "Ciudades",

```

```

      data: points.map((point) => ({
        name: titleCase(point.city),
        lat: point.lat,
        lon: point.lon,
        marker: {
          radius: point.radius,
          fillColor: point.color
        },
        custom: point
      }))
    }
  ]
});
}

```

Frase:

`mapChart` necesita mapa base y una serie `mappoint` con latitud y longitud.

### Backend integraciones: `parseLimit`

Archivo:

```
src/back/v1/citys-stats.js
```

Codigo real:

```

function parseLimit(value, fallback, max) {
  if (value === undefined) return fallback;

  const limit = Number(value);

  if (!Number.isInteger(limit) || limit < 1 || limit > max) {
    return null;
  }

  return limit;
}

```

Frase:

Valida limites de integracion para no pedir o mostrar cantidades absurdas.

### Backend integraciones: `findAllCityStats`

Codigo real:

```

function findAllCityStats() {
  return new Promise((resolve, reject) => {
    db.find({}, (err, docs) => {
      if (err) return reject(err);
      resolve(docs.map(removeDatabaseId));
    });
  });
}

```

```
});
}
```

Frase:

Convierte el callback de NeDB en una Promise para poder usar `await`.

## Backend integraciones: `buildCityCountrySummaries`

Fragmento real:

```
function buildCityCountrySummaries(items) {
  const byCountry = new Map();

  items.forEach((item) => {
    const key = normalizeCountryKey(item.country);
    if (!key) return;

    const population = readFiniteNumber(item.un_2025_population, 0);
    const current = byCountry.get(key) || {
      country: item.country,
      countryKey: key,
      city: item.city,
      topCity: item.city,
      topCityPopulation: population,
      cityCount: 0,
      un_2025_population: 0,
      cities: []
    };

    current.cityCount += 1;
    current.un_2025_population += population;
    current.cities.push({ city: item.city, population });

    if (population > current.topCityPopulation) {
      current.city = item.city;
      current.topCity = item.city;
      current.topCityPopulation = population;
    }

    byCountry.set(key, current);
  });

  return [...byCountry.values()]
    .map((item) => ({
      ...item,
      cities: item.cities.sort((a, b) => b.population - a.population)
    }))
    .sort((a, b) => b.un_2025_population - a.un_2025_population);
}
```

Frase:

Agrupar ciudades por país, sumar población y guardar la ciudad principal para integraciones.

## Backend integraciones: **fetchJson**

Fragmento real:

```
async function fetchJson(url, sourceName, timeoutMs = 20000) {
  const controller = new AbortController();
  const timeout = setTimeout(() => controller.abort(), timeoutMs);

  try {
    const response = await fetch(url, {
      headers: {
        Accept: "application/json",
        "User-Agent": "SOS2526-29 citys-stats integration"
      },
      signal: controller.signal
    });

    const text = await response.text();
    let data = null;

    try {
      data = text ? JSON.parse(text) : null;
    } catch {
      throw new Error(`${sourceName} did not return JSON`);
    }

    if (!response.ok) {
      const reason = data?.message || data?.error || response.statusText;
      throw new Error(`${sourceName} returned ${response.status}: ${reason}`);
    }

    return data;
  } finally {
    clearTimeout(timeout);
  }
}
```

Frase:

Pide JSON externo, controla timeout, valida HTTP y evita aceptar respuestas que no sean JSON.

## Backend integraciones: **safeExternal**

Código real:

```
async function safeExternal(source, task) {
  try {
    return { source, data: await task(), error: null };
  } catch (err) {
    return { source, data: null, error: err.message };
  }
}
```

Frase:



Convierte una excepcion externa en un error parcial controlado.

## Backend integraciones: `buildIntegratedCityBase`

Codigo real:

```
async function buildIntegratedCityBase(item) {
  const [geocodingResult, countryResult] = await Promise.all([
    safeExternal("Open-Meteo Geocoding API", () => getGeocoding(item.city, item.country)),
    safeExternal("REST Countries API", () => getCountryInfo(item.country))
  ]);

  return {
    item,
    geocodingResult,
    countryResult
  };
}
```

Frase:

Lanza Open-Meteo y REST Countries en paralelo porque no dependen entre si.

## Integraciones UI: `safeLoad`

Archivo:

```
frontend-group/src/routes/integrations/citys-stats/+page.svelte
```

Codigo real:

```
async function safeLoad(task) {
  try {
    return { data: await task(), error: "" };
  } catch (e) {
    return { data: null, error: e.message || "No se pudo cargar la integracion." };
  }
}
```

Frase:

Es la version frontend de error parcial: si falla una integracion, no rompe toda la pantalla.

## Integraciones UI: `loadHighcharts`

Fragmento real:

```
async function loadHighcharts() {
  if (Highcharts) return Highcharts;

  const module = await import("highcharts");
```

```

Highcharts = module.default;
window._Highcharts = Highcharts;

const moreModule = await import("highcharts/highcharts-more.js");
const morePlugin = moreModule.default ?? moreModule;
if (typeof morePlugin === "function") morePlugin(Highcharts);

async function loadPlugin(pluginLoader) {
  const pluginModule = await pluginLoader();
  const plugin = pluginModule.default ?? pluginModule;
  if (typeof plugin === "function") plugin(Highcharts);
}

await loadPlugin(() => import("highcharts/modules/dumbbell.js"));
await loadPlugin(() => import("highcharts/modules/lollipop.js"));
await loadPlugin(() => import("highcharts/modules/variwide.js"));
await loadPlugin(() => import("highcharts/modules/bullet.js"));
await loadPlugin(() => import("highcharts/modules/sankey.js"));
await loadPlugin(() => import("highcharts/modules/treemap.js"));
await loadPlugin(() => import("highcharts/modules/sunburst.js"));
await loadPlugin(() => import("highcharts/modules/accessibility.js"));

return Highcharts;
}

```

Frase:

Los tipos avanzados de Highcharts necesitan modulos. Si cambio a otro tipo avanzado, reviso aqui.

## Integraciones UI: **createChart** y **destroyIntegrationCharts**

Codigo real:

```

function destroyIntegrationCharts() {
  integrationCharts.forEach((chart) => chart?.destroy());
  integrationCharts = [];
}

function createChart(container, config) {
  if (!Highcharts || !container) return;

  integrationCharts.push(Highcharts.chart(container, {
    ...config,
    chart: {
      backgroundColor: "transparent",
      ...(config.chart ?? {})
    },
    credits: {
      enabled: false,
      ...(config.credits ?? {})
    }
  }));
}

```

Frase:

Guardo las graficas para poder destruirlas y no duplicar instancias al recargar.

## Integraciones UI: fragmento clave de `loadIntegrations`

Codigo real resumido:

```
async function loadIntegrations() {
  loading = true;
  error = "";
  integrationErrors = [];
  destroyIntegrationCharts();

  try {
    countrySummaries = await getCountrySummaries(selectedLimit);

    if (countrySummaries.length === 0) {
      await loadInitialCityStats();
      countrySummaries = await getCountrySummaries(selectedLimit);
    }

    const geocodingResults = await Promise.all(
      countrySummaries.map((item) =>
        safeLoad(() => getGeocoding(item.topCity, item.country))
      )
    );

    const countryResults = await Promise.all(
      countrySummaries.map((item) => safeLoad(() => getCountryInfo(item.country)))
    );

    const worldBankResults = await Promise.all(
      countryCards.map((row) => {
        if (!row.countryData?.cca3) {
          return Promise.resolve({
            data: null,
            error: "Codigo ISO3 no disponible"
          });
        }

        return safeLoad(() => getWorldBankPopulation(row.countryData.cca3));
      })
    );

    const [tourismResult, earthquakeResult, fifaResult, esportsResult] = await Promise.all([
      safeLoad(getSosTouristArrivals),
      safeLoad(getSosEarthquakes),
      safeLoad(getSosFifaSquadValues),
      safeLoad(getSosEsportsEarnings)
    ]);

    loading = false;
    await tick();
    await loadHighcharts();
    renderIntegrationCharts();
  } catch (e) {
    error = e.message || "No se pudieron cargar las integraciones.";
  }
}
```

```
    loading = false;  
  }  
}
```

Frase:

Esta funcion coordina toda la pantalla: carga datos locales, lanza integraciones en paralelo por bloques, espera DOM, carga Highcharts y pinta widgets.

## Guia para leer funciones si no tienes mucha base

Cuando una persona principiante abre un archivo del proyecto, lo normal es perderse porque ve muchas funciones juntas. La forma segura de leerlas es no intentar entender todo a la vez.

Metodo recomendado:

1. Busca primero la ruta o evento que dispara la funcion.
2. Lee los parametros de entrada.
3. Marca las variables que se crean dentro.
4. Busca llamadas a otras funciones.
5. Mira que devuelve o que estado cambia.
6. Mira que pasa si hay error.
7. Conecta esa funcion con el flujo completo.

Preguntas simples para no perderte:

```
De donde vienen los datos?  
Que forma tienen al entrar?  
Que cambia la funcion?  
Que forma tienen al salir?  
Que pasa si algo viene mal?  
Quien usa el resultado?
```

Ejemplo con `normalizeCityStat`:

```
De donde vienen los datos?  
Del body de un POST o PUT.  
  
Que forma tienen al entrar?  
Pueden venir como texto, con espacios o con poblacion como string.  
  
Que cambia la funcion?  
Limpia ciudad y pais, convierte poblacion a numero.  
  
Que forma tienen al salir?  
Objeto limpio listo para guardar.  
  
Que pasa si algo viene mal?  
Devuelve null y el endpoint responde 400.
```

Quien usa el resultado?  
Los handlers POST y PUT.

Ejemplo con `refreshList`:

De donde vienen los datos?  
Del service `getAllCityStats`.

Que forma tienen al entrar?  
Una query opcional, por ejemplo `{ country: "china" }`.

Que cambia la funcion?  
Actualiza `loading`, `error`, `cityStats`, `activeQuery` y `message`.

Que forma tienen al salir?  
Devuelve array de resultados para que `handleSearch` pueda contar.

Que pasa si algo viene mal?  
Vacía la tabla y muestra error.

Quien usa el resultado?  
La pantalla CRUD.

## Diferencia entre tipos de funciones

No todas las funciones del proyecto tienen el mismo papel. Saber clasificarlas ayuda mucho en defensa.

| Tipo de funcion          | Que hace                           | Ejemplos                                                                                                   |
|--------------------------|------------------------------------|------------------------------------------------------------------------------------------------------------|
| Funcion de registro      | Enchufa rutas al servidor          | <code>module.exports = (app, db) =&gt; { ... }</code>                                                      |
| Funcion de validacion    | Decide si un dato es aceptable     | <code>hasExactCityFields</code> , <code>parseLimit</code>                                                  |
| Funcion de normalizacion | Limpia y transforma datos          | <code>normalizeCityStat</code> , <code>normalizeCountryKey</code>                                          |
| Handler de API           | Atiende una ruta HTTP              | <code>GET /api/v2/citys-stats</code> , <code>POST /api/v2/citys-stats</code>                               |
| Funcion de persistencia  | Lee o modifica NeDB                | llamadas a <code>db.find</code> , <code>db.insert</code> , <code>db.update</code> , <code>db.remove</code> |
| Service frontend         | Hace <code>fetch</code> al backend | <code>getAllCityStats</code> , <code>createCityStat</code>                                                 |
| Funcion de estado UI     | Cambia variables Svelte            | <code>refreshList</code> , <code>handleCreate</code> , <code>handleUpdate</code>                           |
| Funcion de visualizacion | Prepara o pinta graficas           | <code>renderChart</code> , <code>renderMap</code> , <code>createChart</code>                               |
| Funcion de integracion   | Pide o mezcla APIs externas        | <code>fetchJson</code> , <code>safeExternal</code> , <code>buildIntegratedCity</code>                      |

Regla sencilla:

Si valida datos, esta cerca del backend o del formulario.  
Si llama a `fetch`, esta en services.  
Si cambia `loading/message/error`, esta en una pantalla Svelte.  
Si llama a `Highcharts`, esta en `analytics`, mapa o integraciones.  
Si llama a una URL externa, esta en backend de integraciones.

## Como seguir una llamada completa de LCC

Flujo de crear ciudad, con funciones exactas:

```
Usuario pulsa Guardar registro
-> handleCreate
  -> validateCityStatForm
  -> createCityStat
    -> buildUrl
    -> fetch POST /api/v2/citys-stats
      -> normalizeCityStat
        -> hasExactCityFields
      -> db.findOne
      -> db.insert
      -> removeDatabaseId
    -> handleResponse
  -> refreshList
  -> getAllCityStats
```

Como defenderlo:

El frontend valida para dar respuesta rapida, pero el backend vuelve a validar porque no puede fiarse del navegador. Si el dato es valido, se comprueba duplicado y se inserta en NeDB.

Flujo de busqueda:

```
Usuario rellena filtros
-> handleSearch
  -> buildSearchQuery
    -> parseOptionalPositiveInteger
    -> parseOptionalNonNegativeInteger
  -> refreshList
  -> getAllCityStats
    -> buildUrl
    -> fetch GET /api/v2/citys-stats?...query
      -> handler GET /api/v2/citys-stats
        -> db.find
        -> removeDatabaseId
        -> filtros exactos
        -> q
        -> sort
        -> offset/limit
      -> handleResponse
```

Como defenderlo:

La busqueda se divide en dos partes. El frontend construye una query limpia y el backend aplica la logica real sobre los datos.

Flujo de integraciones:

```

Usuario abre /integrations/citys-stats
-> loadIntegrations
-> getCountrySummaries
  -> /api/v1/citys-stats/country-summaries
    -> findAllCityStats
    -> buildCityCountrySummaries
-> getGeocoding
  -> /integrations/geocoding/:city
    -> fetchJson Open-Meteo
-> getCountryInfo
  -> /integrations/country/:country
    -> fetchJson REST Countries
-> getWorldBankPopulation
  -> /integrations/world-bank/:countryCode
    -> fetchJson World Bank
-> getSosTouristArrivals / getSosEarthquakes / getSosFifaSquadValues / getSosEsportsEarnings
  -> fetchJson APIs SOS externas
-> safeLoad
-> renderIntegrationCharts

```

Como defenderlo:

La pantalla no llama directamente a APIs externas. Llama a endpoints propios. El backend hace de proxy, normaliza y controla errores.

## Receta completa: anadir campo **continent** a **citys-stats**

Este es el ejemplo mas probable para demostrar que entiendes funciones, backend, frontend y tests.

Objetivo:

```

{
  "city": "tokyo",
  "country": "japan",
  "continent": "asia",
  "un_2025_population": 33412512
}

```

### 1. Backend v2

Archivo:

```
src/back/v2/citys-stats.js
```

Funcion que tocar:

```
hasExactCityFields
```

Antes:

```
const expected = ["city", "country", "un_2025_population"].sort();
```

Despues:

```
const expected = ["city", "country", "continent", "un_2025_population"].sort();
```

Funcion que tocar:

```
normalizeCityStat
```

Anadir:

```
const continent = String(body.continent).trim().toLowerCase();
```

Validar:

```
if (!city || !country || !continent || ...) return null;
```

Devolver:

```
return { city, country, continent, un_2025_population };
```

Datos iniciales:

```
{ city: "tokyo", country: "japan", continent: "asia", un_2025_population: 33412512 }
```

Filtros:

Si quieres filtrar por continente, en el handler `GET /api/v2/citys-stats` anadir:

```
if (req.query.continent !== undefined) {
  result = result.filter(
    d => d.continent === String(req.query.continent).trim().toLowerCase()
  );
}
```

Ordenacion:

Si quieres ordenar por continente, anadirlo a:

```
const allowedFields = ["city", "country", "continent", "un_2025_population"];
```

## 2. Backend v1

Archivo:

```
src/back/v1/citys-stats.js
```

Hacer lo mismo si se quiere mantener consistencia en v1:

- `hasExactCityFields`



- `normalizeCityStat`
- `initialData`
- filtros si procede
- agregados si `continent` afecta a integraciones

### 3. Service frontend

Archivo:

```
frontend-group/src/services/citysStatsApi.js
```

Normalmente no hay que tocarlo porque envia el objeto completo que recibe. Solo se toca si quieres mensajes de error nuevos en `friendlyApiMessage`.

### 4. Pantalla CRUD

Archivo:

```
frontend-group/src/routes/citys-stats/+page.svelte
```

Funciones que tocar:

```
emptycreateForm  
emptysearchform  
validateCityStatForm  
buildSearchQuery
```

En `emptycreateForm`:

```
continent: ""
```

En `validateCityStatForm`:

```
const continent = String(form.continent ?? "").trim();  
if (!continent) throw new Error("Indique un continente.");  
return { city, country, continent, un_2025_population };
```

En HTML:

- Anadir input de crear.
- Anadir input de busqueda si se filtra.
- Anadir columna en tabla.

### 5. Pantalla de edicion

Archivo:

```
frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte
```

Funciones que tocar:

```
emptyForm
loadResource
validateForm
handleUpdate
```

Añadir `continent` al formulario, cargarlo desde `data.continent`, validarlo y enviarlo en payload.

## 6. Analytics, mapa e integraciones

Revisar:

```
frontend-group/src/routes/analytics/citys-stats/+page.svelte
frontend-group/src/routes/analytics/citys-stats/map/+page.svelte
frontend-group/src/routes/integrations/citys-stats/+page.svelte
```

Si solo es un dato informativo, no hace falta cambiar graficas. Si quieres usarlo visualmente:

- Analytics: agrupar por continente.
- Mapa: mostrar continente en tooltip.
- Integraciones: incluirlo en tarjetas o tablas.

## 7. Tests

Revisar:

```
tests/LCC/pruebas-lcc.json
tests/LCC/pruebas-lcc-v2.json
tests/LCC/e2e/citys-stats.spec.js
```

Actualizar todos los bodies de `POST` y `PUT`, porque la API exige estructura exacta.

Comprobacion:

```
npm.cmd run test-LCC-v2
npm.cmd run test-LCC-e2e
```

Frase de defensa:

Como el campo cambia el contrato del recurso, no es un cambio solo visual. Hay que tocar validacion, normalizacion, datos iniciales, formularios, tablas y tests.

## Receta completa: añadir filtro `min_population`

Objetivo:

Permitir:

```
GET /api/v2/citys-stats?min_population=25000000
```

## Backend

Archivo:

```
src/back/v2/citys-stats.js
```

Funcion:

```
handler GET /api/v2/citys-stats
```

Bloque recomendado despues de filtros exactos:

```
if (req.query.min_population !== undefined) {
  const value = Number(req.query.min_population);
  if (!Number.isFinite(value) || value < 0) {
    return res.status(400).json({ error: "Invalid query" });
  }
  result = result.filter(d => d.un_2025_population >= value);
}
```

Explicacion:

- Query params llegan como texto.
- Se convierten a numero.
- Se valida antes de filtrar.
- Se devuelve **400** si no es valido.

## Frontend

Archivo:

```
frontend-group/src/routes/citys-stats/+page.svelte
```

Funciones:

- **emptySearchForm**
- **buildSearchQuery**

Anadir:

```
min_population: ""
```

En **buildSearchQuery**:

```
min_population: parseOptionalNonNegativeInteger(
  searchForm.min_population,
  "Poblacion minima"
)
```

HTML:

Añadir input numerico en el formulario de busqueda.

## Tests

Añadir caso:

```
/api/v2/citys-stats?min_population=30000000
```

Y caso invalido:

```
/api/v2/citys-stats?min_population=abc -> 400
```

Frase de defensa:

Este filtro se implementa en backend porque el contrato de busqueda pertenece a la API. El frontend solo ofrece un input para construir la query.

## Receta completa: anadir endpoint `/api/v2/citys-stats/count`

Objetivo:

Devolver:

```
{
  "count": 12
}
```

Archivo:

```
src/back/v2/citys-stats.js
```

Donde ponerlo:

Antes de:

```
app.get(`${BASE_API_URL}/:city/:country`, ...)
```

Codigo:

```
app.get(`${BASE_API_URL}/count`, (req, res) => {
  db.count({}, (err, count) => {
    if (err) return res.sendStatus(500);
    return res.status(200).json({ count });
  });
});
```

Por que antes:

Express evalua rutas en orden. Si una ruta parametrizada estuviera antes, podria interpretar `count` como una ciudad.

Si se usa en frontend:

1. Crear funcion en `citysStatsApi.js`.
2. Llamarla desde pantalla.
3. Pintar resultado.

Service:

```
export async function getCitysStatsCount() {
  const response = await fetch(buildUrl("/count"));
  return handleResponse(response);
}
```

Frase de defensa:

Es una ruta de lectura, por eso es `GET`. Devuelve un agregado, no modifica datos.

## Receta completa: cambiar el grafico individual de `pie` a `bar`

Archivo:

```
frontend-group/src/routes/analytics/citys-stats/+page.svelte
```

Funcion:

```
renderChart
```

Cambiar:

```
chart: {
  type: "pie"
}
```

por:

```
chart: {
  type: "bar"
}
```

Pero no basta siempre con eso:

- En `pie`, cada punto es `{ name, y }`.
- En `bar`, normalmente usas `xAxis.categories` y `series`.

Ejemplo conceptual para `bar`:

```
xAxis: {
  categories: data.map((item) => item.name)
},
series: [
```

```
{
  name: "Poblacion 2025",
  data: data.map((item) => item.y)
}
```

Que revisar:

- Tooltip.
- Data labels.
- Accesibilidad.
- Altura del contenedor.

Frase de defensa:

Highcharts no cambia los datos, solo la representacion. La fuente sigue siendo `getAllCityStats`.

## Receta completa: anadir una API externa nueva a integraciones

Supongamos que quieres anadir una API nueva por pais.

### Backend

Archivo:

```
src/back/v1/citys-stats.js
```

Pasos:

1. Crear constante con URL externa.
2. Crear funcion `getNuevaApi`.
3. Crear normalizador `normalizeNuevaApiRow` si devuelve muchas filas.
4. Crear agregador `buildNuevaApiByCountry` si hay que cruzar por pais.
5. Crear endpoint proxy:

```
app.get(`${BASE_API_URL}/integrations/nueva-api`, async (req, res) => {
  try {
    const rows = await getNuevaApi();
    return res.status(200).json({ source: "Nueva API", count: rows.length, rows });
  } catch (err) {
    return res.status(502).json({ error: err.message });
  }
});
```

1. Si debe aparecer en `summary`, anadirla al `Promise.all` del endpoint `/integrations/summary`.
2. Anadir su dato a `studentApis` o al objeto integrado.
3. Anadir error parcial a `integrationErrors`.

Funciones que probablemente tocas:

- `fetchJson`
- `safeExternal`
- `normalizeCountryKey`
- `buildIntegratedCity`

## Service

Archivo:

```
frontend-group/src/services/citysStatsIntegrations.js
```

Anadir:

```
export async function getNuevaApi() {
  const response = await fetch(`${CITYS_STATS_INTEGRATIONS_API_BASE}/integrations/nueva-api`);
  return handleResponse(response);
}
```

## Pantalla

Archivo:

```
frontend-group/src/routes/integrations/citys-stats/+page.svelte
```

Pasos:

1. Importar `getNuevaApi`.
2. Anadir tarjeta en `sourceCards`.
3. Crear estado, por ejemplo `nuevaApiCountries`.
4. Llamarla en `loadIntegrations`.
5. Crear `renderNuevaApiChart`.
6. Anadirla en `renderIntegrationCharts`.
7. Anadir HTML para el nuevo panel.
8. Destruir grafica con las demas.

Frase de defensa:

Para integrar una API no se toca solo la pantalla. Primero se crea proxy en backend, luego service frontend, y por ultimo visualizacion.

## Receta completa: cambiar la clave de integracion

Ahora LCC integra por pais. Si pidieran integrar por ciudad, esto cambiaria muchas cosas.

Funciones afectadas:

```
buildCityCountrySummaries
normalizeCountryKey
buildIntegratedCityBase
```

```
buildIntegratedCity  
combinedCountryRows  
findLocalCountry  
localCountryIndex
```

Riesgo:

- REST Countries y World Bank dejarían de encajar directamente.
- Habría más datos sin coincidencia.
- Muchas APIs SOS trabajan por país.

Respuesta defensiva:

Se puede hacer, pero sería peor para esta entrega porque las fuentes externas comparten mejor el campo `country`. Cambiar a ciudad reduciría coincidencias y haría los widgets menos informativos.

## Receta completa: entender y modificar errores de usuario

Si quieres cambiar un mensaje de error visible en CRUD LCC:

Archivo:

```
frontend-group/src/services/citysStatsApi.js
```

Función:

```
friendlyApiMessage
```

Ejemplo:

Backend devuelve:

```
{ "error": "Resource already exists" }
```

Frontend muestra:

```
Ya existe un registro con esa ciudad y ese país.
```

Si cambias texto visible:

1. Cambiar `friendlyApiMessage`.
2. Revisar tests Playwright si esperan texto.
3. Provocar el error en pantalla.

Frase de defensa:

El backend devuelve errores técnicos estables y el frontend los traduce a mensajes de usuario.



## Receta completa: diagnosticar un test fallido

Si falla Newman:

1. Mirar que endpoint falla.
2. Abrir archivo de API del recurso.
3. Comprobar status esperado.
4. Comprobar body esperado.
5. Ver si cambiaste contrato.
6. Si el contrato nuevo es correcto, actualizar test.
7. Si el contrato viejo era correcto, arreglar código.

Si falla Playwright:

1. Mirar acción que falla.
2. Abrir pantalla Svelte.
3. Comprobar `data-testid`.
4. Comprobar service llamado.
5. Comprobar API con `Invoke-RestMethod`.
6. Ver screenshot o trace si existe.

Frase de defensa:

Primero distingo si falla el contrato de API o el flujo visible del navegador. Newman y Playwright prueban capas distintas.

## Tabla de "si tocas esto, revisa esto"

| Cambio                                 | Revisar tambien                                                              |
|----------------------------------------|------------------------------------------------------------------------------|
| <code>hasExactCityFields</code>        | <code>normalizeCityStat</code> , tests POST/PUT, formularios                 |
| <code>normalizeCityStat</code>         | mensajes de error, datos iniciales, edicion                                  |
| Handler <code>GET</code>               | <code>buildSearchQuery</code> , tests de filtros, analytics si usa query     |
| Handler <code>POST</code>              | <code>createCityStat</code> , <code>handleCreate</code> , tests de duplicado |
| Handler <code>PUT</code>               | <code>updateCityStat</code> , <code>handleUpdate</code> , tests de edicion   |
| <code>buildCityCountrySummaries</code> | integraciones, summary, widgets por pais                                     |
| <code>fetchJson</code>                 | todos los endpoints externos                                                 |
| <code>safeExternal</code>              | errores parciales de integracion                                             |
| <code>buildIntegratedCity</code>       | shape de <code>/integrations/summary</code> , pantalla de integraciones      |
| <code>buildUrl</code>                  | todos los fetch CRUD                                                         |
| <code>handleResponse</code>            | todos los mensajes y errores frontend                                        |
| <code>refreshList</code>               | listado, busqueda, crear, borrar, cargar iniciales                           |
| <code>renderChart</code>               | grafica concreta y limpieza con <code>onDestroy</code>                       |
| <code>renderMap</code>                 | mapa, coordenadas, tooltip, accesibilidad                                    |
| <code>loadIntegrations</code>          | todas las fuentes externas y los siete widgets                               |

## Explicacion ultra sencilla de los conceptos usados en funciones

`req`:

Es la peticion que llega al backend. Dentro tiene parametros de URL, query params y body.

`res`:

Es la respuesta que el backend envia al cliente.

`req.query`:

Es lo que viene despues de `?` en la URL. Por ejemplo, en `?country=china`, `req.query.country` vale `"china"`.

`req.params`:

Son partes variables de la URL. En `/api/v2/citys-stats/tokyo/japan`, `city` es `tokyo` y `country` es `japan`.

`req.body`:

Es el JSON que llega en `POST` o `PUT`.

`db.find`:

Busca documentos en NeDB.

`db.findOne`:

Busca un unico documento.

`db.insert`:

Inserta un documento.

`db.update`:

Actualiza documentos.

`db.remove`:

Borra documentos.

`fetch`:

Hace una peticion HTTP desde frontend o backend.

`async/await`:

Permite escribir codigo asincrono de forma mas legible. Se usa cuando hay que esperar respuestas de red o acciones que tardan.

`Promise.all`:

Lanza varias promesas en paralelo y espera a que todas terminen. Se usa para acelerar integraciones.

Idea clave:

```
await unaFuncion() = espera a que esa funcion termine antes de seguir.  
Promise.all([a(), b(), c()]) = lanza varias a la vez y espera a que terminen todas.
```

`try/catch`:

Permite capturar errores sin romper la aplicacion.

`Map`:

Estructura clave-valor. Se usa para agrupar paises y buscar coincidencias rapido.

`filter`:

Deja en un array solo los elementos que cumplen una condicion.

`map`:

Transforma cada elemento de un array en otro.

`reduce`:

Acumula valores, por ejemplo sumar poblaciones.

`sort`:

Ordena arrays.

`slice`:

Recorta arrays; se usa para paginacion.

## Orden real de ejecucion de funciones asincronas

Esta parte es muy importante para defensa. Si preguntan por funciones asincronas, no basta con decir "usa `await`". Hay que explicar que se espera, que va en paralelo y que pasa si algo falla.

Regla basica:

Una funcion marcada como `async` devuelve una `Promise`.  
Dentro de esa funcion, cada `await` pausa esa funcion hasta que termina la operacion esperada.  
El resto de la aplicacion no se congela; solo se espera dentro de esa cadena.

Frase perfecta:

`async/await` se usa porque las peticiones HTTP y la carga de librerias no responden instantaneamente. Con `await` hago que la siguiente linea no se ejecute hasta tener datos reales o un error capturado.

## Orden asincrono al abrir el CRUD

Archivo:

```
frontend-group/src/routes/citys-stats/+page.svelte
```

Orden real:

1. El navegador abre `/citys-stats`.
2. Svelte monta el componente.
3. `onMount` ejecuta `refreshList({}, "")`.
4. `refreshList` pone `loading = true` y limpia error.
5. `refreshList` hace `await getAllCitysStats(query)`.
6. `getAllCitysStats` construye la URL.
7. `getAllCitysStats` hace `await fetch(...)`.
8. Cuando el backend responde, `handleResponse` lee el body.
9. Si HTTP es correcto, devuelve JSON.
10. `refreshList` guarda `citysStats` y `activeQuery`.
11. `finally` pone `loading = false`.
12. Svelte redibuja la tabla con los datos.

Como decirlo:

La tabla no se pinta antes de tener respuesta. Primero Svelte monta, luego `refreshList` espera a `getAllCitysStats`, el service espera a `fetch`, y cuando vuelve el JSON se actualiza el estado.

## Orden asincrono al crear ciudad

Archivo:

```
frontend-group/src/routes/citys-stats/+page.svelte
```

Orden real:

1. El usuario pulsa crear.
2. `handleCreate` limpia mensajes.
3. `validateCityStatForm` valida el formulario en frontend.
4. `handleCreate` hace `await createCityStat(payload)`.
5. `createCityStat` hace `fetch POST /api/v2/citys-stats`.
6. El backend valida body, busca duplicado e inserta en NeDB.
7. `handleResponse` convierte la respuesta en JSON o error.
8. Si se crea bien, `handleCreate` limpia el formulario.
9. `handleCreate` hace `await refreshList(activeQuery, mensaje)`.
10. La lista se recarga desde backend y se muestra el registro nuevo.

Punto que hay que defender:

Después de crear no meto el registro manualmente en el array. Vuelvo a pedir la lista al backend para que la interfaz refleje el estado real de la base de datos.

Si falla:

```
400 -> formulario/body incorrecto
409 -> ya existe city + country
500 -> error interno
```

## Orden asincrono al editar ciudad sin cambiar clave

Archivo:

```
frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte
```

Orden real:

1. El usuario abre `/citys-stats/editar/:city/:country`.
2. `onMount` ejecuta `loadResource`.
3. `loadResource` hace `await getOneCityStat(params.city, params.country)`.
4. Se rellena form y se guarda `originalKey`.
5. El usuario modifica población u otro dato no clave.
6. `handleUpdate` valida el formulario.
7. `isSameResource` comprueba que `city + country` no han cambiado.
8. `handleUpdate` hace `await updateCityStat(originalKey.city, originalKey.country, payload)`.
9. `updateCityStat` hace `PUT /api/v2/citys-stats/:city/:country`.
10. El backend valida que URL y body coincidan.
11. NeDB actualiza el documento.
12. La pantalla muestra mensaje de éxito.

Frase clave:

El **PUT** se usa cuando estoy actualizando el mismo recurso identificado por la URL. Por eso el backend exige que **city** y **country** del body coincidan con los parámetros de la URL.

## Orden asincrono al editar cambiando ciudad o país

Este caso es especial y conviene saberlo muy bien.

Orden real:

1. handleUpdate valida el formulario.
2. isSameResource detecta que city o country han cambiado.
3. No se puede hacer PUT normal porque URL y body no coincidirían.
4. handleUpdate hace await createCityStat(payload).
5. Si crear funciona, hace await deleteCityStat(originalKey.city, originalKey.country).
6. Actualiza form y originalKey.
7. updateRoute cambia la URL a la nueva clave.
8. Muestra mensaje de éxito.

Por que se hace así:

Como **city + country** es la clave del recurso, cambiar la ciudad o el país equivale a cambiar la identidad del recurso. Para no violar la regla del **PUT**, se crea el recurso nuevo y después se borra el anterior.

Riesgo y defensa:

Si la creación falla por **409**, no se borra el original. Esto evita perder el registro antiguo cuando la nueva clave ya existe.

## Orden asincrónico en integraciones

Archivo:

```
frontend-group/src/routes/integrations/citys-stats/+page.svelte
```

Función principal:

```
loadIntegrations
```

Orden real:

1. onMount llama a loadIntegrations.
2. loading = true, se limpian errores y se destruyen gráficas antiguas.
3. await getCountrySummaries(selectedLimit).
4. Si no hay datos, await loadInitialCityStats() y se vuelve a pedir countrySummaries.
5. Promise.all pide geocoding de varias ciudades en paralelo.
6. Se cruzan resultados de geocoding con los países locales.
7. Promise.all pide REST Countries en paralelo.
8. Se obtienen códigos ISO3 y datos de país.
9. Promise.all pide World Bank en paralelo solo si hay ISO3.
10. Promise.all pide las cuatro APIs SOS externas en paralelo.
11. Se calculan topCountries y se guardan errores parciales.
12. loading = false.
13. await tick() espera a que Svelte pinte los contenedores HTML.
14. await loadHighcharts() carga Highcharts y sus módulos.
15. renderIntegrationCharts pinta los siete widgets.

Por que hay varios **Promise.all**:

Dentro de cada bloque, las llamadas son independientes y se pueden lanzar en paralelo. Pero unos bloques dependen de otros: World Bank necesita el código ISO3 que antes devuelve REST Countries. Por eso no todo se lanza a la vez.

Por que se usa `tick`:

`tick` espera a que Svelte actualice el DOM. Si Highcharts intenta pintar antes de que exista el `div`, la grafica puede fallar o quedarse vacia.

Por que se usa `safeLoad`:

`safeLoad` convierte errores de una API externa en un objeto con `error`. Asi una API caída no rompe toda la pantalla.

## Orden asincrono en el backend de integraciones

Archivo:

```
src/back/v1/citys-stats.js
```

Funciones clave:

```
fetchJson  
safeExternal  
buildIntegratedCityBase  
buildIntegratedCity
```

Orden conceptual:

1. Express recibe una petición de integración.
2. El handler `async` entra en `try/catch`.
3. `fetchJson` hace `fetch` a la API externa.
4. `fetchJson` espera respuesta HTTP.
5. Comprueba `status` y formato JSON.
6. Devuelve datos normalizados o lanza error.
7. `safeExternal` captura errores para que sean parciales.
8. El handler responde 200, 404 o 502 según el caso.

Frase de defensa:

En backend uso `async/await` para llamadas externas porque dependen de red. Uso `try/catch` y `safeExternal` para que los errores no aparezcan como pantallazo roto, sino como respuestas controladas.

## Diferencia entre callbacks de NeDB y `async/await`

En `src/back/v2/citys-stats.js` muchas operaciones de NeDB usan callbacks:

```
db.find({}, (err, docs) => { ... })  
db.findOne(query, (err, doc) => { ... })
```

```
db.insert(item, (err, newDoc) => { ... })
db.update(query, item, {}, (err2) => { ... })
db.remove(query, {}, (err, numRemoved) => { ... })
```

Como explicarlo:

NeDB no devuelve una Promise en estas llamadas; ejecuta una funcion callback cuando termina. El orden sigue siendo asincrono: primero llamo a `db.find`, y cuando NeDB termina entra en `(err, docs) => { ... }`.

Ejemplo `POST`:

```
1. Llega POST.
2. normalizeCityStat valida body.
3. db.findOne busca si existe.
4. Cuando findOne termina, entra en su callback.
5. Si existe, responde 409.
6. Si no existe, db.insert inserta.
7. Cuando insert termina, entra en su callback.
8. Responde 201 con el documento creado.
```

Frase clave:

Aunque no aparezca `await`, sigue siendo asincrono porque la respuesta se envia dentro del callback, cuando la base ya ha terminado.

## Manual universal para modificar el proyecto

Esta parte esta pensada para alguien que sabe poco y necesita orientarse rapido. La pregunta no es "que archivo existe", sino "que tengo que tocar para conseguir X".

Regla de oro:

```
Primero localiza si el cambio afecta a:
1. Datos
2. API
3. Service frontend
4. Pantalla
5. Grafica/mapa
6. Integracion externa
7. Tests
```

Si cambia lo que guarda o devuelve la API, casi seguro toca varias capas. Si solo cambia un texto visible, normalmente toca una pantalla Svelte.



## Como saber donde tocar segun lo que quieres cambiar

| Quiero cambiar...       | Empieza por...                                                                  | Luego revisa...                                               |
|-------------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------|
| Puerto o arranque       | <code>index.js</code>                                                           | <code>package.json</code>                                     |
| Donde se guarda la base | <code>index.js</code>                                                           | archivos <code>.db</code>                                     |
| Campos de un recurso    | backend del recurso                                                             | frontend, analytics, tests                                    |
| Validacion de datos     | <code>normalize...</code> , <code>hasExact...</code> , <code>hasValid...</code> | mensajes y tests                                              |
| Filtros de busqueda     | handler <code>GET</code> del backend                                            | formulario, <code>buildSearchQuery</code> , tests             |
| Ordenacion              | bloque <code>sort</code>                                                        | select del frontend                                           |
| Paginacion              | <code>offset / limit</code>                                                     | inputs de busqueda                                            |
| Crear un registro       | handler <code>POST</code>                                                       | service <code>create...</code> , formulario                   |
| Editar un registro      | handler <code>PUT</code>                                                        | service <code>update...</code> , pantalla <code>editar</code> |
| Borrar registros        | handler <code>DELETE</code>                                                     | botones, mensajes, tests                                      |
| Nueva pantalla          | <code>frontend-group/src/routes/.../+page.svelte</code>                         | <code>Navbar.svelte</code>                                    |
| Menu                    | <code>Navbar.svelte</code>                                                      | rutas existentes                                              |
| Portada                 | <code>routes/+page.svelte</code>                                                | tests e2e LCC                                                 |
| Grafica                 | <code>renderChart</code>                                                        | carga de datos y series                                       |
| Mapa                    | <code>renderMap</code> , <code>coordinates</code>                               | <code>keyFor</code> , tooltip                                 |
| Integracion externa     | <code>src/back/v1/citys-stats.js</code>                                         | service y pantalla integraciones                              |
| Error visible           | <code>friendlyApiMessage</code> o pantalla                                      | tests Playwright                                              |
| Tests API               | <code>tests/**/*.json</code>                                                    | contrato backend                                              |
| Tests navegador         | <code>tests/*/e2e/*.spec.js</code>                                              | <code>data-testid</code> y textos                             |

## Como buscar en el proyecto

Comando principal:

```
rg "texto_a_buscar"
```

Ejemplos utiles:

```
rg "un_2025_population"
rg "citys-stats"
rg "normalizeCityStat"
rg "handleCreate"
rg "renderChart"
rg "integrationErrors"
rg "data-testid"
```

Interpretacion:

- Si aparece en backend, afecta a API o datos.
- Si aparece en `services`, afecta a llamadas `fetch`.
- Si aparece en `routes`, afecta a pantalla.
- Si aparece en `tests`, hay una prueba que puede romperse si cambias ese texto o contrato.
- Si aparece en `public`, normalmente es build generado; no se edita a mano.

Regla:

Edita fuente real (`index.js`, `src/back`, `frontend-group/src`, `tests`). No edites `public` manualmente porque se regenera con `npm run build`.

## Anatomia de un archivo backend

Ejemplo:

```
src/back/v2/citys-stats.js
```

Partes habituales:

1. `module.exports = (app, db) => {`
2. Constantes: `BASE_API_URL`, `DOCS_URL`, URLs externas si las hay.
3. `initialData`.
4. Helpers: validacion, normalizacion, limpieza de `_id`.
5. Rutas especiales: `/docs`, `/loadInitialData`.
6. Ruta `GET` de coleccion.
7. Ruta `GET` de recurso concreto.
8. Ruta `POST`.
9. Rutas no permitidas `405`.
10. Ruta `PUT`.
11. Rutas `DELETE`.

Como leerlo:

```
Arriba se define el contrato.
En medio se preparan funciones auxiliares.
Abajo se registran rutas.
Cada ruta responde a una URL real.
```

Si modificas:

- `initialData`: cambia carga inicial.
- `hasExact...`: cambia campos aceptados.
- `normalize...`: cambia limpieza y validacion.
- `GET BASE_API_URL`: cambia busquedas/listado.
- `POST BASE_API_URL`: cambia creacion.

- `PUT .../:id`: cambia edicion.
- `DELETE`: cambia borrado.

## Anatomia de un service frontend

Ejemplo:

```
frontend-group/src/services/citysStatsApi.js
```

Partes habituales:

1. Define la URL base.
2. Construye URLs con `buildUrl`.
3. Codifica parametros de ruta con `encodePathValue`.
4. Traduce errores con `friendlyApiMessage`.
5. Procesa respuestas con `handleResponse`.
6. Exporta funciones concretas:

- `getAllCitysStats`
- `createCityStat`
- `deleteAllCitysStats`
- `loadInitialCitysStats`
- `deleteCityStat`
- `getOneCityStat`
- `updateCityStat`

Como leerlo:

```
El service no pinta nada.  
El service no valida toda la logica de negocio.  
El service solo sabe llamar a la API y devolver datos o errores.
```

Si modificas:

- Nueva ruta API: crea nueva funcion exportada.
- Nuevo mensaje de error: toca `friendlyApiMessage`.
- Cambio de version API: cambia la constante base.
- Cambio de identificador: revisa `encodePathValue` y rutas que construye.

## Anatomia de una pantalla Svelte

Ejemplo:

```
frontend-group/src/routes/citys-stats/+page.svelte
```

Partes habituales:

1. `<script>` con imports.

2. Constantes de opciones.
3. Estados con `let`.
4. Funciones de validacion.
5. Funciones de carga y eventos.
6. `onMount`.
7. HTML de la pantalla.
8. CSS dentro de `<style>`.

Como leerlo:

```
El <script> contiene la logica.  
El HTML muestra el estado.  
El CSS solo cambia apariencia.
```

Estados importantes:

- `loading`: muestra que se esta cargando.
- `error`: mensaje de error.
- `message`: mensaje de exito.
- `citysStats`: datos que se pintan.
- `createForm`: formulario de creacion.
- `searchForm`: formulario de busqueda.
- `activeQuery`: ultima busqueda aplicada.

Si modificas:

- Nuevo input: toca objeto de formulario, validacion y HTML.
- Nueva columna: toca tabla.
- Nuevo boton: crea funcion y boton.
- Nuevo mensaje: toca funciones `handle...` o service.
- Cambio visual: toca CSS.

## Manual para cambiar cualquier campo

Este es el cambio mas peligroso porque afecta al contrato.

Pasos universales:

1. Busca el campo actual:

```
rg "nombre_del_campo"
```

1. Cambia backend:

- Campos esperados.
- Normalizacion.
- Datos iniciales.
- Filtros.

- Ordenacion si procede.

#### 1. Cambia frontend:

- Formularios.
- Tablas.
- Pantallas de edicion.
- Services si construyen payloads.

#### 1. Cambia visualizaciones:

- Analytics si suma o muestra el campo.
- Mapa si lo usa en tooltip, color o radio.
- Integraciones si cruza por ese campo.

#### 1. Cambia tests:

- Newman si el body cambia.
- Playwright si la pantalla cambia.

#### 1. Ejecuta:

```
npm.cmd run build
npm.cmd run test-LCC-v2
npm.cmd run test-LCC-e2e
```

Frase de defensa:

Cambiar un campo no es cambiar una etiqueta; cambia el contrato de la API y por eso se revisa de punta a punta.

## Manual para cambiar cualquier validacion

Validar significa decidir que datos entran y que datos se rechazan.

Funciones tipicas:

```
hasExactCityFields
normalizeCityStat
hasValidDisasterBody
normalizeWineStat
parsePositiveInteger
parseOptionalNonNegativeInteger
```

Preguntas antes de cambiar:

```
El campo es obligatorio u opcional?
Puede estar vacio?
Debe ser numero?
Debe ser entero?
Puede ser negativo?
```

Debe normalizarse a minúsculas?  
Debe coincidir con la URL?

Ejemplo de campo opcional en `citys-stats`:

```
const required = ["city", "country", "un_2025_population"];  
const optional = ["continent"];  
const allowed = [...required, ...optional];
```

La idea:

- Exigir obligatorios.
- Permitir opcionales.
- Rechazar desconocidos.

Frase de defensa:

La validacion protege el contrato. Si acepto campos opcionales, separo campos obligatorios de campos permitidos.

## Manual para cambiar cualquier endpoint

Un endpoint es una ruta de API.

Antes de escribir código decide:

Que recurso representa?  
Es lectura o modifica datos?  
Va en v1, v2 o nueva version?  
Devuelve coleccion, elemento o agregado?  
Necesita parametros en URL?  
Necesita query params?  
Que codigos HTTP devuelve?

Eleccion de metodo:

| Necesidad                   | Metodo |
|-----------------------------|--------|
| Consultar datos             | GET    |
| Crear recurso               | POST   |
| Actualizar recurso completo | PUT    |
| Borrar recurso              | DELETE |

Orden de rutas en Express:

```
1. /docs  
2. /loadInitialData  
3. /count u otras rutas concretas
```

```
4. /
5. /:city/:country o /:id
```

Por que:

Express revisa en orden. Las rutas concretas deben ir antes que las rutas con parametros.

Checklist de endpoint nuevo:

- Ruta creada.
- Codigo HTTP correcto.
- `_id` eliminado si devuelve documentos.
- Errores controlados.
- Service frontend si se usa en pantalla.
- Test Newman.
- Documentacion actualizada.

## Manual para cambiar cualquier pantalla

Preguntas:

```
La pantalla ya existe o hay que crear una nueva?
Necesita datos de API?
Necesita formulario?
Necesita tabla?
Necesita grafica?
Necesita ruta dinamica?
Debe aparecer en el menu?
```

Pantalla nueva:

```
frontend-group/src/routes/nombre-ruta/+page.svelte
```

Pantalla con parametro:

```
frontend-group/src/routes/recurso/[id]/+page.svelte
```

No hay que registrar manualmente en `App.svelte`.

Estructura minima:

```
<script>
  import { onMount } from "svelte";

  let loading = true;
  let error = "";

  async function loadData() {
    try {
      loading = true;
      error = "";
```

```

    // pedir datos
  } catch (e) {
    error = e.message;
  } finally {
    loading = false;
  }
}

onMount(loadData);
</script>

{#if loading}
  <p>Cargando...</p>
{:else if error}
  <p>{error}</p>
{:else}
  <main>Contenido</main>
{/if}

```

Si debe aparecer en menu:

```
frontend-group/src/components/Navbar.svelte
```

## Manual para cambiar cualquier grafica

Archivos de graficas:

```

frontend-group/src/routes/analytics/+page.svelte
frontend-group/src/routes/analytics/citys-stats/+page.svelte
frontend-group/src/routes/analytics/citys-stats/map/+page.svelte
frontend-group/src/routes/integrations/citys-stats/+page.svelte

```

Partes que mirar:

```

loadAnalytics / loadMapData / loadIntegrations
buildMetrics o transformaciones
loadHighcharts
renderChart / renderMap
onDestroy

```

Para cambiar datos:

- Toca la funcion que transforma datos.

Para cambiar tipo visual:

- Toca `chart.type` y estructura de `series`.

Para cambiar tooltip:

- Toca `tooltip.formatter` o `pointFormatter`.

Para cambiar etiquetas:



- Toca `xAxis.categories`, `dataLabels` o `title`.

Frase de defensa:

La grafica no es fuente de verdad. La fuente es la API; Highcharts solo representa datos ya transformados.

### Tipos de graficas actuales y como cambiarlas

Esta tabla resume las graficas reales que usa el proyecto. Sirve para defenderlas y para saber que tocar si piden cambiar una.

| Pantalla                   | Archivo                                                                       | Tipo actual                                           | Donde se cambia                                      | Datos que necesita                                                                             |
|----------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------|------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Analytics grupal           | <code>frontend-group/src/routes/analytics/+page.svelte</code>                 | <code>column</code>                                   | <code>chart.type</code> en <code>renderChart</code>  | categorias + series numericas                                                                  |
| Analytics LCC              | <code>frontend-group/src/routes/analytics/citys-stats/+page.svelte</code>     | <code>pie</code>                                      | <code>chart.type</code> en <code>renderChart</code>  | puntos <code>{ name, y }</code>                                                                |
| Mapa LCC                   | <code>frontend-group/src/routes/analytics/citys-stats/map/+page.svelte</code> | <code>mappoint</code> dentro de <code>mapChart</code> | <code>series[].type</code> en <code>renderMap</code> | puntos con <code>lat</code> , <code>lon</code> , <code>name</code> , <code>z</code>            |
| Integracion Open-Meteo     | <code>frontend-group/src/routes/integrations/citys-stats/+page.svelte</code>  | <code>treemap</code>                                  | <code>renderGeocodingChart</code>                    | nodos con <code>id</code> , <code>parent</code> , <code>value</code> , <code>colorValue</code> |
| Integracion REST Countries | mismo archivo                                                                 | <code>sankey</code>                                   | <code>renderCountryChart</code>                      | enlaces <code>[origen, destino, peso]</code>                                                   |
| Integracion World Bank     | mismo archivo                                                                 | <code>lollipop</code>                                 | <code>renderWorldBankChart</code>                    | categorias + puntos numericos                                                                  |
| Integracion turismo SOS    | mismo archivo                                                                 | <code>variwide</code>                                 | <code>renderTourismChart</code>                      | puntos <code>[categoria, y, z]</code>                                                          |
| Integracion terremotos SOS | mismo archivo                                                                 | <code>bullet</code>                                   | <code>renderEarthquakeChart</code>                   | valor + objetivo/target                                                                        |
| Integracion FIFA SOS       | mismo archivo                                                                 | <code>dumbbell</code>                                 | <code>renderFifaChart</code>                         | puntos con <code>low</code> y <code>high</code>                                                |
| Integracion eSports SOS    | mismo archivo                                                                 | <code>sunburst</code>                                 | <code>renderEsportsChart</code>                      | arbol con <code>id</code> , <code>parent</code> , <code>value</code>                           |

Frase perfecta:

Cambiar el tipo de grafica no es solo cambiar una palabra. Cada tipo espera una forma de datos distinta. Primero miro que datos tengo, despues elijo el tipo visual y por ultimo adapto `series`.

## Que significa cambiar `type`

En Highcharts suele haber dos sitios:

```
chart: {
  type: "pie"
}
```

O:

```
series: [
  {
    type: "mappoint",
    data: points
  }
]
```

Como explicarlo:

`chart.type` define el tipo por defecto de la grafica. `series.type` define el tipo de una serie concreta. Si hay varias series o un mapa, normalmente se usa `series.type`.

## Cambiar `pie` a `bar` o `column`

Archivo:

```
frontend-group/src/routes/analytics/citys-stats/+page.svelte
```

Antes:

```
chart: {
  type: "pie"
}
```

Datos tipicos de `pie`:

```
[
  { name: "tokyo", y: 33412512 },
  { name: "delhi", y: 32226000 }
]
```

Para `bar` o `column`, conviene separar categorias y valores:

```
chart: {
  type: "bar"
},
xAxis: {
  categories: citysStats.map((item) => item.city)
},
series: [
  {
```

```
name: "Poblacion 2025",
data: citysStats.map((item) => item.un_2025_population)
}
]
```

Defensa:

En **pie** cada punto lleva **name** e **y**. En **bar** o **column**, Highcharts suele trabajar mejor con  **xAxis.categories** y  **series.data**.

## Cambiar el mapa

Archivo:

```
frontend-group/src/routes/analytics/citys-stats/map/+page.svelte
```

El mapa no es una grafica normal:

```
Highcharts.mapChart(mapContainer, {
  series: [
    {
      type: "mappoint",
      data: points
    }
  ]
})
```

Cada punto necesita coordenadas:

```
{
  name: "Tokyo",
  lat: 35.6762,
  lon: 139.6503,
  z: 33412512
}
```

Defensa:

En un mapa, no basta con poblacion. Highcharts necesita latitud y longitud para colocar cada punto.

## Cambiar una grafica avanzada de integraciones

Archivo:

```
frontend-group/src/routes/integrations/citys-stats/+page.svelte
```

Algunos tipos necesitan modulos:

```
await loadPlugin(() => import("highcharts/modules/dumbbell.js"));
await loadPlugin(() => import("highcharts/modules/lollipop.js"));
await loadPlugin(() => import("highcharts/modules/variwide.js"));
```

```
await loadPlugin(() => import("highcharts/modules/bullet.js"));
await loadPlugin(() => import("highcharts/modules/sankey.js"));
await loadPlugin(() => import("highcharts/modules/treemap.js"));
await loadPlugin(() => import("highcharts/modules/sunburst.js"));
```

Si cambias a un tipo avanzado:

1. Comprobar si necesita modulo.
2. Importar el modulo en `loadHighcharts`.
3. Adaptar `series`.
4. Adaptar tooltip.
5. Adaptar leyenda/ejes.
6. Probar que no se repite tipo si la entrega pide variedad.

Ejemplos de estructuras:

| Tipo     | Forma de datos                                                               |
|----------|------------------------------------------------------------------------------|
| sankey   | <code>[["citys-stats", "spain", 10], ["spain", "REST Countries", 10]]</code> |
| treemap  | <code>{ id: "spain", parent: "", value: 10 }</code>                          |
| sunburst | <code>{ id: "spain-football", parent: "spain", value: 10 }</code>            |
| dumbbell | <code>{ name: "Spain", low: 2, high: 8 }</code>                              |
| bullet   | <code>{ y: 7, target: 9 }</code>                                             |
| variwide | <code>{ name: "Spain", y: 100, z: 4 }</code>                                 |
| lollipop | <code>{ name: "Spain", y: 47000000 }</code>                                  |

Frase para examen:

Si me piden cambiar una grafica, no empiezo por CSS. Empiezo por la estructura de datos que espera Highcharts. Despues cambio el `type`, los modulos si hacen falta, `series`, ejes, tooltip y destruccion de la grafica anterior.

## Checklist para que una grafica no falle

- Que los datos lleguen antes de pintar.
- Que exista el contenedor HTML.
- Que `await tick()` se use si el contenedor depende de estado Svelte.
- Que Highcharts y sus modulos esten cargados.
- Que `series.data` tenga la forma correcta.
- Que se destruya la grafica anterior con `chart?.destroy()`.
- Que no haya llamada a Highcharts durante SSR o antes de `onMount`.

## Manual para cambiar cualquier integracion externa

Capas obligatorias:

```
Backend proxy -> Service frontend -> Pantalla -> Widget/HTML -> Tests/manual check
```

Backend:

```
src/back/v1/citys-stats.js
```

Service:

```
frontend-group/src/services/citysStatsIntegrations.js
```

Pantalla:

```
frontend-group/src/routes/integrations/citys-stats/+page.svelte
```

Preguntas:

```
La API externa devuelve array u objeto?  
Tiene pais, ciudad o codigo ISO?  
Necesita normalizacion?  
Puede fallar por CORS si la llamo desde navegador?  
Que metricas quiero visualizar?  
Que pasa si no coincide ningun pais?  
Que widget Highcharts encaja mejor?
```

Regla:

En este proyecto, las integraciones LCC deben pasar por backend, no por llamadas directas desde navegador.

## Manual para cambiar tests

No cambies tests a ciegas. Primero decide:

```
El test falla porque el codigo esta mal?  
O falla porque he cambiado intencionadamente el contrato?
```

Si el contrato no cambio:

- Arregla codigo.

Si el contrato cambio:

- Actualiza tests.

Tipos de tests:

| Test           | Comprueba                                  |
|----------------|--------------------------------------------|
| Newman/Postman | API, status, body, contrato HTTP           |
| Playwright     | Pantalla, formularios, botones, navegacion |

Donde mirar:

```
tests/LCC/pruebas-lcc.json
tests/LCC/pruebas-lcc-v2.json
tests/LCC/e2e/citys-stats.spec.js
```

Regla:

Si cambias campos obligatorios, todos los POST y PUT de Newman tienen que cambiar.

## Manual de defensa despues de modificar algo

Cuando hagas una modificacion, explicala siempre con esta estructura:

1. Que he cambiado.
2. Por que lo he cambiado.
3. Que capa he tocado.
4. Que funcion concreta he modificado.
5. Que riesgo tenia.
6. Como lo he probado.

Ejemplo:

He anadido el filtro `min_population` a `citys-stats`. Lo he puesto en el handler `GET /api/v2/citys-stats` porque el filtrado pertenece al contrato de API. En frontend he anadido el input en `searchForm` y lo convierto en query desde `buildSearchQuery`. He validado que sea numerico para devolver `400` si llega mal. Lo he probado con `?min_population=25000000` y con un valor invalido.

## Guia para explicar codigo linea por linea

No leas literalmente cada simbolo. Explica por bloques.

Mal:

```
Aqui pone const, aqui pone Number, aqui pone if...
```

Bien:

```
Este bloque convierte el dato a numero.
Este bloque comprueba si es valido.
Este bloque responde error si no lo es.
Este bloque filtra los resultados si todo esta bien.
```

Ejemplo:

```
if (req.query.min_population !== undefined) {
  const value = Number(req.query.min_population);
  if (!Number.isFinite(value) || value < 0) {
    return res.status(400).json({ error: "Invalid query" });
  }
}
```

```
}  
result = result.filter(d => d.un_2025_population >= value);  
}
```

Explicacion defendible:

Primero compruebo si el usuario ha enviado el filtro. Como los query params llegan como texto, convierto a numero. Si no es numero valido o es negativo, respondo `400`. Si es valido, dejo solo las ciudades cuya poblacion es mayor o igual al minimo.

## Lo que nunca hay que olvidar al modificar

- Si cambias backend, reinicia servidor.
- Si cambias frontend y pruebas con `npm start`, ejecuta `npm run build`.
- Si cambias campos, busca todas las apariciones con `rg`.
- Si cambias API, revisa Newman.
- Si cambias pantalla, revisa Playwright.
- Si cambias integraciones, asume que alguna API externa puede fallar.
- Si cambias rutas, recuerda que Express tiene fallback para no API.
- Si cambias nombres publicos, puedes romper tests, enlaces y documentacion.
- Si cambias datos iniciales, borra la coleccion antes de recargar.
- Si cambias una grafica, revisa que el contenedor exista antes de llamar a Highcharts.

## 17. Flujos de ejecucion

### Arranque general

1. El usuario ejecuta `npm start`.
2. Node ejecuta `index.js`.
3. Express crea `app`.
4. Se activa CORS.
5. Se activa lectura JSON.
6. Se crean las bases NeDB.
7. Se registran APIs v1 y v2.
8. Se sirve `public`.
9. Se registra fallback para rutas no API.
10. El servidor escucha en `10000` o `process.env.PORT`.

### Abrir una pantalla

1. El usuario abre `/citys-stats`.
2. Express no lo trata como API.
3. Express devuelve `public/index.html`.

4. Svelte arranca desde `main.js`.
5. `App.svelte` resuelve `window.location.pathname`.
6. Se carga `routes/citys-stats/+page.svelte`.
7. `onMount` llama a `refreshList`.
8. La pantalla pide datos al backend.

## Crear ciudad

1. El usuario rellena ciudad, pais y poblacion.
2. Pulsa guardar.
3. `handleCreate` valida el formulario.
4. `createCityStat` hace `POST /api/v2/citys-stats`.
5. Express ejecuta `normalizeCityStat`.
6. El backend busca duplicados por `city + country`.
7. NeDB inserta el documento.
8. La API devuelve `201`.
9. El service devuelve datos al componente.
10. `refreshList` recarga la tabla.

## Buscar ciudad

1. El usuario rellena filtros.
2. `buildSearchQuery` construye query.
3. `getAllCityStats(query)` construye URL.
4. El backend aplica filtros, `q`, `sort`, `offset` y `limit`.
5. Devuelve una lista.
6. La pantalla muestra resultados y mensaje.

## Editar ciudad sin cambiar clave

1. El usuario pulsa editar.
2. `openEdit` navega a `/citys-stats/editar/:city/:country`.
3. La pantalla de edicion carga el registro.
4. El usuario cambia poblacion.
5. `handleUpdate` detecta que la clave sigue igual.
6. Llama a `updateCityStat`.
7. El backend hace `PUT`.
8. La pantalla muestra exito.

## Editar ciudad cambiando clave

1. El usuario cambia `city` o `country`.
2. `handleUpdate` detecta que ya no es el mismo recurso.
3. Crea el nuevo registro con `POST`.



4. Borra el antiguo con `DELETE`.
5. Reemplaza la URL con la nueva clave.

## Analytics grupal

1. El usuario abre `/analytics`.
2. La pantalla llama en paralelo a:
  - `getAllCityStats`
  - `getDisasters`
  - `getAllWineStats`
1. `buildMetrics` calcula registros e indicadores.
2. `loadHighcharts` carga la librería.
3. `renderChart` pinta columnas con dos ejes.

## Integraciones LCC

1. El usuario abre `/integrations/citys-stats`.
2. Se cargan países locales con `country-summaries`.
3. Se piden APIs externas mediante endpoints propios.
4. Se transforman datos a arrays y métricas.
5. Se pintan siete widgets Highcharts.

---

# 18. Codigos HTTP y contrato REST

| Codigo | Uso en este proyecto                                    |
|--------|---------------------------------------------------------|
| 200    | Lectura correcta o actualización con respuesta          |
| 201    | Recurso creado o datos iniciales insertados             |
| 204    | Borrado correcto sin cuerpo, usado en ciudades y vinos  |
| 400    | Body incorrecto, query inválida o URL/body no coinciden |
| 404    | Recurso concreto no encontrado                          |
| 405    | Método no permitido para esa URL                        |
| 409    | Duplicado o colección ya cargada según recurso          |
| 500    | Error interno de servidor o base de datos               |
| 502    | Error controlado al consultar API externa               |

Frases útiles:

El código HTTP forma parte del contrato de la API, no es un detalle decorativo.

No usamos URLs tipo `/crearCiudad` porque el metodo HTTP ya expresa la accion.

**POST** crea en la coleccion; **PUT** actualiza un recurso que ya existe.

## Contrato real de `citys-stats v2`

Esta es la tabla que conviene saberse para defensa LCC. Es la API principal del CRUD.

| Metodo | Ruta                                             | Que hace                                                                                                   | Respuestas esperadas                                                                                                              |
|--------|--------------------------------------------------|------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| GET    | <code>/api/v2/citys-stats/docs</code>            | Redirige a documentacion Postman                                                                           | <code>302</code> redireccion                                                                                                      |
| GET    | <code>/api/v2/citys-stats/loadInitialData</code> | Carga datos iniciales si la base esta vacia                                                                | <code>201</code> si inserta, <code>200</code> si ya habia datos, <code>500</code> si falla NeDB                                   |
| GET    | <code>/api/v2/citys-stats</code>                 | Lista coleccion con filtros, <code>q</code> , <code>sort</code> , <code>offset</code> , <code>limit</code> | <code>200</code> , <code>400</code> si query/sort/paginacion invalida, <code>500</code>                                           |
| GET    | <code>/api/v2/citys-stats/:city/:country</code>  | Devuelve un registro concreto                                                                              | <code>200</code> , <code>404</code> si no existe, <code>500</code>                                                                |
| POST   | <code>/api/v2/citys-stats</code>                 | Crea un registro nuevo                                                                                     | <code>201</code> , <code>400</code> body invalido, <code>409</code> duplicado, <code>500</code>                                   |
| POST   | <code>/api/v2/citys-stats/:city/:country</code>  | No permitido sobre recurso concreto                                                                        | <code>405</code>                                                                                                                  |
| PUT    | <code>/api/v2/citys-stats</code>                 | No permitido sobre coleccion completa                                                                      | <code>405</code>                                                                                                                  |
| PUT    | <code>/api/v2/citys-stats/:city/:country</code>  | Actualiza un registro concreto                                                                             | <code>200</code> , <code>400</code> body invalido, <code>400</code> si URL/body no coinciden, <code>404</code> , <code>500</code> |
| DELETE | <code>/api/v2/citys-stats</code>                 | Borra toda la coleccion                                                                                    | <code>204</code> , <code>500</code>                                                                                               |
| DELETE | <code>/api/v2/citys-stats/:city/:country</code>  | Borra un registro concreto                                                                                 | <code>204</code> , <code>404</code> , <code>500</code>                                                                            |

Frase perfecta:

En REST, la ruta identifica el recurso y el metodo identifica la accion. Por eso `GET /api/v2/citys-stats` lee la coleccion, `POST /api/v2/citys-stats` crea en la coleccion, `PUT /api/v2/citys-stats/:city/:country` actualiza un registro concreto y `DELETE` borra.

## Orden interno del `GET /api/v2/citys-stats`

El handler de listado aplica las operaciones en este orden:

1. `db.find({})` lee todos los documentos.
2. `removeDatabaseId` quita `_id` de NeDB.
3. Filtro exacto por `city`.

```

4. Filtro exacto por country.
5. Filtro exacto por un_2025_population.
6. Búsqueda libre q en city o country.
7. sort por city, country o un_2025_population.
8. offset y limit para paginacion.
9. res.status(200).json(...)

```

Por que ese orden:

Primero se obtiene una coleccion limpia, despues se reduce con filtros y busqueda, luego se ordena y al final se pagina. Si paginara antes de filtrar, podria perder resultados correctos.

Ejemplos:

```

GET /api/v2/citys-stats
GET /api/v2/citys-stats?country=china
GET /api/v2/citys-stats?q=india
GET /api/v2/citys-stats?sort=-un_2025_population&limit=5
GET /api/v2/citys-stats?offset=5&limit=5

```

## Diferencia entre **POST** y **PUT**

**POST**:

```
POST /api/v2/citys-stats
```

Se usa sobre la coleccion porque el recurso todavia no existe. El backend decide si se puede crear.

Orden:

```

1. normalizeCityStat(req.body)
2. Si body invalido -> 400
3. db.findOne busca duplicado por city + country
4. Si ya existe -> 409
5. db.insert crea
6. Devuelve 201 con el recurso creado

```

**PUT**:

```
PUT /api/v2/citys-stats/:city/:country
```

Se usa sobre un recurso concreto porque el recurso ya existe o deberia existir.

Orden:

```

1. Lee city y country de req.params.
2. normalizeCityStat(req.body).
3. Si body invalido -> 400.
4. Si body.city/body.country no coinciden con URL -> 400.
5. db.findOne comprueba si existe.
6. Si no existe -> 404.

```

```
7. db.update actualiza.  
8. db.findOne vuelve a leer actualizado.  
9. Devuelve 200 con el recurso actualizado.
```

Frase perfecta:

**POST** no necesita identificador en la URL porque esta creando dentro de la coleccion. **PUT** si lo necesita porque modifica un recurso concreto y por eso compruebo que URL y body coincidan.

## Codigos que mas preguntan

### 400 Bad Request :

El cliente ha mandado mal la peticion: body con estructura incorrecta, query invalida, sort no permitido, offset/limit invalidos o URL/body que no coinciden.

### 404 Not Found :

La ruta existe, pero el recurso concreto no esta en la base de datos.

### 405 Method Not Allowed :

La URL existe pero ese metodo no tiene sentido ahi. Por ejemplo, **PUT /api/v2/citys-stats** no se permite porque **PUT** debe ir contra un recurso concreto.

### 409 Conflict :

La peticion esta bien formada, pero choca con el estado actual. En **POST**, ocurre si ya existe la misma combinacion **city + country**.

### 500 Internal Server Error :

Error interno de servidor o base de datos. No es culpa del cliente.

### 502 Bad Gateway :

Se usa en integraciones cuando nuestro backend actua como proxy y falla una API externa.

## Contrato de integraciones LCC

Las integraciones estan bajo:

```
/api/v1/citys-stats/integrations/...
```

Endpoints principales:

| Ruta                                               | Fuente            | Codigos habituales |
|----------------------------------------------------|-------------------|--------------------|
| <code>/integrations/geocoding/:city</code>         | Open-Meteo        | 200, 404, 502      |
| <code>/integrations/country/:country</code>        | REST Countries    | 200, 404, 502      |
| <code>/integrations/world-bank/:countryCode</code> | World Bank        | 200, 404, 502      |
| <code>/integrations/sos-tourist-arrivals</code>    | API SOS externa   | 200, 502           |
| <code>/integrations/sos-earthquakes</code>         | API SOS externa   | 200, 502           |
| <code>/integrations/sos-fifa-squad-values</code>   | API SOS externa   | 200, 502           |
| <code>/integrations/sos-esports-earnings</code>    | API SOS externa   | 200, 502           |
| <code>/integrations/summary?limit=5</code>         | Resumen combinado | 200, 400, 500      |

Frase de defensa:

En integraciones mi backend no es la fuente original de todos los datos, sino un proxy controlador. Por eso si una API externa falla, respondo con errores controlados como 502 o con errores parciales dentro del resumen.

## 19. Cambios que pueden pedir en directo

### Cambiar puerto

Archivo:

```
index.js
```

Pasos:

1. Buscar `const port = process.env.PORT || 10000`.
2. Cambiar 10000 si se pide otro puerto local.
3. Arrancar con `npm.cmd start`.
4. Ver consola.

Frase:

En despliegue manda `process.env.PORT`; el 10000 es el valor por defecto local.

### Cambiar donde se guarda una base

Archivo:

```
index.js
```

Buscar:

- `naturalDisastersDb`
- `citysStatsDb`
- `wineStatsDb`

Cambiar:

```
filename: path.join(__dirname, "src", "back", "citys-stats.db")
```

## Cambiar datos iniciales

Archivos:

```
src/back/v1/natural-disasters.js  
src/back/v2/natural-disasters.js  
src/back/v1/citys-stats.js  
src/back/v2/citys-stats.js  
src/back/v1/wine-stats.js
```

Pasos generales:

1. Cambiar `initialData`.
2. Respetar estructura exacta.
3. Borrar coleccion.
4. Cargar datos iniciales.
5. Ejecutar test del recurso.

Ciudades:

```
Invoke-RestMethod -Method Delete http://localhost:10000/api/v2/citys-stats  
Invoke-RestMethod http://localhost:10000/api/v2/citys-stats/loadInitialData
```

## Anadir campo a `citys-stats`

Tocar:

```
src/back/v1/citys-stats.js  
src/back/v2/citys-stats.js  
frontend-group/src/routes/citys-stats/+page.svelte  
frontend-group/src/routes/citys-stats/editar/[city]/[country]/+page.svelte  
tests/LCC/pruebas-lcc*.json  
tests/LCC/e2e/citys-stats.spec.js
```

Pasos:

1. Anadir campo en `hasExactCityFields`.
2. Limpiar y validar en `normalizeCityStat`.
3. Anadirlo a `initialData`.

4. Anadir input en formulario de crear.
5. Anadir input en formulario de editar.
6. Anadir columna en tabla.
7. Revisar analytics si usa el campo.
8. Actualizar tests.

Frase:

Como la API valida estructura exacta, no basta con anadir un input. Hay que aceptar el campo en backend, normalizarlo, mostrarlo y probarlo.

## Renombrar campo

Ejemplo:

```
un_2025_population -> population_2025
```

Pasos:

1. Ejecutar `rg "un_2025_population"`.
2. Cambiar backend v1 y v2.
3. Cambiar `initialData`.
4. Cambiar services si construyen payloads.
5. Cambiar formularios y tablas.
6. Cambiar analytics, mapa e integraciones.
7. Cambiar tests.
8. Ejecutar `rg "un_2025_population"` otra vez.

## Anadir filtro

Ciudades:

1. Abrir `src/back/v2/citys-stats.js`.
2. Tocar `app.get(BASE_API_URL, ...)`.
3. Validar query param.
4. Abrir `frontend-group/src/routes/citys-stats/+page.svelte`.
5. Anadir campo a `emptySearchForm`.
6. Anadir input.
7. Anadirlo en `buildSearchQuery`.
8. Probar URL.

Vinos:

1. Abrir `src/back/v1/wine-stats.js`.
2. Si texto, tocar `TEXT_FILTER_FIELDS`.
3. Si numero, tocar `NUMBER_FILTER_FIELDS` y `applyNumberFilters`.

4. Tocar pantalla de vinos si debe verse.

## Anadir ordenacion

Ciudades ya tiene patron:

```
?sort=campo  
?sort=-campo
```

Archivo:

```
src/back/v2/citys-stats.js
```

Buscar:

```
allowedFields
```

Anadir el campo permitido. No dejar ordenacion por campos arbitrarios.

## Crear una ruta nueva de API

Pasos:

1. Elegir recurso y version.
2. Abrir archivo de API.
3. Poner ruta concreta antes de rutas parametrizadas.
4. Usar metodo correcto (**GET**, **POST**, **PUT**, **DELETE**).
5. Quitar **\_id** si devuelve documentos.
6. Crear service si la usa frontend.
7. Pintarla si hace falta.
8. Crear o adaptar tests.

Ejemplo:

```
app.get(`${BASE_API_URL}/count`, (req, res) => {  
  db.count({}, (err, count) => {  
    if (err) return res.sendStatus(500);  
    return res.status(200).json({ count });  
  });  
});
```

Importante:

En Express importa el orden: **/count** debe ir antes de **/:city/:country**.

## Crear v3

Pasos:



1. Crear `src/back/v3`.
2. Copiar como base v2 del recurso.
3. Cambiar `BASE_API_URL` a `/api/v3/...`.
4. Implementar diferencias.
5. Importar en `index.js`.
6. Registrar con `app` y la base correcta.
7. Crear tests v3.
8. Actualizar enlaces si procede.

## Cambiar una pantalla o ruta frontend

Rutas actuales se crean por carpetas:

```
frontend-group/src/routes/nueva-ruta/+page.svelte
```

Si hay parametros:

```
frontend-group/src/routes/recurso/editar/[id]/+page.svelte
```

No hace falta registrar manualmente en `App.svelte`, porque se detecta con `import.meta.glob`.

Si quieres enlace visible:

```
frontend-group/src/components/Navbar.svelte
```

## Cambiar menu

Archivo:

```
frontend-group/src/components/Navbar.svelte
```

Los enlaces actuales son rutas directas:

```
/
/wine-stats
/citys-stats
/natural-disasters
/analytics
/integrations
```

## Cambiar portada

Archivo:

```
frontend-group/src/routes/+page.svelte
```

Cuidado:

- Playwright LCC revisa `data-testid` de la tarjeta LCC.
- No cambiar `member-citys-stats`, `frontend-citys-stats`, `api-v1-citys-stats`, etc. sin actualizar tests.

## Cambiar grafica Highcharts

Archivos:

```
frontend-group/src/routes/analytics/+page.svelte
frontend-group/src/routes/analytics/citys-stats/+page.svelte
frontend-group/src/routes/analytics/citys-stats/map/+page.svelte
frontend-group/src/routes/integrations/citys-stats/+page.svelte
```

Patron:

1. Cargar datos.
2. Transformar datos.
3. Cargar Highcharts.
4. Renderizar.
5. Destruir grafica en `onDestroy`.

## Anadir ciudad al mapa

Archivo:

```
frontend-group/src/routes/analytics/citys-stats/map/+page.svelte
```

Objeto:

```
coordinates
```

Clave:

```
city|country
```

Ejemplo:

```
"madrid|spain": { lat: 40.4168, lon: -3.7038 }
```

---

## 20. Errores comunes y soluciones

### `npm.ps1` no se puede cargar

Causa:

PowerShell bloquea scripts.

Solucion:

```
npm.cmd install
npm.cmd run build
npm.cmd start
```

## npm start falla

Comprobar:

- Estas en la raiz del proyecto.
- Ejecutaste `npm install`.
- El puerto `10000` no esta ocupado.
- No hay error de sintaxis en el ultimo cambio.

## La web se ve antigua

Causa:

`npm start` sirve `public`, no el codigo fuente de `frontend-group/src`.

Solucion:

```
npm.cmd run build
npm.cmd start
```

## Puerto ocupado

Soluciones:

- Cerrar proceso que usa `10000`.
- Cambiar `PORT`.
- Cambiar temporalmente el valor por defecto en `index.js`.

## loadInitialData dice que ya hay datos

Causa:

La base `.db` no esta vacia.

Solucion:

- Borrar desde la interfaz.
- O hacer `DELETE` a la coleccion y despues cargar iniciales.

## API devuelve 400

Puede ser:

- Body incompleto.
- Campo extra.

- Numero no valido.
- Query invalida.
- URL y body no coinciden en `PUT`.

Mirar:

- `normalize...`
- `hasExact...`
- `hasValid...`
- `buildSearchQuery`

## API devuelve `409`

Significa conflicto.

Ejemplos:

- Ciudad duplicada por `city + country`.
- Vino duplicado por `title + year`.
- Coleccion de vinos ya cargada.

## API devuelve `404`

Significa recurso no encontrado.

Comprobar:

- Identificador correcto.
- Texto normalizado.
- Recurso no borrado previamente.

## API devuelve `405`

Significa metodo no permitido.

Ejemplos:

```
POST /api/v2/citys-stats/tokyo/japan
PUT /api/v2/citys-stats
```

Frase:

No es un fallo, es parte del contrato REST.

## Integraciones fallan

Puede pasar por:

- API externa dormida.
- Timeout.

- Error en formato JSON externo.
- Problema temporal de red.

Mirar:

- `fetchJson`
- `safeExternal`
- `integrationErrors`

Frase:

Si una fuente externa falla, el backend conserva el resto y comunica el fallo como error parcial.

---

## 21. Decisiones tecnicas

### Express como servidor unico

Decision:

Usar un unico servidor Express para API y frontend compilado.

Ventaja:

- Facil despliegue en Render.
- Misma origin para frontend y API en produccion.
- Fallback sencillo para rutas SPA.

### NeDB como persistencia

Decision:

Usar NeDB con un archivo `.db` por recurso.

Ventaja:

- Sencillo para la practica.
- No requiere servidor de base de datos externo.
- Documentos JSON faciles de entender.

Limitacion:

- No es una base preparada para alta concurrencia o produccion real a gran escala.

### Versionado v1/v2

Decision:

Mantener v1 y v2 conviviendo.

Ventaja:

- No se rompe el contrato anterior.
- Se pueden añadir mejoras.
- El frontend puede elegir la version que necesita.

## Validacion estricta

Decision:

Rechazar campos extra o cuerpos incompletos.

Ventaja:

- Contrato claro.
- Tests mas predecibles.
- Evita datos basura.

## Proxy propio para integraciones

Decision:

El navegador no llama directamente a APIs externas en LCC; llama a nuestro backend.

Ventaja:

- Evita problemas de Same Origin Policy.
- Centraliza timeout y errores.
- Normaliza formatos antes de pintar.

## Rutas por carpetas en frontend

Decision:

Usar `+page.svelte` y `import.meta.glob` para descubrir pantallas.

Ventaja:

- Anadir ruta suele ser crear carpeta y archivo.
- Las rutas directas funcionan con fallback Express.

---

## 22. Limitaciones y mejoras futuras

Limitaciones actuales:

- NeDB es local y sencillo, no una base empresarial.
- Las integraciones externas dependen de servicios de terceros y Render puede estar dormido.
- Algunas APIs externas pueden cambiar formato.
- Algunas partes de otros recursos tienen estilos y tests menos homogeneos que LCC.
- No hay autenticacion de usuarios.
- No hay migraciones de esquema para cambios grandes.

Mejoras futuras:

- Migrar persistencia a PostgreSQL, MongoDB o similar.
- Anadir tests e2e homogeneos para todos los recursos.
- Anadir cache persistente para integraciones externas.
- Anadir observabilidad: logs estructurados y metricas.
- Mejorar validacion con un schema formal como JSON Schema o Zod.
- Crear una API v3 si se necesitan cambios incompatibles.
- Unificar criterios visuales entre todas las pantallas.

Comprobaciones externas no verificables desde el codigo:

- Videos personales, si la entrega los exige.
- Informes Toggl/efforts, si la entrega los exige.
- PR formal asociado a issue done y milestone/release, si la entrega lo pide.

Estos puntos no se pueden confirmar mirando solo el repositorio local, asi que no deben presentarse como errores del proyecto ni como funcionalidades pendientes del codigo.

---

## 23. Guion de defensa oral

Esta seccion esta pensada para estudiar y tambien para usarla como guion real el dia de la defensa. La idea no es leer palabra por palabra como si fuera un texto memorizado, sino entender el orden, repetir las frases clave y saber que pantalla o archivo abrir en cada momento.

Regla de oro:

```
problema -> arquitectura -> API -> frontend -> funciones -> integraciones -> pruebas -> cierre
```

Si te pierdes durante la defensa, vuelve a esta frase:

Mi proyecto no es solo una tabla. Es una cadena completa: el usuario actua en Svelte, el frontend llama con `fetch`, Express valida la peticion, NeDB guarda o consulta datos, y la respuesta se transforma en CRUD, graficas, mapa e integraciones.

## Preparacion antes de entrar

Tener abiertas estas pestanas en el navegador:

1. <https://sos2526-29.onrender.com/>
2. <https://sos2526-29.onrender.com/citys-stats>
3. <https://sos2526-29.onrender.com/analytics/citys-stats>
4. <https://sos2526-29.onrender.com/analytics/citys-stats/map>
5. <https://sos2526-29.onrender.com/integrations/citys-stats>
6. <https://sos2526-29.onrender.com/analytics>

Tener abiertos estos archivos en el editor:

1. `index.js`
2. `src/back/v2/citys-stats.js`
3. `src/back/v1/citys-stats.js`
4. `frontend-group/src/services/citysStatsApi.js`
5. `frontend-group/src/routes/citys-stats/+page.svelte`
6. `frontend-group/src/routes/analytics/citys-stats/+page.svelte`
7. `frontend-group/src/routes/analytics/citys-stats/map/+page.svelte`
8. `frontend-group/src/routes/integrations/citys-stats/+page.svelte`
9. `frontend-group/src/routes/analytics/+page.svelte`

Comandos preparados por si piden ejecutar en local:

```
npm.cmd install
npm.cmd --prefix frontend-group install
npm.cmd run build
npm.cmd start
```

Pruebas preparadas por si piden validar:

```
npm.cmd run test-LCC-v2
npm.cmd run test-LCC-e2e
```

## Guion de 30 segundos

Usalo si el profesor pide una explicacion muy breve.

Nuestro proyecto es una aplicacion web con backend Express, base de datos NeDB y frontend Svelte. El grupo trabaja con tres recursos: vinos, desastres naturales y estadisticas de ciudades. Mi parte es `citys-stats`, una API REST para gestionar ciudades, paises y poblacion estimada en 2025. He implementado CRUD, validacion estricta, busqueda, filtros, ordenacion, paginacion, analytics, mapa e integraciones externas. La defensa importante es que el dato recorre todo el sistema: se crea o consulta en la interfaz, pasa por un service con `fetch`, llega a Express, se valida, se guarda o lee en NeDB y vuelve al frontend convertido en tabla, grafico, mapa o widget.

## Guion de 90 segundos

Usalo cuando haya poco tiempo, pero quieras cubrir todo lo importante.

1. Abrir `index.js`.
2. Decir:

Aqui arranca el servidor. Se crea la aplicacion Express, se activan middlewares como `cors` y `express.json`, se inicializan las bases NeDB de cada recurso, se registran las APIs y se sirve el frontend compilado desde `public`.



1. Abrir `src/back/v2/citys-stats.js`.

2. Decir:

Este es el contrato REST principal de mi recurso. Aqui estan `GET`, `POST`, `PUT` y `DELETE`. Tambien estan la busqueda libre con `q`, los filtros exactos, la ordenacion, la paginacion y la validacion estricta para que no entren campos incorrectos.

1. Abrir `frontend-group/src/services/citysStatsApi.js`.

2. Decir:

Este service separa la interfaz de la API. Las pantallas no construyen todas las peticiones a mano, sino que llaman a funciones como `getAllCityStats`, `createCityStat`, `updateCityStat` o `deleteCityStat`.

1. Abrir `frontend-group/src/routes/citys-stats/+page.svelte`.

2. Decir:

Esta es la pantalla CRUD. Aqui se cargan datos, se busca, se crea, se edita y se borra. Los eventos del usuario acaban llamando al service y despues se refresca la lista.

1. Abrir `src/back/v1/citys-stats.js`.

2. Decir:

Aqui estan las integraciones externas. He decidido hacerlas desde el backend para no depender directamente del navegador, controlar errores y normalizar respuestas distintas.

1. Abrir `frontend-group/src/routes/integrations/citys-stats/+page.svelte`.

2. Decir:

Esta vista consume los endpoints proxy y transforma los datos externos en widgets y graficas. Si una API externa falla, la pantalla puede seguir mostrando el resto.

1. Abrir `frontend-group/src/routes/analytics/+page.svelte`.

2. Decir:

Aqui esta el trabajo grupal: un widget comun que combina los tres recursos sin mezclar unidades directamente, usando metricas normalizadas.

## Guion principal de 8 a 10 minutos

Este es el guion recomendado si la defensa permite explicar con calma.

### Minuto 0: apertura

Pantalla:

```
/
```

Frase:

Buenos dias. Somos el grupo SOS2526-29. La aplicacion permite consultar y visualizar datos de tres recursos: `wine-stats`, `natural-disasters` y `citys-stats`. Mi parte es `citys-stats`, que trabaja con ciudades, paises y poblacion estimada para 2025. Voy a explicar mi recurso de punta a punta: API, base de datos, frontend, graficas, mapa, integraciones y pruebas.

Idea que debe quedar clara:

No presento solo una pagina visual. Presento una aplicacion completa con backend, persistencia, contrato REST y frontend conectado.

## Minuto 1: arquitectura general

Archivo:

```
index.js
```

Frase:

El servidor se centraliza en `index.js`. Aqui Express recibe peticiones HTTP, interpreta JSON con `express.json`, permite llamadas desde el frontend con CORS, inicializa las bases NeDB y registra las rutas de cada recurso. Despues, para produccion, sirve el frontend compilado desde `public`.

Explicacion sencilla:

Express es la entrada del backend. NeDB guarda los datos. Svelte es la parte que ve el usuario. El navegador no toca la base de datos directamente; siempre pasa por la API.

Si preguntan por el flujo:

```
Usuario -> Svelte -> service fetch -> Express -> NeDB -> Express -> service -> Svelte
```

Frase para defender:

Esta separacion es importante porque cada capa tiene una responsabilidad concreta. La pantalla no valida como servidor, el servidor no dibuja graficas y la base de datos no sabe nada de botones.

## Minuto 2: API REST principal

Archivo:

```
src/back/v2/citys-stats.js
```

Pantalla o URL:

```
/api/v2/citys-stats
```

Frase:

La API principal de mi recurso esta en `/api/v2/citys-stats`. Uso REST porque el recurso es `citys-stats` y las operaciones se expresan con metodos HTTP: `GET` para leer, `POST` para crear, `PUT` para actualizar y `DELETE` para borrar.

Explicacion para alguien sin base:

Una API REST es como un mostrador ordenado. La URL dice sobre que datos hablamos y el metodo dice que queremos hacer con esos datos.

Ejemplos que se pueden decir:

```
GET /api/v2/citys-stats
POST /api/v2/citys-stats
PUT /api/v2/citys-stats/madrid/spain
DELETE /api/v2/citys-stats/madrid/spain
```

Frase clave:

En mi caso la clave del recurso no es un `id`, sino la combinacion `city + country`, porque una ciudad siempre se entiende dentro de un pais y ese par identifica el registro.

## Minuto 3: validacion y reglas de datos

Archivo:

```
src/back/v2/citys-stats.js
```

Funciones que conviene senalar:

```
hasExactCityFields
normalizeCityStat
```

Frase:

Antes de guardar datos, la API comprueba que el objeto tenga exactamente los campos esperados y que los valores sean coherentes. Esto evita que entren registros con campos de mas, campos ausentes o tipos incorrectos.

Explicacion sencilla:

La validacion es como una puerta de entrada. Si el dato no tiene la forma correcta, no pasa a la base de datos.

Ejemplo de defensa:

Si envio un campo extra, la API responde **400**. Si intento crear una ciudad que ya existe para el mismo pais, responde **409**. Si intento actualizar una ciudad inexistente, responde **404**.

Frase para funciones:

**hasExactCityFields** comprueba la estructura. **normalizeCityStat** limpia y transforma datos para que la base guarde una version uniforme.

## Minuto 4: CRUD en frontend

Pantalla:

```
/citys-stats
```

Archivos:

```
frontend-group/src/services/citysStatsApi.js  
frontend-group/src/routes/citys-stats/+page.svelte
```

Frase:

La pantalla CRUD no llama directamente a rutas escritas por todas partes. Para eso existe el service **citysStatsApi.js**, que agrupa las llamadas **fetch**. La pantalla Svelte se centra en estado, formularios, eventos y renderizado.

Demo recomendada:

1. Pulsar cargar datos iniciales si la base esta vacia.
2. Mostrar la tabla.
3. Buscar un pais o ciudad.
4. Ordenar por poblacion descendente.
5. Crear un registro sencillo.
6. Editarlo.
7. Borrarlo.

Frase durante la demo:

Al crear, editar o borrar no modifico la tabla a mano como si fuera una maqueta. Llamo a la API real y despues refresco la lista con los datos que devuelve el backend.

Si preguntan por funciones:

`refreshList` obtiene los datos actuales; `buildSearchQuery` prepara los parametros de busqueda; `validateCityStatForm` evita enviar formularios claramente incorrectos; `handleCreate` envia el `POST`; `openEdit` carga un registro en el formulario de edicion; `handleUpdate` envia el `PUT`.

## Minuto 5: busqueda, filtros, ordenacion y paginacion

Pantalla:

```
/citys-stats
```

URLs de ejemplo:

```
/api/v2/citys-stats?q=india  
/api/v2/citys-stats?country=china  
/api/v2/citys-stats?sort=-un_2025_population&limit=5
```

Frase:

La v2 no se queda en listar todo. Permite consultar mejor: busqueda libre con `q`, filtros exactos por campo, ordenacion con `sort` y paginacion con `limit` y `offset`.

Explicacion sencilla:

Esto es lo que convierte una API basica en una API util. Si los datos crecen, no quieres traer todo siempre; quieres filtrar, ordenar y pedir solo una parte.

Frase de defensa:

La busqueda y los filtros se expresan como query params porque no cambian el recurso, solo cambian la vista de los datos que estoy pidiendo.

## Minuto 6: analytics individual

Pantalla:

```
/analytics/citys-stats
```

Archivo:

```
frontend-group/src/routes/analytics/citys-stats/+page.svelte
```

Frase:

Esta pantalla cumple la visualizacion individual del recurso. Toma los datos reales de `citys-stats`, calcula metricas y los representa con Highcharts. No uso un grafico lineal simple, sino una visualizacion distinta para

resumir poblacion por ciudad o pais.

Explicacion sencilla:

La grafica no inventa datos. Solo cambia la forma de mirarlos. La fuente sigue siendo la API.

Funciones importantes:

```
loadResource  
buildMetrics  
renderChart
```

Frase para funciones:

`loadResource` carga los datos; `buildMetrics` prepara numeros utiles; `renderChart` destruye y vuelve a crear la grafica cuando cambian los datos.

## Minuto 7: mapa

Pantalla:

```
/analytics/citys-stats/map
```

Archivo:

```
frontend-group/src/routes/analytics/citys-stats/map/+page.svelte
```

Frase:

El mapa anade una lectura geografica. Cada ciudad se coloca con coordenadas y se representa con una burbuja cuyo tamano o color depende de la poblacion.

Explicacion sencilla:

Una tabla te dice los numeros, pero un mapa te ayuda a entender distribucion y concentracion geografica.

Funciones importantes:

```
keyFor  
colorFor  
radiusFor  
renderMap
```

Frase para funciones:

`keyFor` identifica cada ciudad, `colorFor` decide el color, `radiusFor` calcula el tamaño y `renderMap` pinta o actualiza el mapa.

## Minuto 8: integraciones externas

Pantalla:

```
/integrations/citys-stats
```

Archivos:

```
src/back/v1/citys-stats.js  
frontend-group/src/routes/integrations/citys-stats/+page.svelte
```

Frase:

Las integraciones externas están pensadas como proxy propio. El frontend no llama directamente a todas las APIs externas. Llama a mi backend y mi backend consulta, normaliza y devuelve una respuesta controlada.

Explicación sencilla:

Cada API externa responde de una forma distinta. El proxy sirve para traducir todas esas respuestas a una forma más cómoda para mi frontend.

Funciones backend importantes:

```
fetchJson  
safeExternal  
buildIntegratedCityBase  
buildIntegratedCity  
buildCityCountrySummaries
```

Funciones frontend importantes:

```
safeLoad  
loadIntegrations  
renderIntegrationCharts  
destroyIntegrationCharts
```

Frase de defensa:

La decisión de integrar por país y no por ciudad se debe a que muchas APIs externas trabajan mejor con códigos de país, ISO3 o nombres de país. Por eso primero agrego mis ciudades por país y después cruzo con fuentes externas.

## Minuto 9: analytics grupal

Pantalla:

```
/analytics
```

Archivo:

```
frontend-group/src/routes/analytics/+page.svelte
```

Frase:

La vista grupal combina los tres recursos en un unico widget. Como cada recurso mide cosas distintas, no mezclamos unidades directamente; usamos una comparacion normalizada para que la grafica sea defendible.

Explicacion sencilla:

No tiene sentido sumar poblacion, numero de desastres y datos de vino como si fueran la misma unidad. Por eso se normaliza y se muestra una comparacion relativa.

Frase de defensa:

Aqui se ve el trabajo de grupo: tres APIs independientes conectadas en una visualizacion comun.

## Minuto 10: pruebas, calidad y cierre

Archivos o comandos:

```
tests/LCC/pruebas-lcc-v2.json  
tests/LCC/playwright.config.js  
npm.cmd run test-LCC-v2  
npm.cmd run test-LCC-e2e
```

Frase:

Para validar que no es solo visual, hay pruebas de API con Newman/Postman y pruebas e2e con Playwright. Las primeras verifican endpoints, codigos HTTP y respuestas. Las segundas simulan acciones reales en navegador.

Cierre:

En resumen, mi parte cubre el ciclo completo: modelo de datos, API REST versionada, validacion, persistencia, CRUD, busqueda, filtros, ordenacion, paginacion, analytics, mapa, integraciones externas, proxy propio y pruebas. Lo importante es que cada pantalla se puede explicar desde el codigo y cada funcion tiene una responsabilidad concreta.



## Demo exacta recomendada

Orden ideal para enseñar en directo:

1. Abrir `/`.
2. Entrar en `/citys-stats`.
3. Pulsar `Load initial data` si hace falta.
4. Buscar `india` o `china`.
5. Ordenar por poblacion descendente.
6. Crear una ciudad de prueba.
7. Editar la ciudad creada.
8. Borrar la ciudad creada.
9. Abrir `/api/v2/citys-stats?sort=-un_2025_population&limit=5`.
10. Abrir `/analytics/citys-stats`.
11. Abrir `/analytics/citys-stats/map`.
12. Abrir `/integrations/citys-stats`.
13. Abrir `/analytics`.
14. Volver al codigo y explicar una funcion concreta si lo piden.

Frase para acompañar la demo:

Voy a hacer una operacion real. Si creo un registro, pasa por el formulario, el service, la API, la validacion y la base de datos. Despues la tabla se refresca pidiendo de nuevo al backend.

## Guion para explicar una funcion cualquiera

Cuando el profesor senale una funcion, usa siempre este orden:

1. Decir en que archivo esta.
2. Decir quien la llama.
3. Decir que recibe.
4. Decir que comprueba.
5. Decir que transforma.
6. Decir que devuelve.
7. Decir que error evita.

Plantilla:

Esta funcion esta en `[archivo]`. La llama `[otra funcion o pantalla]` cuando `[situacion]`. Recibe `[parametros]`. Primero valida o prepara `[dato]`, despues transforma `[dato]` y finalmente devuelve `[resultado]`. Es importante porque evita `[error]` y mantiene separada la responsabilidad de `[capa]`.

Ejemplo con `normalizeCityStat`:

`normalizeCityStat` esta en el backend de `citys-stats`. Se usa antes de guardar o devolver datos. Recibe un objeto de ciudad, limpia campos de texto y convierte la poblacion a numero. Es importante porque evita

guardar datos con formatos inconsistentes.

Ejemplo con `refreshList`:

`refreshList` esta en la pantalla CRUD. La llama la pantalla al cargar, buscar, crear, editar o borrar. Prepara la consulta, llama al service y actualiza el estado visible. Es importante porque centraliza la recarga de datos y evita repetir la misma llamada en muchos sitios.

Ejemplo con `safeExternal`:

`safeExternal` esta en el backend de integraciones. Envuelve llamadas a APIs externas para que un fallo externo no rompa toda la respuesta. Es importante porque en integraciones reales no controlamos la disponibilidad de terceros.

## Frases perfectas para momentos difíciles

Si preguntan "por que v1 y v2":

Porque versionar permite evolucionar la API sin romper clientes anteriores. En mi caso, v2 concentra el CRUD mejorado y v1 mantiene endpoints anteriores e integraciones.

Si preguntan "por que el frontend llama a v1 en integraciones":

Porque esas rutas concretas de integracion estan implementadas bajo `/api/v1/citys-stats/integrations`. La v2 se usa como contrato CRUD principal.

Si preguntan "por que `citys` y no `cities`":

Porque `citys-stats` ya es el contrato publico del proyecto. Aunque linguisticamente `cities` seria mejor, cambiar la ruta ahora romperia tests, enlaces y documentacion.

Si preguntan "por que NeDB":

Porque encaja con una practica academica: persistencia local, documentos JSON, instalacion sencilla y sin depender de un servidor de base de datos externo.

Si preguntan "por que no se llama directamente a APIs externas desde Svelte":

Porque el backend puede controlar errores, evitar problemas de CORS, ocultar detalles de integracion y normalizar respuestas antes de que lleguen al frontend.

Si preguntan "que pasa si algo falla":

El proyecto intenta fallar de forma controlada. En API se devuelven codigos HTTP coherentes. En integraciones se capturan fallos externos. En frontend se muestran mensajes y se evita que toda la pantalla

quede rota.

## Plan B si falla algo en directo

Si Render tarda en despertar:

Render puede dormir la aplicacion cuando lleva tiempo sin uso. Mientras arranca, puedo explicar el codigo y despues volver a la pestana.

Si falla una API externa:

Esta es precisamente una razon para usar proxy y gestion de errores. Las integraciones dependen de servicios externos; el proyecto captura el fallo y puede seguir mostrando el resto.

Si la base ya tiene datos y `loadInitialData` no carga:

El endpoint evita duplicar datos iniciales. Si la base no esta vacia, responde indicando que ya hay datos.

Si una demo de crear ciudad da conflicto:

Eso significa que ya existe el par `city + country`. La API devuelve `409`, que es el codigo correcto para conflicto de recurso.

Si preguntan algo que no recuerdas:

Lo miraria desde el flujo. Primero localizo la pantalla, despues el service que llama al backend y por ultimo el endpoint de Express que resuelve la peticion.

## Cierre recomendable

La idea importante es que no hay una pantalla aislada: hay una cadena completa. Svelte recoge una accion, el service hace `fetch`, Express valida, NeDB guarda o consulta, y el frontend transforma la respuesta en tabla, formulario, grafica, mapa o integracion. Por eso el proyecto se puede defender desde usuario, desde API y desde codigo.

---

## 24. Preguntas dificiles y respuestas

### Por que REST y no rutas tipo `/crearCiudad`

Porque REST identifica recursos con URLs y usa metodos HTTP para las acciones. La URL dice "que recurso" y el metodo dice "que accion".

## Por que **POST** crea en la coleccion

Porque antes de crear no existe la URL del recurso concreto. La coleccion es donde nace el nuevo recurso.

## Por que **PUT** lleva identificador en URL

Porque actualiza un recurso existente. Ademas se comprueba que URL y body coincidan para evitar modificar otro registro por error.

## Por que se usan query params

Porque filtros, busqueda, orden y paginacion no cambian el recurso base; solo cambian la vista que se devuelve.

## Por que NeDB

Porque para esta practica necesitamos persistencia sencilla en archivos. NeDB permite documentos JSON sin instalar una base externa. Cada recurso tiene su propio `.db`.

## Por que se quita `_id`

Porque `_id` es interno de NeDB. La API publica no debe depender de detalles de almacenamiento.

## Por que hay v1 y v2

Porque v1 mantiene compatibilidad y v2 anade mejoras sin romper clientes previos. En LCC, v2 se usa para CRUD y v1 aloja integraciones.

## Por que algunas rutas usan v1 en frontend

Porque `citysStatsIntegrations.js` llama a integraciones que estan implementadas en `/api/v1/citys-stats/integrations/...`.

## Por que `citys-stats` esta escrito asi

Porque es el contrato publico usado por rutas, tests y documentacion. Cambiarlo romperia enlaces y entregables.

## Por que claves compuestas

Porque no todos los datasets tienen `id` natural. Desastres usa `country + year`; ciudades usa `city + country`; vinos genera `id`.

## Que pasa si mando un campo de mas

La API responde `400`, porque la validacion exige estructura exacta.

## Por que **DELETE** devuelve a veces `204`

Porque el borrado fue correcto y no hace falta devolver cuerpo.

## Que hace CORS

Permite que el frontend en desarrollo, que puede estar en otro puerto, llame al backend sin bloqueo del navegador.

## Como se despliega el frontend

Vite compila `frontend-group/src` hacia `public`. Express sirve `public` con `express.static`.

## Por que funcionan rutas directas como `/analytics`

Porque Express devuelve `index.html` para rutas que no son `/api`, y Svelte resuelve la pantalla por `window.location.pathname`.

## Donde esta el proxy propio

En `src/back/v1/citys-stats.js`, bajo `/api/v1/citys-stats/integrations/...`.

## Que pasa si una API externa falla

`fetchJson` y `safeExternal` controlan el error. La vista puede mostrar datos parciales e informar avisos.

## Por que no se muestra JSON crudo

Porque JSON es el formato de intercambio. En la interfaz se transforma en widgets, tablas, tarjetas y metricas.

## Como se relacionan datos locales y externos

Primero se agregan ciudades por pais. Despues se normalizan nombres de pais e ISO3 para cruzarlos con las fuentes externas.

## Donde se ve el trabajo de grupo

En los tres recursos, la portada, las APIs, tests, `/analytics` y `/integrations`.

## 25. Glosario para personas no tecnicas

| Termino     | Explicacion                                                               |
|-------------|---------------------------------------------------------------------------|
| Backend     | Parte del sistema que recibe peticiones, valida datos y habla con la base |
| Frontend    | Parte visible que usa el usuario en el navegador                          |
| API         | Forma ordenada de pedir o enviar datos a un servidor                      |
| REST        | Estilo de API basado en recursos, URLs y metodos HTTP                     |
| Recurso     | Tipo de dato gestionado, por ejemplo <code>citys-stats</code>             |
| CRUD        | Crear, leer, actualizar y borrar                                          |
| JSON        | Formato de texto para intercambiar datos                                  |
| NeDB        | Base de datos local que guarda documentos en archivos                     |
| Endpoint    | Ruta concreta de una API                                                  |
| Query param | Parametro de URL como <code>?limit=5</code>                               |
| Proxy       | Servidor intermedio que llama a otra API por nosotros                     |
| SPA         | Aplicacion web que carga una vez y cambia pantallas con JavaScript        |
| Build       | Version compilada y lista para servir                                     |
| Test e2e    | Prueba que simula acciones reales en navegador                            |
| Newman      | Herramienta para ejecutar colecciones Postman                             |
| Highcharts  | Libreria para graficas y mapas                                            |

## 26. Checklists finales

### Antes de defender

```
npm.cmd install
npm.cmd --prefix frontend-group install
npm.cmd run build
npm.cmd start
```

Abrir:

- `http://localhost:10000/`
- `http://localhost:10000/citys-stats`
- `http://localhost:10000/analytics`
- `http://localhost:10000/analytics/citys-stats`
- `http://localhost:10000/analytics/citys-stats/map`

- `http://localhost:10000/integrations`
- `http://localhost:10000/integrations/citys-stats`
- `http://localhost:10000/api/v2/citys-stats`
- `http://localhost:10000/api/v1/citys-stats/integrations/summary?limit=8`

## Tests utiles

```
npm.cmd run test-LCC-v2
npm.cmd run test-LCC-e2e
npm.cmd run test-ALG
npm.cmd run test-RMP
```

## Despues de tocar backend

- Reiniciar `npm start`.
- Probar endpoint directo.
- Ejecutar Newman del recurso.

## Despues de tocar frontend

- Ejecutar `npm.cmd run build` si se va a probar con `npm start`.
- Abrir ruta afectada.
- Revisar consola del navegador.
- Ejecutar Playwright si afecta a flujo visible.

## Despues de tocar campos

- Ejecutar `ng "nombre_del_campo"`.
- Actualizar backend, frontend, analytics, integraciones y tests.
- Probar crear y editar.

## Checklist LCC D03

- CRUD v2 accesible.
- Documentacion Postman v2 enlazada.
- Carga inicial disponible.
- Búsqueda libre `q`.
- Filtros exactos.
- Ordenacion `sort`.
- Paginacion `limit` y `offset`.
- Grafico individual no `line`.
- Mapa geoespacial.
- Integraciones por pais.
- 7 APIs integradas.
- 3 APIs no SOS.

- 4 APIs SOS externas.
- Proxy propio.
- Vista `/integrations`.
- Widget grupal unico en `/analytics`.

## 27. Anexos tecnicos

### Comandos API rapidos para `citys-stats`

```
Invoke-RestMethod http://localhost:10000/api/v2/citys-stats
Invoke-RestMethod http://localhost:10000/api/v2/citys-stats/loadInitialData
Invoke-RestMethod "http://localhost:10000/api/v2/citys-stats?q=india"
Invoke-RestMethod "http://localhost:10000/api/v2/citys-stats?sort=-un_2025_population&limit=5"
```

Crear ciudad:

```
Invoke-RestMethod -Method Post `
  -Uri http://localhost:10000/api/v2/citys-stats `
  -ContentType "application/json" `
  -Body '{"city":"madrid","country":"spain","un_2025_population":7000000}'
```

Borrar todo:

```
Invoke-RestMethod -Method Delete http://localhost:10000/api/v2/citys-stats
```

### Comandos API rapidos para `natural-disasters`

```
Invoke-RestMethod http://localhost:10000/api/v2/natural-disasters
Invoke-RestMethod "http://localhost:10000/api/v2/natural-disasters?country=spa"
Invoke-RestMethod "http://localhost:10000/api/v2/natural-disasters?from=1990&to=2010"
```

### Comandos API rapidos para `wine-stats`

```
Invoke-RestMethod http://localhost:10000/api/v1/wine-stats
Invoke-RestMethod "http://localhost:10000/api/v1/wine-stats?country=spain"
Invoke-RestMethod "http://localhost:10000/api/v1/wine-stats?year=2026"
```

### Rutas de documentacion Postman

```
/api/v1/wine-stats/docs
/api/v1/citys-stats/docs
/api/v2/citys-stats/docs
/api/v1/natural-disasters/docs
/api/v2/natural-disasters/docs
```



## Regla de oro para cambios

Si cambia una API:

```
backend -> service -> pantalla -> tests -> build
```

Si cambia solo una grafica:

```
pantalla analytics/integrations -> renderChart/renderMap -> probar ruta
```

Si cambia un campo:

```
backend + initialData + frontend + analytics + tests
```

## Estado final de docs

Los documentos antiguos de defensa dentro de `docs/` ya no deben usarse para estudiar, preparar cambios ni defender el proyecto. Se eliminan como fuentes independientes para evitar duplicar informacion y para que nadie tenga que buscar explicaciones repartidas en varios sitios.

Las unicas referencias activas dentro de `docs/` deben ser:

```
docs/DEFENSA_COMPLETA.md  
docs/DEFENSA_COMPLETA.pdf
```

Si estas leyendo este archivo desde la carpeta `docs/`, abre la copia PDF con el enlace relativo [DEFENSA\\_COMPLETA.pdf](#).